

OS1 kolokvijumi

kategorisani zadaci sa kolokvijuma 2006-2023 sa stranice predmeta

pripremili Luka Simić i Aleksa Marković

Jul 2024

Sadržaj

Predgovor	4
Prekidi	5
2. zadatak, drugi kolokvijum, maj 2023.	5
2. zadatak, drugi kolokvijum, maj 2022.	6
1. zadatak, kolokvijum, jun 2020.	6
2. zadatak, prvi kolokvijum, mart 2019.	8
2. zadatak, prvi kolokvijum, maj 2019.	9
2. zadatak, prvi kolokvijum, mart 2018.	10
2. zadatak, prvi kolokvijum, april 2018.	11
2. zadatak, prvi kolokvijum, jun 2018.	13
2. zadatak, prvi kolokvijum, mart 2017.	14
2. zadatak, prvi kolokvijum, mart 2016.	15
2. zadatak, prvi kolokvijum, mart 2015.	16
2. zadatak, prvi kolokvijum, maj 2015.	17
3. zadatak, prvi kolokvijum, mart 2014.	18
3. zadatak, prvi kolokvijum, april 2014.	19
3. zadatak, prvi kolokvijum, septembar 2014.	19
3. zadatak, prvi kolokvijum, mart 2013.	20
3. zadatak, prvi kolokvijum, april 2013.	21
3. zadatak, prvi kolokvijum, septembar 2011.	22
4. zadatak, prvi kolokvijum, april 2010.	23
Sistemske pozive	25
1. zadatak, drugi kolokvijum, april 2024.	25
1. zadatak, drugi kolokvijum, maj 2023.	25
1. zadatak, drugi kolokvijum, avgust 2023.	26
1. zadatak, drugi kolokvijum, maj 2022.	27
1. zadatak, drugi kolokvijum, jun 2022.	28
1. zadatak, kolokvijum, oktobar 2020.	29
1. zadatak, kolokvijum, jul 2020.	30
4. zadatak, kolokvijum, avgust 2020.	31
2. zadatak, kolokvijum, septembar 2020.	32
3. zadatak, prvi kolokvijum, mart 2019.	33
3. zadatak, prvi kolokvijum, maj 2019.	34
3. zadatak, prvi kolokvijum, jun 2019.	35
3. zadatak, prvi kolokvijum, mart 2018.	36
3. zadatak, prvi kolokvijum, april 2018.	37
3. zadatak, prvi kolokvijum, jun 2018.	38
3. zadatak, prvi kolokvijum, mart 2017.	39
3. zadatak, prvi kolokvijum, mart 2016.	40

3. zadatak, prvi kolokvijum, maj 2016.	41
3. zadatak, prvi kolokvijum, septembar 2015.	41
4. zadatak, prvi kolokvijum, mart 2014.	42
4. zadatak, prvi kolokvijum, mart 2013.	43
4. zadatak, prvi kolokvijum, april 2013.	44
4. zadatak, prvi kolokvijum, septembar 2013.	45
4. zadatak, prvi kolokvijum, mart 2012.	46
4. zadatak, prvi kolokvijum, maj 2012.	47
4. zadatak, prvi kolokvijum, septembar 2012.	48
4. zadatak, prvi kolokvijum, maj 2011.	48
4. zadatak, prvi kolokvijum, septembar 2011.	49
4. zadatak, prvi kolokvijum, april 2009.	50
5. zadatak, prvi kolokvijum, april 2009.	51
4. zadatak, prvi kolokvijum, april 2008.	52
5. zadatak, prvi kolokvijum, april 2007.	52
6. zadatak, prvi kolokvijum, april 2006.	53

Interfejs niti **55**

2. zadatak, kolokvijum, avgust 2021.	55
3. zadatak, prvi kolokvijum, april 2017.	56
3. zadatak, prvi kolokvijum, jun 2017.	56
3. zadatak, prvi kolokvijum, septembar 2016.	57
3. zadatak, prvi kolokvijum, mart 2015.	58
3. zadatak, prvi kolokvijum, maj 2015.	59
4. zadatak, prvi kolokvijum, septembar 2014.	60
3. zadatak, prvi kolokvijum, septembar 2013.	61
2. zadatak, drugi kolokvijum, maj 2012.	62
4. zadatak, prvi kolokvijum, april 2011.	62
5. zadatak, prvi kolokvijum, april 2010.	63

Promena konteksta **66**

1. zadatak, kolokvijum, septembar 2020.	66
2. zadatak, prvi kolokvijum, jun 2019.	67
2. zadatak, prvi kolokvijum, april 2017.	68
2. zadatak, prvi kolokvijum, jun 2017.	69
2. zadatak, prvi kolokvijum, maj 2016.	70
2. zadatak, prvi kolokvijum, septembar 2015.	71
4. zadatak, prvi kolokvijum, april 2014.	72
3. zadatak, prvi kolokvijum, maj 2012.	73
3. zadatak, prvi kolokvijum, septembar 2012.	74
3. zadatak, prvi kolokvijum, april 2010.	75
5. zadatak, prvi kolokvijum, april 2008.	75
4. zadatak, prvi kolokvijum, april 2007.	77
5. zadatak, prvi kolokvijum, april 2006.	79
1. zadatak, drugi kolokvijum, maj 2006.	80

Sinhronizacija procesa (implementacija) **83**

2. zadatak, drugi kolokvijum, april 2024.	83
3. zadatak, drugi kolokvijum, maj 2023.	84
2. zadatak, drugi kolokvijum, avgust 2023.	85
3. zadatak, drugi kolokvijum, avgust 2023.	86
2. zadatak, drugi kolokvijum, jun 2022.	87
2. zadatak, kolokvijum, jun 2021.	88
1. zadatak, kolokvijum, avgust 2020.	89
1. zadatak, drugi kolokvijum, maj 2019.	90
1. zadatak, drugi kolokvijum, april 2017.	91
1. zadatak, drugi kolokvijum, jun 2017.	92

1. zadatak, drugi kolokvijum, maj 2016.	93
2. zadatak, prvi kolokvijum, septembar 2016.	94
1. zadatak, drugi kolokvijum, septembar 2015.	95
1. zadatak, drugi kolokvijum, april 2014.	96
1. zadatak, drugi kolokvijum, septembar 2014.	97
2. zadatak, drugi kolokvijum, april 2013.	98
2. zadatak, drugi kolokvijum, septembar 2013.	98
2. zadatak, drugi kolokvijum, septembar 2012.	99
2. zadatak, drugi kolokvijum, maj 2011.	100
2. zadatak, drugi kolokvijum, septembar 2011.	100
1. zadatak, drugi kolokvijum, maj 2010.	100
1. zadatak, drugi kolokvijum, maj 2009.	101
1. zadatak, drugi kolokvijum, maj 2008.	102
1. zadatak, drugi kolokvijum, maj 2007.	103
Sinhronizacija procesa (interfejs)	105
3. zadatak, drugi kolokvijum, april 2024.	105
3. zadatak, drugi kolokvijum, jun 2022.	105
2. zadatak, kolokvijum, jul 2021.	106
2. zadatak, kolokvijum, jun 2020.	107
1. zadatak, drugi kolokvijum, jun 2019.	108
1. zadatak, drugi kolokvijum, april 2018.	109
1. zadatak, drugi kolokvijum, jun 2018.	110
1. zadatak, treći kolokvijum, jun 2016.	112
1. zadatak, drugi kolokvijum, septembar 2016.	113
1. zadatak, drugi kolokvijum, maj 2015.	114
1. zadatak, drugi kolokvijum, april 2013.	115
1. zadatak, drugi kolokvijum, septembar 2013.	116
1. zadatak, drugi kolokvijum, maj 2012.	116
1. zadatak, drugi kolokvijum, septembar 2012.	117
1. zadatak, drugi kolokvijum, maj 2011.	118
1. zadatak, drugi kolokvijum, septembar 2011.	118
2. zadatak, drugi kolokvijum, maj 2010.	119
2. zadatak, drugi kolokvijum, maj 2009.	120
2. zadatak, drugi kolokvijum, maj 2006.	121
Baferi, proizvođač/potrošač	123
3. zadatak, drugi kolokvijum, maj 2022.	123
3. zadatak, kolokvijum, jul 2020.	124
1. zadatak, treći kolokvijum, septembar 2013.	125
1. zadatak, treći kolokvijum, septembar 2012.	127
1. zadatak, treći kolokvijum, jun 2011.	128
2. zadatak, drugi kolokvijum, maj 2008.	129
2. zadatak, drugi kolokvijum, maj 2007.	132

Predgovor

Svrha ove zbirke jeste da objedini sve do sada dostupne kolokvijume iz Operativnih sistema 1 iz nekoliko razloga:

- Na kolokvijumima iz OS1 je dozvoljena literatura, i studenti često pristupaju tome tako što odštampaju kolokvijume i njihova rešenja pa se na kolokvijumu snalaze kroz taj odštampani materijal ako vide neki sličan zadatak. Ovom zbirkom postiže se organizacija takvih materijala radi lakšeg snalaženja na kolokvijumu i ušteda u papirima potrebnim za štampanje svih tih rokova.
- Zadaci su kategorisani po oblastima i sličnosti kako bi se lakše vežbali određeni tipovi zadataka na kolokvijumu.

Poglavlja zbirke idu istim redom kao kolokvijumsko gradivo na predmetu, konkretno:

1. Kompajler/linker, stranična organizacija, kontinualna alokacija, segmentna organizacija, segmentno-stranična organizacija, dinamičko učitavanje, preklopi
2. Prekidi, sistemski pozivi, interfejs niti, promena konteksta, sinhronizacija procesa, baferi, proizvođač/potrošač
3. Ulaz/izlaz, komandna linija, fajl sistem

Raspored ovih oblasti po kolokvijumima se retko menja, ali se takva situacija može desiti (usled predviđenih ili nepredviđenih okolnosti). U tom slučaju, proverite koje oblasti dolaze na kolokvijumu sa predmetnim asistentima. Bitno je napomenuti da se ovaj redosled u prošlosti menjao, pa ako se neki zadatak ranije pojavljivao na prvom kolokvijumu to ne mora da znači da je ta oblast i tekuće godine na prvom kolokvijumu. U suštini, **da li neki zadatak može doći na kolokvijumu određujete prvenstveno na osnovu toga iz koje je oblasti, a ne na osnovu toga na kom se kolokvijumu pojavio u prošlosti.**

Greške u formatiranju i kategorizaciji su sigurno prisutne. Ukoliko ih uočite, možete se javiti jednom od autora ili poslati *pull request* na repozitorijum projekta: <https://github.com/siwiki/OS-kolokvijumi>. Ispravljeni dokumenti će biti dostupni u [Releases](#). Svaka pomoć je dobrodošla.

Srećno na kolokvijumu.

Autori

Prekidi

2. zadatak, drugi kolokvijum, maj 2023.

- [Postavka](#)
- [Rešenje](#)

Neki procesor pri obradi prekida, sistemskog poziva i izuzetka prelazi na sistemski stek. Taj stek alocirani je u delu memorije koju koristi kernel, a na vrh tog steka ukazuje poseban registar SSP procesora koji je dostupan samo u privilegovanom režimu.

Prilikom obrade ovih situacija procesor ništa ne stavlja na stek, već zatečenu, neizmenjenu vrednost registra PC kog je koristio prekinuti proces sačuva u poseban, za to namenjen registar SPC (procesor nema PSW), a vrednosti registara SP i SSP međusobno zameni (*swap*). Za pristup steku procesor tako uvek koristi SP. Prilikom povratka iz prekidne rutine instrukcijom `iret` procesor radi inverznu operaciju. Procesor je RISC, sa *load/store* arhitekturom i ima 32 registra opšte namene (R0..R31).

Kernel je višenitni, a svakom toku kontrole (procesu ili niti), uključujući i niti kernela, pridružen je poseban stek koji se koristi u sistemskom režimu. Promenu konteksta u kernelu radi data operacija `yield`. Kontekst procesora kernel čuva na steku, dok se informacija o vrhu steka čuva u polju `PCB::sp` čiji je pomeraj u odnosu na početak strukture PCB u assembleru označen istoimenom simboličkom konstantom. U kodu kernela postoji statički pokazivač `oldRunning` koji ukazuje na PCB tekućeg procesa, kao i pokazivač `newRunning` koji ukazuje na PCB procesa koji je izabran za izvršavanje.

Napisati kod funkcije `initContext` koju koristi kernel kada inicijalizuje procesorski kontekst za proces sa datim PCB-om tako da ovaj proces može dobiti procesor funkcijom `yield`. Na vrh već alociraniog steka procesa (prvu praznu lokaciju) koji se koristi u neprivilegovanom režimu ukazuje `usrStk`, dok na vrh već alociraniog steka procesa koji se koristi u kernelu ukazuje `krnlStk`, a početna adresa programa je `startAddr`. Stek raste ka nižim adresama. Svi pokazivači su 32-bitni, a tip `uint32` predstavlja 32-bitni ceo neoznačen broj.

```
void yield () {
    asm {
        ; Save the current context
        push spc ; save regs on the process stack
        push ssp
        push r0
        push r1
        ...
        push r31
        load r0, oldRunning ; r0 now points to the running PCB
        store sp, [r0+PCB::sp] ; save SP

        ; Restore the new context
        load r0, newRunning
        load sp, [r0+PCB::sp] ; restore SP
        pop r31 ; restore regs
        ...
        pop r0
        pop ssp
        pop spc
    }
}
```

```
void initContext (PCB* pcb, void* usrStk, void* krnlStk, void* startAddr);
```

Rešenje

```
void initContext(PCB* pcb, void* usrStk, void* krnlStk, void* startAddr) {
    uint32* kst = (uint32*)krnlStk;
    *(kst-0) = (uint32)startAddr;
    *(kst-1) = (uint32)usrStk;
}
```

```
    pcb->sp = kst-34;
}
```

2. zadatak, drugi kolokvijum, maj 2022.

- [Postavka](#)
- [Rešenje](#)

Neki procesor pri obradi prekida, sistemskog poziva i izuzetka prelazi na sistemski stek. Taj stek alociran je u delu memorije koju koristi kernel, a na vrh tog steka ukazuje poseban registar SSP procesora koji je dostupan samo u privilegovanom režimu.

Prilikom obrade ovih situacija, procesor ništa ne stavlja na stek, već zatečene, neizmenjene vrednosti registara PC i PSW koje je koristio prekinuti proces sačuva u posebne, za to namenjene registre, SPC i SPSW, respektivno, a vrednosti registara SP i SSP međusobno zameni (*swap*). Prilikom povratka iz prekidne rutine instrukcijom *iret* procesor radi inverznu operaciju. Procesor je RISC, sa *load/store* arhitekturom i ima 32 registra opšte namene (R0..R31).

Kernel je višenitni, a svakom toku kontrole (procesu ili niti), uključujući i niti kernela, pridružen je poseban stek koji se koristi u sistemskom režimu. Prilikom promene konteksta, kontekst procesora treba sačuvati na tom steku, dok informaciju o vrhu steka treba čuvati u polju PCB čiji je pomeraj u odnosu na početak strukture PCB označen simboličkom konstantom *offsSP*.

U kodu kernela postoji statički pokazivač *oldRunning* koji ukazuje na PCB tekućeg procesa, kao i pokazivač *newRunning* koji ukazuje na PCB procesa koji je izabran za izvršavanje.

Napisati kod funkcije *yield* koju koristi kernel kada želi da promeni kontekst (prebaci se sa izvršavanja jednog toka kontrole, *oldRunning*, na drugi, *newRunning*), na bilo kom mestu gde se za to odluči.

Rešenje

```
void yield() {
    asm {
        ; Save the current context
        push spc ; save regs on the process stack
        push ssp
        push spsw
        push r0
        push r1
        ...
        push r31
        load r0, oldRunning ; r0 now points to the running PCB
        store sp, [r0 + offsSP] ; save SP
        ; Restore the new context
        load r0, newRunning
        load sp, [r0 + offsSP] ; restore SP
        pop r31 ; restore regs
        ...
        pop r0
        pop spsw
        pop ssp
        pop spc
    }
}
```

1. zadatak, kolokvijum, jun 2020.

- [Postavka](#)
- [Rešenje](#)

Neki procesor obrađuje prekide (hardverske i softverske) tako što tokom izvršavanja prekidne rutine koristi poseban stek koji se koristi u sistemskom, privilegovanom režimu rada procesora, u kome se izvršava kod kernela (čiji je deo i prekidna rutina). Taj stek alociran je u delu memorije koju koristi kernel, a na vrh tog steka ukazuje poseban registar SSP procesora koji je dostupan samo u privilegovanom režimu.

Prilikom obrade prekida, procesor ništa ne stavlja na stek, pa tako ni ne menja sadržaj pokazivača vrha korisničkog steka (SP) koji je koristio prekinuti proces (SP zadržava staru vrednost), dok tekuću (staru) vrednost statusnog registra (PSW) i programskog brojača (PC) sačuva u posebne, za to namenjene registre, SPSW i SPC, respektivno. Prilikom povratka iz prekidne rutine instrukcijom `iret`, procesor restaurira registre PSW i PC prepisujući vrednosti iz registara SPSW i SPC i vraća se u korisnički režim, a time i na korisnički stek.

Ovaj isti sistemski stek se koristi tokom izvršavanja bilo kog koda kernela; kernel ima samo jedan takav stek (nema više niti). Prilikom promene konteksta, kontekst procesora treba sačuvati u odgovarajućim poljima strukture PCB, u kojoj postoji polje za čuvanje svakog od programski dostupnih registara; pomeraj ovog polja u odnosu na početak strukture PCB označen je simboličkim konstantama `offsPC`, `offsSP`, `offsPSW`, `offsR0`, `offsR1` itd. Procesor je RISC, sa *load/store* arhitekturom i ima 32 registra opšte namene (R0..R31).

U kodu kernela postoji statički definisan pokazivač `running` koji ukazuje na PCB tekućeg procesa. Potprogram `s_call`, koji nema argumente, realizuje sistemski poziv zadat od strane korisničkog procesa vrednostima u registrima i po potrebi raspoređivanje, tako što smešta PCB na koji ukazuje pokazivač `running` u listu spremnih procesa ili na drugo mesto, a iz liste spremnih uzima jedan odabrani proces i postavlja pokazivač `running` na njega. Na assembleru datog procesora napisati kod prekidne rutine `sys_call` koja vrši sistemski poziv korišćenjem potprograma `s_call`. Ova prekidna rutina poziva se softverskim prekidom iz korisničkog procesa, pri čemu se sistemski poziv realizuje softverskim prekidom.

Rešenje

```
sys_call: ; Save the current context
    push r0 ; save r0 temporarily on the (kernel) stack
    load r0, [running] ; r0 now points to the running PCB
    store r1, #offsR1[r0] ; save r1
    store r2, #offsR2[r0] ; save r2
    ... ; save r3-r30
    store r31, #offsR31[r0] ; save r31
    pop r1 ; save r0
    push r1
    store r1, #offsR0[r0]
    store SPC, #offsPC[r0] ; save PC
    store SPSW, #offsPSW[r0] ; save PSW
    store SP, #offsSP[r0] ; save SP
    load r1, #offsR1[r0] ; restore sys call args in r1
    pop r0 ; and r0

    ; Execute the system call
    call s_call

    ; Restore the new context
    load r0, [running]
    load SP, #offsSP[r0] ; restore SP
    load SPSW, #offsPSW[r0] ; restore PSW
    load SPC, #offsPC[r0] ; restore PC
    load r31, #offsR31[r0] ; restore R31
    ... ; restore r30-r2
    load r1, #offsR1[r0] ; restore R1
    load r0, #offsR0[r0] ; restore R0

    ; Return
    iret
```

2. zadatak, prvi kolokvijum, mart 2019.

- [Postavka](#)
- [Rešenje](#)

Neki procesor obrađuje prekide (hardverske i softverske) tako što tokom izvršavanja prekidne rutine koristi poseban stek koji se koristi u sistemskom, privilegovanom režimu rada procesora, u kome se izvršava kod kernela (čiji je deo i prekidna rutina). Taj stek alociran je u delu memorije koju koristi kernel, a na vrh tog steka ukazuje poseban registar SSP procesora koji je dostupan samo u privilegovanom režimu.

Prilikom obrade prekida, procesor na ovom steku čuva: pokazivač vrha korisničkog steka (SP) koji je koristio prekinuti proces, programsku statusnu reč (PSW) i programski brojač (PC), tim redom, ali *ne* i ostale programski dostupne registre. Prilikom povratka iz prekidne rutine instrukcijom `iret`, procesor restaurira ove registre sa sistemskog steka i vraća se u korisnički režim, a time i na korisnički stek.

Ovaj isti sistemski stek se koristi tokom izvršavanja bilo kog koda kernela; kernel ima samo jedan takav stek (nema više niti). Prilikom promene konteksta, programski dostupne registre treba sačuvati u odgovarajućim poljima strukture PCB, u kojoj postoji polje za čuvanje svakog od programski dostupnih registara; pomeraj ovog polja u odnosu na početak strukture PCB označen je simboličkim konstantama `offsPC`, `offsSP`, `offsPSW`, `offsR0`, `offsR1` itd.

Procesor je RISC, sa *load/store* arhitekturom i ima sledeće programski dostupne registre: 32 registra opšte namene (R0..R31), SP, PSW i PC.

U kodu kernela postoji statički definisan pokazivač `running` koji ukazuje na PCB tekućeg procesa. Potprogram `scheduler`, koji nema argumente, realizuje raspoređivanje, tako što smešta PCB na koji ukazuje pokazivač `running` u listu spremnih procesa, a iz nje uzima jedan odabrani proces i postavlja pokazivač `running` na njega.

Na assembleru datog procesora napisati kod prekidne rutine `dispatch` koja vrši promenu konteksta korišćenjem potprograma `scheduler`. Ova prekidna rutina poziva se sistemskim pozivom iz korisničkog procesa, pri čemu se sistemski poziv realizuje softverskim prekidom.

Rešenje

```
dispatch: ; Save the current context
    push r0 ; save r0 temporarily on the stack
    load r0, [running] ; r0 now points to the running PCB
    store r1, #offsR1[r0] ; save r1
    store r2, #offsR2[r0] ; save r2
    ... ; save r3-r30
    store r31, #offsR31[r0] ; save r31
    pop r1 ; save r0
    store r1, #offsR0[r0]
    pop r1 ; save PC
    store r1, #offsPC[r0]
    pop r1 ; save PSW
    store r1, #offsPSW[r0]
    pop r1 ; save SP
    store r1, #offsSP[r0]

    ; Select the next running process
    call scheduler

    ; Restore the new context
    load r0, [running]
    load r1, #offsSP[r0] ; restore SP
    push r1
    load r1, #offsPSW[r0] ; restore PSW
    push r1
    load r1, #offsPC[r0] ; restore PC
```



```

push r1
load r31, #offsR31[r0] ; restore R31
... ; restore r30-r2
load r1, #offsR1[r0] ; restore R1
load r0, #offsR0[r0] ; restore R0

; Return
iret

```

2. zadatak, prvi kolokvijum, maj 2019.

- [Postavka](#)
- [Rešenje](#)

Neki sistem čuva kontekst niti u strukturi PCB u kojoj postoje polja za čuvanje vrednosti svih programski dostupnih registara procesora: pokazivača na vrh steka niti (SP), programske statusne reči (PSW) i programskog brojača (PC), kao i registara opšte namene R0..R31. Adrese i svi registri su 32-bitni, pa su takvi i tipovi `int` i svi pokazivački tipovi na jeziku C.

Dole je dat segment asemblerskog koda kojim ovaj sistem restaurira kontekst niti nakon svakog sistemskog poziva. Simboličke konstante `pc`, `psw`, `sp` itd. predstavljaju pomeraje (engl. *offset*) istoimenih polja u strukturi PCB. Registar R0 prenosi vrednost koja se vraća iz sistemskog poziva, pa se zato ne restaurira.

```

; Restore the new context
load r1, [running]
load r2, #sp[r1] ; restore SP
push r2
load r2, #psw[r1] ; restore PSW
push r2
load r2, #pc[r1] ; restore PC
push r2
load r31, #r31[r1] ; restore R31
... ; restore r30-r2
load r1, #r1[r1] ; restore R1
; R0 holds the return value from the system call
; Return from system call (interrupt)
iret

```

Potrebno je dopuniti implementaciju operacije `createThread` kodom na mestu označenom sa `/**/`. Ovu funkciju poziva kernel u implementaciji sistemskog poziva za kreiranje nove niti, a kod koji nedostaje treba da formira inicijalni procesorski kontekst. Nit treba kreirati nad korisničkom funkcijom na koju ukazuje argument `pf`, tako da ta nit izvršava tu funkciju sa argumentom datim u `arg`. Nakon izvršavanja ove korisničke funkcije `pf`, nit treba da se ugasi sistemskim pozivom implementiranim u funkciji `exit`.

Funkcija `createPCB` alocira novu strukturu PCB i adresu te strukture upisuje u pokazivač na koga ukazuje argument. Funkcija `allocateStack` alocira stek za novu nit i adresu najviše slobodne lokacije steka upisuje u pokazivač na koga ukazuje argument. Stek raste ka nižim adresama, a pokazivač vrha steka ukazuje na prvu slobodnu lokaciju.

```

void exit (); // Terminate the calling thread
const int PSW_INIT = ...; // Initial value for PSW

int createThread (void (*pf) (void*), void* arg) {
    PCB* pcb;
    int stat = createPCB(&pcb);
    if (stat!=0) return stat;
    void** sp;
    stat = allocateStack(&sp);
    if (stat!=0) return stat;
    /**/
    pcb->psw = PSW_INIT;
}

```

```

    stat = Scheduler::put(pcb);
    return stat;
}

```

Rešenje

```

*sp-- = arg;
*sp-- = &exit;
pcb->sp = sp;
pcb->pc = pf;

```

2. zadatak, prvi kolokvijum, mart 2018.

- [Postavka](#)
- [Rešenje](#)

U implementaciji jezgra nekog jednog procesorskog *time-sharing* operativnog sistema, radi pojednostavljenja celog mehanizma promene konteksta, primenjeno je sledeće neobično rešenje. Promena konteksta vrši se isključivo kao posledica prekida, na samo jednom mestu u kodu koji se izvršava na prekid. Prekidi dolaze od raznih uređaja, u najmanju ruku od vremenskog brojača, jer on u svakom slučaju generiše prekid zbog toga što je tekućoj niti isteklo dodeljeno procesorsko vreme. Zbog toga tekuća nit nikada ne gubi procesor sinhrono, čak ni kada poziva blokirajuću operaciju (sistemska poziv). Umesto toga, ukoliko je potrebno da se nit suspenduje (blokira) u nekom blokirajućem pozivu, nit se samo „označi“ suspendovanom i nastavlja sa izvršavanjem (uposlenim čekanjem) sve dok ne stigne sledeći prekid. Kada takav prekid stigne, prekidna rutina vrši samu promenu konteksta.

Na primer, implementacija operacije `suspend`, koja suspenduje pozivajući proces, i `resume`, koja ponovo deblokira dati proces, izgledaju ovako:

```

void suspend () {
    running->status = suspended; // Mark as suspended
    while (running->status==suspended); // and then busy-wait
    // until it is preempted, suspended, and then resumed later
}

void resume (int pid) {
    processes[pid].status = ready; // Mark as ready
}

```

Procesor je RISC sa *load/store* arhitekturom, ima 32 registra opšte namene i SP. Prilikom prekida na steku čuva samo PC i PSW. Tajmer se restartuje upisom odgovarajuće vrednosti u registar koji se nalazi na adresi simbolički označenoj sa `Timer`. PCB procesa je dat strukturom definisanom dole, a svi procesi zapisani su u nizu `processes`. U assembleru, simboličko ime polja strukture ima vrednost pomeraja (engl. *offset*) tog polja od početka strukture.

```

enum ProcessStatus { unused, initiating, terminating, ready, suspended };
typedef unsigned long Time;
typedef unsigned Register;

struct PCB {
    ProcessStatus status; // Process status
    Time timeSlice;       // Time slice for time sharing
    Register savedSP;      // Saved stack pointer
    ...
}

const unsigned long NumOfProcesses = ...;
PCB processes[NumOfProcesses];
PCB* running; // Running process

```

1. Na assembleru datog procesora napisati kod prekidne rutine koja vrši promenu konteksta. Ova prekidna rutina, pored čuvanja i restauracije konteksta na steku procesa, treba da pozove potprogram `scheduler` koji će u

pokazivač **running** smestiti vrednost koja ukazuje na novoizabrani tekući proces, i da tajmer restartuje sa vremenskim kvantom dodeljenim tom procesu. Pretpostavlja se da uvek postoji barem jedan spreman proces.

- Na jeziku C napisati potprogram **scheduler** koji bira sledeći proces za izvršavanje. Spremne procese treba da bira redom, u krug, a ne svaki put od početka niza **processes**.

Rešenje

```
dispatch: ; Save the current context
          push r0 ; save regs
          push r1
          ...
          push r31
          load r0, running
          store sp, #savedSP[r0] ; save sp

          ; Select the next running process
          call scheduler

          ; Restore the new context
          load r0, running
          load r1, #timeSlice[r0]; restart timer
          store r1, [Timer]
          load sp, #savedSP[r0] ; restore sp
          pop r31
          pop r30 ; restore regs
          ...
          pop r0

          ; Return
          iredt

void scheduler () {
    do {
        running = processes + (running - processes + 1) % NUM_OF_PROCESSES;
    } while (running->status!=ready);
}
```

2. zadatak, prvi kolokvijum, april 2018.

- [Postavka](#)
- [Rešenje](#)

Neki procesor obrađuje prekide (hardverske i softverske) tako što tokom izvršavanja prekidne rutine koristi poseban stek koji se koristi u sistemskom, privilegovanom režimu rada procesora, u kome se izvršava kod kernela (čiji je deo i prekidna rutina). Taj stek alociran je u delu memorije koju koristi kernel, a na vrh tog steka ukazuje poseban registar SSP procesora koji je dostupan samo u privilegovanom režimu.

Prilikom obrade prekida, procesor na ovom steku čuva programski brojač (PC) i programsku statusnu reč (PSW), tim redom, ali ne i ostale programski dostupne registre. Prilikom povratka iz prekidne rutine instrukcijom **iredt**, procesor restaurira ove registre sa sistemskog steka i vraća se u korisnički režim, a time i na korisnički stek.

Postoji samo jedan kernel stek koji se koristi za izvršavanje celog koda kernela. Svi potprogrami kernela pisani su tako da na ovom steku čuvaju (i sa njega potom restauriraju) sve registre procesora koje koriste. Za sve vreme izvršavanja kernel koda prekidi su maskirani (procesor ih implicitno maskira prilikom prihvatanja prekida, a kernel kod ih ne demaskira).

U strukturi PCB postoje polja za čuvanje vrednosti svih programski dostupnih registara procesora; pomenaj polja za neki registar R_i u odnosu na početak strukture PCB označen je simboličkom konstantom **offsRi** (npr. **offsSP**,

offsPC, offsPSW, offsR0, ..., offsR31).

Procesor je RISC, sa *load/store* arhitekturom i ima sledeće programski dostupne registre: 32 registra opšte namene (R0..R31), SP, PSW i PC. Svi registri su 32-bitni.

U kodu kernela postoji statički definisan pokazivač **running** koji ukazuje na PCB tekućeg procesa. Potprogram **sys_call_proc**, koji nema argumente, realizuje obradu sistemskog poziva, kao i raspoređivanje, tako što, nakon obrade samog sisteskog poziva, postavlja pokazivač **running** da ukazuje na PCB odabranog novog tekućeg procesa.

Na assembleru datog procesora napisati kod prekidne rutine **sys_call** za sistemske pozive softverskim prekidom, s tim da ona, zbog efikasnijeg rada, ne treba da vrši promenu konteksta (čuvanje i restauraciju registara) ako za tim nema potrebe, odnosno ukoliko procedura **sys_call_proc** nije promenila pokazivač **running**.

Rešenje

```
sys_call: ; Save r0 and r1 on the (kernel) stack
    push r0
    push r1
    load r0, [running] ; the old running is in r0

    ; Perform the system call:
    call sys_call_proc

    ; Compare the old and the new running,
    load r1, [running] ; the new running is in r1
    cmp r0, r1
    jne switch

    ; and do not switch the context if they are equal,
    ; but restore r0 and r1, and return
    pop r1
    pop r0
    iret

switch: ; Save the context of the old running
    store r2, #offsR2[r0] ; save other regs
    store r3, #offsR3[r0]
    ...
    store r31, #offsR31[r0]
    store sp, #offsSP[r0] ; save sp
    pop r1 ; save r1 through r1
    store r1, #offsR1[r0]
    pop r1 ; save r0 through r1
    store r1, #offsR0[r0]
    pop r1 ; save psw through r1
    store r1, #offsPSW[r0]
    pop r1 ; save pc through r1
    store r1, #offsPC[r0]

    ; Restore the context of the new running
    load r0, [running]
    load r1, #offsPC[r0] ; restore pc through the stack
    push r1
    load r1, #offsPSW[r0] ; restore psw through the stack
    push r1
    load sp, #offsSP[r0] ; restore sp
    load r31, #offsR31[r0] ; restore r31
    ... ; restore other regs
    load r1, #offsR1[r0]
```

```

load r0,#offsR0[r0] ; restore r0
; and return
iret

```

2. zadatak, prvi kolokvijum, jun 2018.

- Postavka
- Rešenje

Neki procesor obrađuje prekidne (hardverske i softverske) tako što tokom izvršavanja prekidne rutine koristi poseban stek koji se koristi u sistemskom, privilegovanom režimu rada procesora, u kome se izvršava kod kernela (čiji je deo i prekidna rutina). Taj stek alociran je u delu memorije koju koristi kernel, a na vrh tog steka ukazuje poseban registar SSP procesora koji je dostupan samo u privilegovanom režimu.

Prilikom obrade prekida, procesor na ovom steku čuva: pokazivač vrha korisničkog steka (SP) koji je koristio prekinuti proces, programsku statusnu reč (PSW) i programski brojač (PC), tim redom, ali ne i ostale programski dostupne registre. Prilikom povratka iz prekidne rutine instrukcijom `iret`, procesor restaurira ove registre sa sistemskog steka i vraća se u korisnički režim, a time i na korisnički stek.

Svaki proces ima sopstveni takav sistemski stek. Prilikom promene konteksta, ostale programski dostupne registre treba sačuvati na ovom sistemskom steku prekinutog procesa. U strukturi PCB postoji polje za čuvanje vrednosti SSP steka procesa; pomeraj ovog polja u odnosu na početak strukture PCB označen je simboličkom konstantom `offsSSP`.

Procesor je RISC, sa *load/store* arhitekturom i ima sledeće programski dostupne registre: 32 registra opšte namene (R0..R31), SP, PSW i PC.

Sistem je multiprocesorski. Poseban registar procesora RPID služi za identifikaciju tekućeg procesa koji taj procesor trenutno izvršava i dostupan je samo u privilegovanom režimu. Potprogram `scheduler`, koji nema argumente, realizuje raspoređivanje, tako što smešta adresu PCB onog procesa koji se raspoređuje na datom procesoru (na kome se izvršava kod) u registar RPID.

1. Na assembleru datog procesora napisati kod prekidne rutine `dispatch` koja vrši promenu konteksta korišćenjem potprograma `scheduler`. Ova prekidna rutina poziva se sistemskim pozivom iz korisničkog procesa, pri čemu se sistemski poziv realizuje softverskim prekidom.
2. Pod pretpostavkom da procedura `scheduler` obezbeđuje međusobno isključenje pristupa deljenim strukturama podataka (npr. redu spremnih procesa) od strane različitih procesora, da li i ostatak prekidne rutine `dispatch` treba da obezbedi isto, tj. da li je u nju potrebno ugraditi *spin lock*? Precizno obrazložiti odgovor.

Rešenje

```

1. dispatch:      ; Save the current context
push r0           ; save regs
push r1
...
push r31
store ssp, #offsSSP[rpid] ; save ssp

; Select the next running process and store its PCB* in rpid
call scheduler

; Restore the new context
load ssp, #offsSSP[rpid] ; restore ssp
pop r31
pop r30           ; restore regs
...
pop r0
; Return
iret

```

2. Ne treba. Ova prekidna rutina pristupa samo registrima procesora, strukturi PCB tekućeg procesa i njegovom sistemskom steku. Kako su sve tri stvari korišćene isključivo od strane tog procesora (jer je taj proces raspoređen samo tom procesoru), nema potrebe za međusobnim isključenjem.

2. zadatak, prvi kolokvijum, mart 2017.

- [Postavka](#)
- [Rešenje](#)

Neki procesor obrađuje prekide (hardverske i softverske) tako što tokom izvršavanja prekidne rutine koristi poseban stek koji se koristi u sistemskom, privilegovanom režimu rada procesora, u kome se izvršava kod kernela (čiji je deo i prekidna rutina). Taj stek alociran je u delu memorije koju koristi kernel, a na vrh tog steka ukazuje poseban registar SSP procesora koji je dostupan samo u privilegovanom režimu.

Prilikom obrade prekida, procesor na ovom steku čuva: pokazivač vrha korisničkog steka (SP) koji je koristio prekinuti proces, programsku statusnu reč (PSW) i programski brojač (PC), tim redom, ali ne i ostale programski dostupne registre. Prilikom povratka iz prekidne rutine instrukcijom `iret`, procesor restaurira ove registre sa sistemskog steka i vraća se u korisnički režim, a time i na korisnički stek.

Svaki proces ima takav sopstveni sistemski stek. Prilikom promene konteksta, ostale programski dostupne registre treba sačuvati na ovom sistemskom steku prekinutog procesa. U strukturi PCB postoji polje za čuvanje vrednosti SSP steka procesa; pomeraj ovog polja u odnosu na početak strukture PCB označen je simboličkom konstantom `offsSSP`.

Procesor je RISC, sa *load/store* arhitekturom i ima sledeće programski dostupne registre: 32 registra opšte namene (R0..R31), SP, PSW i PC.

U kodu kernela postoji statički definisan pokazivač `running` koji ukazuje na PCB tekućeg procesa. Potprogram `scheduler`, koji nema argumente, realizuje raspoređivanje, tako što smešta PCB na koji ukazuje pokazivač `running` u listu spremnih procesa, a iz nje uzima jedan odabrani proces i postavlja pokazivač `running` na njega.

Na assembleru datog procesora napisati kod prekidne rutine `dispatch` koja vrši promenu konteksta korišćenjem potprograma `scheduler`. Ova prekidna rutina poziva se sistemskim pozivom iz korisničkog procesa, pri čemu se sistemski poziv realizuje softverskim prekidom.

Rešenje

```
dispatch:    ; Save the current context
            push r0 ; save regs
            push r1
            ...
            push r31
            load r0, running
            store ssp, #offsSSP[r0] ; save ssp

            ; Select the next running process
            call scheduler

            ; Restore the new context
            load r0, running
            load ssp, #offsSSP[r0] ; restore ssp
            pop r31
            pop r30 ; restore regs
            ...
            pop r0
            ; Return
            ired
```

2. zadatak, prvi kolokvijum, mart 2016.

- [Postavka](#)
- [Rešenje](#)

Neki procesor obrađuje prekide (hardverske i softverske) u sistemskom, privilegovanom režimu rada procesora, u kome se izvršava kod kernela (čiji je deo i prekidna rutina), koristeći posebne registre SPC (*System Program Counter*), SPSW (*System Processor Status Word*) i SSP (*System Stack Pointer*) koji nisu dostupni u korisničkom (neprivilegovanom) režimu. Registre PC, PSW i SP, kao ni sve druge programski dostupne registre koji se koriste u neprivilegovanom režimu, procesor ne čuva nigde implicitno prilikom obrade prekida, jer se oni ni ne menjaju implicitno tokom skoka u prekidnu rutinu niti tokom izvršavanja te rutine (izvršavanje instrukcija u privilegovanom režimu koristi SPC, SPSW i SSP umesto PC, PSW i SP); njihove vrednosti se mogu menjati eksplicitno, uobičajenim instrukcijama u privilegovanom režimu (npr. `load` i `store`). Prema tome, izvršavanje u privilegovanom režimu koristi i poseban stek alociran u delu memorije koju koristi kernel. Taj stek je jedan i kernel ga ne menja.

Prilikom povratka iz prekidne rutine instrukcijom `iret` procesor ništa ne restaurira, samo se prebacuje na korišćenje registara PC, SP i PSW umesto njihovih sistemskih parnjaka SPC, SSP i SPSW.

Procesor je RISC, sa *load/store* arhitekturom i ima sledeće registre dostupne u neprivilegovanom režimu: 32 registra opšte namene (R0..R31), SP, PSW i PC. Poseban registar RX je dostupan samo u privilegovanom režimu rada procesora, pa ga korisnički procesi ne koriste i kernel ga može koristiti samo za svoje potrebe.

Registre dostupne u neprivilegovanom režimu treba sačuvati u odgovarajuća polja strukture PCB. U strukturi PCB postoje polja za čuvanje svih tih registara; pomerači ovih polja u odnosu na početak strukture PCB označeni su simboličkim konstantama `offsPC`, `offsPSW`, `offsSP`, `offsR0`, ..., `offsR31`.

U kodu kernela postoji statički definisan pokazivač `running` koji ukazuje na PCB tekućeg procesa. Potprogram `scheduler`, koji nema argumente, realizuje raspoređivanje, tako što smešta PCB na koji ukazuje pokazivač `running` u listu spremnih procesa, a iz nje uzima jedan odabrani proces i postavlja pokazivač `running` na njega.

Na assembleru datog procesora napisati kod prekidne rutine `dispatch` koja vrši promenu konteksta korišćenjem potprograma `scheduler`. Ova prekidna rutina poziva se sistemskim pozivom iz korisničkog procesa, pri čemu se sistemski poziv realizuje softverskim prekidom.

Rešenje

```
dispatch:    ; Save the current context
             load rx,running
             store r0,#offsR0[rx] ; save regs
             store r1,#offsR1[rx]
             ...
             store r31,#offsR31[rx]
             store pc,#offsPC[rx] ; save pc
             store psw,#offsPSW[rx] ; same psw
             store sp,#offsSP[rx] ; save sp

             ; Select the next running process
             call scheduler

             ; Restore the new context
             load rx,running
             load sp,#offsSP[rx] ; restore sp
             load psw,#offsPSW[rx] ; restore psw
             load pc,#offsPC[rx] ; restore pc
             load r0,#offsR0[rx] ; restore regs
             load r1,#offsR1[rx]
             ...
             load r31,#offsR31[rx]
             ; Return
             iret
```

2. zadatak, prvi kolokvijum, mart 2015.

- [Postavka](#)
- [Rešenje](#)

Neki procesor obrađuje prekide (hardverske i softverske) tako što tokom izvršavanja prekidne rutine koristi poseban stek koji se koristi u sistemskom. privilegovanom režimu rada procesora, u kome se izvršava kod kernela (čiji je deo i prekidna rutina). Taj stek alociran je u delu memorije koju koristi kernel, a na vrh tog steka ukazuje poseban registar procesora koji je dostupan samo u privilegovanom režimu. Taj stek je uvek isti i kernel ga ne menja.

Prilikom obrade prekida. procesor na ovom sistemskom steku čuva: pokazivač vrha korisničkog steka (SP) koji je koristio prekinuti program, programsku statusnu reg (PSW) i programski brojai (PC). tim redom, ali *ne* i ostale programski dostupne registre. Prilikom povratka iz prekidne rutine instrukcijom `iret`, procesor restaurira ove registre sa sistemskog steka i vraća se u korisnički režim, a time i na korisnički stek.

Procesor je RISC. sa *load/store* arhitekturom i ima sledeće programski dostupne registre: 32 registra opšte namene (R0..R31). SP. PSW i PC. Poseban registar RX je dostupan samo u privilegovanom režimu rada procesora. pa ga korisnički procesi ne koriste i kernel ga može koristiti samo za svoje potrebe.

Registre PC. SP. PSW i R0..R31 treba sačuvati u odgovarajuća polja strukture PCB. U strukturi PCB postoje polja za čuvanje svih tih registara; pomeraji ovih polja u odnosu na početak strukture PCB označeni su simboličkim konstantama `offsPC`, `offsPSW`, `offsSP`, `offsR0`, ..., `offsR31`.

U kodu kernela postoji statički definisan pokazivač `running` koji ukazuje na PCB tekućeg procesa. Potprogram `scheduler`. koji nema argumente. realizuje raspoređivanje. tako što smešta PCB na koji ukazuje pokazivač `running` u listu spremnih procesa, a iz nje uzima jedan odabrani proces i postavlja pokazivač `running` na njega.

Na assembleru datog procesora napisati kod prekidne rutine `dispatch` koja vrši promenu konteksta korišćenjem potprograma `scheduler`. Ova prekidna rutina poziva se sistemskim pozivom iz korisničkog procesa, pri čemu se sistemski poziv realizuje softverskim prekidom.

Rešenje

```
dispatch:    ; Save the current context
             load rx,running
             store r0,#offsR0[rx] ; save regs
             store r1,#offsR1[rx]
             ...
             store r31,#offsR31[rx]
             pop r0 ; save pc
             store r0,#offsPC[rx]
             pop r0 ; save psw
             store r0,#offsPSW[rx]
             pop r0 ; save original sp
             store r0,#offsSP[rx]

             ; Select the next running process
             call scheduler

             ; Restore the new context
             load rx,running
             load r0,#offsSP[rx] ; restore original sp
             push r0
             load r0,#offsPSW[rx] ; restore original psw
             push r0
             load r0,#offsPC[rx] ; restore pc
             push r0
             load r0,#offsR0[rx] ; restore regs
             load r1,#offsR1[rx]
             ...
```



```
load r31,#offsR31[rx]
; Return
iret
```

2. zadatak, prvi kolokvijum, maj 2015.

- Postavka
- Rešenje

Neki procesor obrađuje prekide (hardverske i softverske) tako što tokom izvršavanja prekidne rutine koristi poseban stek. Taj stek alociran je u delu memorije koju koristi kernel, a na vrh tog steka ukazuje poseban registar procesora koji je dostupan samo u privilegovanom režimu rada procesora. Na tom steku samo se obrađuju prekidi, i uvek je isti (kernel ga ne menja).

Prilikom obrade prekida, procesor na ovom posebnom steku čuva: pokazivač vrha steka (SP) koji je koristio prekinuti tok kontrole, programsku statusnu reč (PSW) i programski brojač (PC), tim redom, ali *ne* i ostale programski dostupne registre. Prilikom povratka iz prekidne rutine instrukcijom **iret**, procesor restaurira ove registre sa ovog steka, a time prelazi na drugi stek.

Procesor ima dva režima rada: privilegovani (sistemske), u kome se izvršava kod kernela, i korisnički, u kome se izvršavaju korisnički procesi. Prilikom prihvatanja prekida, procesor obavezno prelazi u privilegovani režim rada. Prilikom povratka iz prekidne rutine, procesor ne menja implicitno režim rada, već ostaje u istom režimu rada u kome se nalazi prilikom obrade instrukcije **iret**. Ako je potrebno preći u korisnički režim rada, potrebno je izvršiti eksplicitnu instrukciju **setusr** koja prebacuje procesor u korisnički režim.

Operativni sistem za ovaj procesor je višenitni (engl. *multithreaded*). Sve funkcije jezgra, uključujući i obradu sistemskog poziva, raspoređivanje itd. obavljaју interne kernel niti. Svi sistemski pozivi izvršavaju se kao softverski prekid koji skače na prekidnu rutinu označenu kao **sys_call**, dok se sama identifikacija sistemskog poziva i njegovi parametri prenose kroz registre procesora.

Procesor je RISC, sa *load/store* arhitekturom i ima sledeće programski dostupne registre: 32 registra opšte namene (R0..R31), SP, PSW i PC. Registre PC, SP, PSW i R0..R31 treba sačuvati u odgovarajuća polja strukture PCB. U strukturi PCB, koja postoji i ista je i za korisničke procese i za kernel niti, postoje polja za čuvanje svih tih registara; pomeraji ovih polja u odnosu na početak strukture PCB označeni su simboličkim konstantama **offsPC**, **offsPSW**, **offsSP**, **offsR0**, ..., **offsR31**.

U kodu kernela postoje dva statički definisana pokazivača, **userRunning** i **kernelRunning**, koji ukazuju na PCB tekućeg korisničkog procesa, odnosno kernel niti, respektivno.

1. Na assembleru datog procesora napisati prekidnu rutinu **sys_call**. Ova rutina treba samo da izvrši promenu konteksta sa tekućeg korisničkog procesa na tekuću kernel nit.
2. Na assembleru datog procesora napisati prekidnu rutinu **switch_to_user** (dovoljno je i precizno objasniti razlike u odnosu na rutinu **sys_call**) koja se poziva softverskim prekidom iz kernel koda, kada je kernel završio ceo posao i kada želi da promeni kontekst sa tekuće kernel niti na korisnički proces i vrati se u korisnički režim rada.

Rešenje

1.

```
sys_call:    ; Save the current context
push r0 ; save r0 temporarily on the stack
load r0,userRunning
store r1,#offsR1[r0] ; save regs
pop r1
store r1,#offsR0[r0]
store r2,#offsR2[r0]
...
store r31,#offsR31[r0]
pop r1 ; save pc
store r1,#offsPC[r0]
```

```

pop r1 ; save psw
store r1,#offsPSW[r0]
pop r1 ; save original sp
store r1,#offsSP[r0]

; Restore the new context
load r0,kernelRunning
load r1,#offsSP[r0] ; restore original sp
push r1
load r1,#offsPSW[r0] ; restore original psw
push r1
load r1,#offsPC[r0] ; restore pc
push r1
load r1,#offsR1[r0] ; restore regs
load r2,#offsR2[r0]
...
load r31,#offsR31[r0]
load r0,#offsR0[r0]
; Return (but stay in kernel mode)
iret

```

2. Procedura `switch_to_user` izgleda potpuno analogno (skoro potpuno isto) kao i data procedura `sys_call`, samo što promenljive `userRunning` i `kernelRunning` zamenjuju mesta (uloge), a neposredno pre instrukcije `iret` stoji još samo instrukcija `setusr`.

3. zadatak, prvi kolokvijum, mart 2014.

- Postavka
- Rešenje

Na assembleru datog procesora napisati kod operacije

```
void yield (PCB* cur, PCB* nxt);
```

poput one date na predavanjima, a koja u PCB na koji ukazuje prvi argument čuva kontekst izvršavanja koje se napušta i restaurira kontekst koji je sačuvan u PCB na koga ukazuje drugi argument.

Procesor je RISC, sa *load/store* arhitekturom i ima sledeće programski dostupne registre: 32 registra opšte namene (R0..R31), SP, PSW i PC. Registre (PSW, R0..R31 i SP) treba sačuvati u odgovarajuća polja strukture PCB. U strukturi PCB postoje polja za čuvanje svih tih registara; pomeraji ovih polja u odnosu na početak strukture PCB označeni su simboličkim konstantama `offsPSW`, `offsSP`, `offsR0`, ..., `offsR31`.

Rešenje

```

yield: ; Save current context
push r0
load r0,#cur[sp]
store r1,#offsR1[r0] ; save r1
pop r1 ; save r0 through r1
store r1,#offsR0[r0]
store r2,#offsR2[r0] ; save other regs
store r3,#offsR3[r0]
...
store r31,#offsR31[r0]
store psw,#offsPSW[r0] ; save psw
store sp,#offsSP[r0] ; save sp

; Restore new context
load r0,#nxt[sp]

```

```

load sp,#offsSP[r0] ; restore sp
load psw,#offsPSW[r0] ; restore psw
load r31,#offsR31[r0] ; restore r31
... ; restore other regs
load r1,#offsR1[r0]
load r0,#offsR0[r0] ; restore r0
; Return
ret

```

3. zadatak, prvi kolokvijum, april 2014.

- [Postavka](#)
- [Rešenje](#)

Na assembleru datog procesora napisati kod operacije

```
void yield (PCB* cur, PCB* nxt);
```

poput one date na predavanjima, a koja u PCB na koji ukazuje prvi argument čuva kontekst izvršavanja koje se napušta i restaurira kontekst koji je sačuvan u PCB na koga ukazuje drugi argument.

Procesor je RISC, sa *load/store* arhitekturom i ima sledeće registre dostupne korisničkim procesima: 32 registra opšte namene (R0..R31), SP, PSW i PC. Za podršku efikasnoj promeni konteksta, ovaj procesor ima sledeće:

- registar **base** kome se može pristupiti samo u privilegovanom režimu rada procesora (nije dostupan korisničkim procesima i nije deo konteksta procesa); sve instrukcije i načini adresiranja koriste ovaj registar kao i svaki drugi registar opšte namene;
- instrukciju **saveregs** koja prepisuje sadržaj redom svih registara dostupnih korisničkim procesima, osim PC, u memoriju počev od adrese na koju ukazuje sadržaj registra **base**; ova instrukcija je dostupna samo u privilegovanom režimu rada procesora;
- instrukciju **loadregs** koja učitava redom sve registre dostupne korisničkim procesima, osim PC, iz memorije počev od adrese na koju ukazuje sadržaj registra **base**; ova instrukcija je dostupna samo u privilegovanom režimu rada procesora.

U strukturi PCB postoji polje za čuvanje svih registara dostupnih procesima; pomerač ovog polja u odnosu na početak strukture PCB označen je simboličkom konstantom **offsContext**.

Rešenje

```

yield: ; Save current context
      load base,#cur[sp]
      add base,base,#offsContext
      saveregs
      ; Restore new context
      load base,#nxt[sp]
      add base,base,#offsContext
      loadregs
      ; Return
      ret

```

3. zadatak, prvi kolokvijum, septembar 2014.

- [Postavka](#)
- [Rešenje](#)

Neki troadresni RISC procesor sa *load/store* arhitekturom, poput onog opisanog na predavanjima, poseduje 32 registra opšte namene, označenih sa *R0..R31*, statusnu reč PSW i pokazivač steka SP, koji su dostupni instrukcijama koje se izvršavaju u korisničkom režimu rada procesora, kao i dva posebna registra *Rx* i *Rp* koji su dostupni samo u privilegovanom (sistemskom) režimu rada procesora. Registar *Rx* se može koristiti kao i bilo koji registar *R0..R31*, i operativni sistem ga može koristiti proizvoljno za sopstvene potrebe (npr. prilikom promene konteksta). Registar *Rp*

se može samo čitati, jer je ožičen tako da njegova vrednost predstavlja jedinstveni identifikator svakog pojedinačnog procesora u multiprocesorskom sistemu. Svi registri su 32-bitni, a adresibilna jedinica je bajt.

Za multiprocesorski sistem sa ovim procesorom pravi se operativni sistem. U kernelu tog sistema postoje sledeće definisane konstante i strukture podataka:

```
const int NumOfProcessors = ... // Number of processors
struct PCB; // Process Control Block
PCB* runningProcesses[NumOfProcessors]; // Running processes
```

U strukturi PCB postoje polja za čuvanje vrednosti svih registara $R0..R31$, PSW i SP. Pomeraji ovih polja u odnosu na početak strukture PCB simbolički označavaju sa `offs_r0` itd. Niz `runningProcesses`, u svakom svom elementu n , sadrži pokazivač na PCB onog procesa koji se trenutno izvršava na procesoru broj n ($n=0..NumOfProcesses-1$). Na raspolaganju je operacija `schedule()` (bez argumenata), koja vrši odabir narednog procesa za izvršavanje na procesoru na kome se izvršava i koja upisuje adresu PCB tog procesa u odgovarajući element niza `runningProcesses`.

1. Na assembleru datog procesora napisati operaciju `dispatch()` (bez argumenata) koja čuva kontekst tekućeg izvršavanja i restaurira kontekst izvršavanja procesa kome treba dati procesor. U assembleru datog procesora može se koristiti identifikator statički alociranog podatka iz C programa, pri čemu se takva upotreba prevodi u konstantu sa vrednošću adrese tog podatka. Ova operacija se izvršava u kodu kernela, u sistemskom režimu. U ovu operaciju ulazi se iz prekidne rutine koja obrađuje sistemski poziv, pa su prekidi već maskirani (ne treba ih maskirati i demaskirati).
2. Da li je u ovu operaciju neophodno ubaciti kod za međusobno isključenje konkurentnog izvršavanja od strane različitih procesora tehnikom uposlenog čekanja (*spin lock*)? Obrazložiti.

Rešenje

1.

```
dispatch: ; Save the current context
        LOAD Rx,Rp      ; Rx:=# of current processor
        SHL Rx,2        ; Rx:=Rx*4
        LOAD Rx,#runningProcesses[Rx]; Rx:=&running process' PCB
        STORE #offs_r0[Rx],R0 ; store R0
        STORE #offs_r1[Rx],R1 ; store R1
        ...
        STORE #offs_psw[Rx],PSW ; store PSW
        STORE #offs_sp[Rx],SP ; store SP
        ; Call scheduler
        CALL schedule
        ; Restore the next context
        LOAD Rx,Rp      ; Rx:=# of current processor
        SHL Rx,2        ; Rx:=Rx*4
        LOAD Rx,#runningProcesses[Rx]; Rx:=&next process' PCB
        LOAD R0,#offs_r0[Rx] ; restore R0
        LOAD R1,#offs_r1[Rx] ; restore R1
        ...
        LOAD PSW,#offs_psw[Rx] ; restore PSW
        LOAD SP,#offs_sp[Rx] ; restore SP
        RTS              ; return from subroutine
```

2. Nije potrebno, jer svaki procesor jedini pristupa samo svom odgovarajućem elementu n niza `runningProcesses`, pa nema potencijalnog konflikta između procesora (nema deljenih podataka u memoriji kojima pristupaju različiti procesori u ovoj operaciji).

3. zadatak, prvi kolokvijum, mart 2013.

- [Postavka](#)
- [Rešenje](#)

U nekom operativnom sistemu svi sistemski pozivi izvršavaju se kao softverski prekid koji skače na prekidnu rutinu označenu kao `sys_call`, dok se sama identifikacija sistemskog poziva i njegovi parametri prenose kroz registre procesora. Jezgro tog operativnog sistema je višenitno – poseduje više internih kernel niti koje obavljaju različite poslove: izvršavaju uporedne I/O operacije, vrše druge interne poslove jezgra, pa čak postoje i niti koje obavljaju sve potrebne radnje prilikom promene konteksta korisničkih procesa (osim samog čuvanja i restauracije konteksta procesora), kao što su smeštanje PCB korisničkog procesa koji je do tada bio tekući u odgovarajući red (spremnih ili blokiranih, u zavisnosti od situacije), izbor novog tekućeg procesa iz skupa spremnih, promenu memorijskog konteksta, obradu samog konkretnog sistemskog poziva, itd. Prilikom obrade sistemskog poziva `sys_call`, prema tome, treba samo oduzeti procesor tekućem korisničkom procesu i dodeliti ga tekućoj kernel niti.

Na PCB tekućeg korisničkog procesa ukazuje globalni pokazivač `runningUserProcess`, a na PCB tekuće interne niti jezgra ukazuje globalni pokazivač `runningKernelThread`. I interne niti jezgra vrše promenu konteksta između sebe na isti način, pozivom softverskog prekida koji ima istu internu strukturu kao i rutina `sys_call`.

Prilikom obrade prekida, procesor čuva ceo kontekst svog izvršavanja, odnosno sve programski dostupne registre (uključujući i PC, SP i PSW) u memoriji. Međutim, kako se ne bi dogodilo prekoračenje steka korisničkog procesa ili stranična greška prilikom tog postupka, procesor ove registre čuva u strukturu u memoriji čija se adresa nalazi u posebnom namenskom registru procesora SPX. Prilikom povratka iz prekidne rutine, procesor sam restaurira kontekst izvršavanja iz iste ove strukture na koju tada ukazuje SPX. Ovaj registar dostupan je samo iz privilegovanog sistemskog režima rada procesora u koji se prelazi softverskim prekidom.

I korisnički procesi i interne niti jezgra u svojim PCB strukturama imaju polje pod nazivom `context` u kojima se čuva kontekst procesora. Pomeraj ove strukture u odnosu na početak PCB dat je simboličkom vrednošću `offsContext`. Procesor je RISC, sa *load/store* arhitekturom.

Upotreba imena globalne promenljive iz C koda u assembleru datog procesora predstavlja memorijsku adresu te globalne statičke promenljive. Deo memorije koju koristi jezgro (kod i podaci) preslikava se u virtuelni adresni prostor svih korisničkih procesa, tako da prilikom obrade sistemskog poziva i prelaska u kod kernela nema promene memorijskog konteksta.

Na assembleru datog procesora napisati prekidnu rutinu `sys_call`. Obrazložiti rešenje!

Rešenje

Kada se pri obradi prekida kontekst procesora prepisuje u memoriju, bitno je da u registru SPX bude adresa polja `context` iz PCB-a tekuće niti koja gubi procesor. Isto važi pri svakoj promeni konteksta, pa i pri napuštanju kernel niti. Vrednost samog SPX se ne mora čuvati, jer se jednostavno restaurira dodavanjem pomeraja polja `context` na vrednost adrese PCB-a. Dakle, jedino što je prilikom promene konteksta potrebno uraditi jeste postaviti SPX na vrednost adrese polja `context` niti koja dobija procesor. Tako rutina `sys_call` izgleda ovako:

```
sys_call: load spx, [runningKernelThread]
          add spx, #offsContext
          iret
```

Povratak iz rutine će restaurirati kontekst kernel niti iz strukture `context` PCB-a niti koja je dobila procesor, jer na tu strukturu tada ukazuje SPX. Potpuno analogno izgleda i rutina kojom se kontrola iz kernel niti predaje korisničkoj niti, samo što je umesto `runningKernelThread` upotrebljen `runningUserProcess`.

3. zadatak, prvi kolokvijum, april 2013.

- [Postavka](#)
- [Rešenje](#)

U nekom operativnom sistemu svi sistemski pozivi izvršavaju se kao softverski prekid koji skače na prekidnu rutinu označenu kao `sys_call`, dok se sama identifikacija sistemskog poziva i njegovi parametri prenose kroz registre procesora. Prilikom obrade sistemskog poziva `sys_call` treba uraditi redom sledeće:

1. Sačuvati kontekst tekućeg korisničkog procesa u njegov PCB. Procesor je RISC, sa *load/store* arhitekturom i ima sledeće programski dostupne registre: 32 registra opšte namene (R0..R31), SP, PSW i PC. Prilikom obrade prekida, procesor prelazi u sistemski (zaštićeni, kernel) režim, a na steku čuva samo PSW i PC. Na PCB tekućeg korisničkog procesa ukazuje globalni pokazivač koji se nalazi na adresi simbolički označenoj sa

running. Ostale registre (R0..R31 i SP) treba sačuvati u odgovarajuća polja strukture PCB. U strukturi PCB postoje polja za čuvanje svih tih registara; pomeraji ovih polja u odnosu na početak strukture PCB označeni su simboličkim konstantama `offsSP`, `offsR0`, ..., `offsR31`.

2. Preći na stek kernela u kome se izvršava ceo kernel kod. Na vrh tog steka (zapravo dno, pošto je stek prazan inicijalno, kao i posle svakog izlaska iz kernel koda) ukazuje pokazivač na lokaciji simbolički označenoj sa `kernelStack`.
3. Dozvoliti spoljašnje maskirajuće prekide instrukcijom `inte`, pošto je kod kernela i svih prekidnih rutina napravljen tako da se pojava prekida samo pamti u softverskim indikatorima, a kernel kod obrađuje prekid odloženo, kada je to bezbedno i odgovarajuće.
4. Pozvati potprogram na adresi simbolički označenoj sa `kernel`. Ovaj potprogram (bez argumenata) obavlja sve potrebne poslove, kao što su obrada sistemskog poziva i raspoređivanje (nakon čega `running` pokazuje na PCB novog tekućeg procesa).
5. Po povratku iz potprograma `kernel`, maskirati prekide instrukcijom `intd`, povratiti kontekst novog tekućeg procesa iz njegovog PCB-a, i vratiti se u korisnički režim i proces iz sistemskog poziva. Instrukcija `iret` prebacuje procesor u korisnički režim i vraća sačuvane registre PC i PSW sa tekućeg steka. Na assembleru datog procesora napisati prekidnu rutinu `sys_call`.

Rešenje

```
sys_call: ; Save current context
    push r0
    load r0,[running]
    store r1,#offsR1[r0] ; save r1
    pop r1 ; save r0 through r1
    store r1,#offsR0[r0]
    store r2,#offsR2[r0] ; save other regs
    store r3,#offsR3[r0]
    ...
    store r31,#offsR31[r0]
    store sp,#offsSP[r0] ; save sp

    ; Switch to kernel code
    load sp,[kernelStack] ; switch to kernel stack
    inte ; enable interrupts
    call kernel ; go to kernel code
    intd ; disable interrupts

    ; Restore new context
    load r0,[running]
    load sp,#offsSP[r0] ; restore sp
    load r31,#offsR31[r0] ; restore r31
    ... ; restore other regs
    load r1,#offsR1[r0]
    load r0,#offsR0[r0] ; restore r0
    ; Return
    iret
```

3. zadatak, prvi kolokvijum, septembar 2011.

- [Postavka](#)
- [Rešenje](#)

U implementaciji jezgra nekog jednoprocorskog *time-sharing* operativnog sistema, radi pojednostavljenja celog mehanizma promene konteksta, primenjeno je sledeće neobično rešenje. Promena konteksta vrši se isključivo kao posledica prekida, na samo jednom mestu u kodu koji se izvršava na prekid. Prekidi dolaze od raznih uređaja, u najmanju ruku od vremenskog brojača, jer on u svakom slučaju generiše prekid u konačnom vremenu, bilo zbog toga što je tekućoj niti isteklo dodeljeno procesorsko vreme, ili iz nekog drugog razloga zbog koga je vremenski brojač od

strane sistema bio startovan da meri interval i generiše prekid kada on istekne. Zbog toga tekuća nit nikada ne gubi procesor sinhrono, čak ni kada poziva blokirajuću operaciju npr. na semaforu. Umesto toga, ukoliko je potrebno da se nit blokira u nekom blokirajućem pozivu, nit se samo „označi“ blokiranom i nastavlja sa izvršavanjem (uposlenim čekanjem) sve dok ne stigne sledeći prekid. Kada takav prekid stigne, kod koji se poziva proverava stanje niti koja je prekinuta i ako je ona označen kao blokirana, vrši samu promenu konteksta.

Na primer, implementacija operacije `wait()` na semaforu u ovom sistemu izgleda ovako:

```
class Thread {
    int isBlocked;
    jmpbuf context;
    static Thread* running;
    ...;
};

void Semaphore::wait () {
    lock(); // Disable interrupts
    if (--val<0) {
        Thread::running->isBlocked = 1; // Mark as blocked,
        queue->put(Thread::running); // put it in the semaphore queue,
        unlock(); // enable interrupts,
        while (Thread::running->isBlocked); // and then busy-wait
    } // until it is preempted, suspended, and
    // then resumed later
    else unlock(); // Enable interrupts
}
```

Korišćenjem standardnih bibliotečnih operacija `setjmp()` i `longjmp()`, realizovati prekidnu rutinu `yield()` koja vrši opisanu promenu konteksta na prekid. Klasa `Schedule` koja realizuje red spremnih niti sa raspoređivanjem ima interfejs kao u školskom jezgru.

Rešenje

```
interrupt void yield () {
    if (setjmp(Thread::running->context)==0) {

        if (!Thread::running->isBlocked) Scheduler::put(Thread::running);
        Thread::running = Scheduler::get();
        Thread::running->isBlocked = 0;

        longjmp(Thread::running->context,1);
    }
}
```

4. zadatak, prvi kolokvijum, april 2010.

- [Postavka](#)
- [Rešenje](#)

U nekom operativnom sistemu svi sistemski pozivi izvršavaju se kao softverski prekid koji skače na prekidnu rutinu označenu kao `sys_call`, dok se sama identifikacija sistemskog poziva i njegovi parametri prenose kroz registre procesora. Jezgro tog operativnog sistema je višenitno – poseduje više internih kernel niti koje obavljaju različite poslove: izvršavaju uporedne I/O operacije, vrše druge interne poslove jezgra, pa čak postoje i niti koje obavljaju sve potrebne radnje prilikom promene konteksta korisničkih procesa (osim samog čuvanja i restauracije konteksta procesora), kao što su smeštanje PCB korisničkog procesa koji je do tada bio tekući u odgovarajući red (spremnih ili blokiranih, u zavisnosti od situacije), izbor novog tekućeg procesa iz skupa spremnih, promenu memorijskog konteksta, obradu samog konkretnog sistemskog poziva, itd. Prilikom obrade sistemskog poziva `sys_call`, prema tome, treba samo oduzeti procesor tekućem korisničkom procesu i dodeliti ga tekućoj kernel niti.

Na PCB tekućeg korisničkog procesa ukazuje globalni pokazivač `runningUserProces`, a na PCB tekuće interne niti jezgra ukazuje globalni pokazivač `runningKernelThread`. I interne niti jezgra vrše promenu konteksta između sebe na isti način, pozivom softverskog prekida koji ima istu internu strukturu kao i rutina `sys_call`. I korisnički iprocesi i interne niti jezgra u svojim PCB strukturama imaju polje u kojima se čuva SP i čiji je pomeraj u odnosu na početak PCB dat simboličkom vrednošću `offsSP`.

Procesor je RISC, sa *load/store* arhitekturom. Od registara opšte namene poseduje registarski fajl sa registrima R0..R63, SP, PSW i PC. Prilikom prihvatanja prekida i pre skoka u prekidnu rutinu, hardver procesora na steku čuva sve programski dostupne registre, osim SP.

Napomene: Upotreba imena globalne promenljive iz C koda u assembleru datog procesora predstavlja memorijsku adresu te globalne statičke promenljive. Deo memorije koju koristi jezgro (kod i podaci) preslikava se u virtuelni adresni prostor svih korisničkih procesa, tako da prilikom obrade sistemskog poziva i prelaska u kod kernela nema promene memorijskog konteksta.

Na assembleru datog procesora napisati prekidnu rutinu `sys_call`.

Rešenje

```
sys_call: load r0, [runningUserProcess]
          store sp, offsSP[r0]
          load r0, [runningKernelThread]
          load sp, offsSP[r0]
          ired
```


Sistemi pozivi

1. zadatak, drugi kolokvijum, april 2024.

- [Postavka](#)
- [Rešenje](#)

Koristeći Unix sistemske pozive *fork*, *exit*, *wait* i *getpid* koji vraća *pid* pozivajućeg procesa, kao i funkciju *printf*, napisati C program koji, kada se nad njim pokrene (roditeljski) proces, pokreće *N* procesa dece, gde je *N* predefinisana konstanta. Svaki *i*-ti proces dete treba da se vrti u praznoj petlji $i \cdot 1.000.000$ puta i potom završi. Nakon što pokrene sve procese – svoju decu, roditeljski proces treba da ih sačekuje da završe, jednog po jednog, u bilo kom redosledu, i da za svaki završeni proces dete ispiše njegov redni broj *i* i njegov *pid*. Kada sačeka da sva njegova deca završe, završava se i roditeljski proces.

Rešenje

```
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#define handle_error(msg) \
    do { printf("Error: %s\n",msg); exit(-1); } while(0)

const int N = 10;

int main() {
    for (int i = 0; i < N; i++) {
        pid_t pid = fork();
        if (pid < 0) handle_error("Cannot create a process.");
        if (pid == 0) {
            for (unsigned long j = 0; j < (unsigned long)(i + 1) * 1000000; j++);
            exit(i);
        }
    }
    for (int i = 0; i < N; i++) {
        int pi;
        pid_t pid = wait(&pi);
        printf("Child process #%d with pid=%d complete.\n", pi, pid);
    }
    return 0;
}
```

1. zadatak, drugi kolokvijum, maj 2023.

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu nalik sistemu Unix postoji sistemski poziv *signal* čiji je potpis dat dole.¹ Ovim pozivom proces može da preusmeri obradu signala datog prvim parametrom na rutinu (*signal handler*) datu drugim parametrom. Ukoliko uspe, ovaj sistemski poziv vraća pokazivač na rutinu koja je bila postavljena za dati signal pre ove promene, dok u slučaju neuspeha vraća *SIG_ERR*. Proces dete nasleđuje rutine za obradu signala od procesa roditelja, ali može nezavisno da ih redefiniše.

Koristeći sistemske pozive *fork*, *exit*, *signal* i *kill*, kao i funkcije *printf* i *getchar*, napisati C program koji, kada se nad njim pokrene (roditeljski) proces, pokrene jedan proces dete, a onda taj proces dete gasi slanjem signala *SIGTERM*. Proces dete izvršava praznu petlju sve dok ne stigne ovaj signal. Kada primi signal za gašenje *SIGTERM*, proces dete treba da postavi pitanje korisniku ispisom na standardni izlaz rečenice “Da li ste sigurni da želite da prekinete izvršavanje? (D/N)”. Ukoliko se na standardni ulaz procesa deteta unose znak ‘D’ ili ‘d’, proces dete treba

¹Ovo je stari i prevaziđen Unix sistemski poziv. Noviji oblik je nešto složeniji i robusniji, ali principijelno sličan.

da se ugasi, a u suprotnom da nastavi da izvršava praznu petlju i tako dalje. Da bi se izbeglo utrkivanje u slučaju da proces roditelj pošalje signal pre nego što je proces dete preusmerio rutinu za njegovu obradu, proces roditelj treba najpre da preusmeri svoju rutinu za ovaj signal, potom pokrene proces dete koje će to naslediti, a onda sebi preusmeru obradu ovog signala na staru rutinu. Sve greške koje onemogućavaju dalje funkcionisanje kako je opisano obraditi ispisom poruke o grešci i završetkom procesa.

```
typedef void (*SIGHANDLER)(int);
SIGHANDLER signal(int signal, SIGHANDLER newHandler);
```

Rešenje

```
#include <signal.h>
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>

#define handle_error(msg) \
    do { printf("Error: %s\n", msg); exit(-1); } while(0)

typedef void (*SIGHANDLER)(int);

volatile int complete = 0;

void handler(int) { complete = 1; }

int main() {
    SIGHANDLER oldh = signal(SIGTERM, handler);
    if (oldh == SIG_ERR) {
        handle_error("Cannot set the signal handler.");
    }
    pid_t pid = fork();
    if (pid < 0) {
        handle_error("Cannot create the child process.");
    } else if (pid == 0) {
        while (1) {
            while (!complete);
            printf("Are you sure you want to exit? (y/n)");
            int c = getchar();
            if (c == 'Y' || c == 'y') exit(0);
            complete = 0;
        } else {
            signal(SIGTERM, oldh);
            kill(pid, SIGTERM);
        }
    }
}
```

1. zadatak, drugi kolokvijum, avgust 2023.

- [Postavka](#)
- [Rešenje](#)

1. Korišćenjem Unix sistemskih poziva *fork*, *exit* i *wait/waitpid* implementirati funkciju *max* koja pronalazi maksimum celobrojnih vrednosti *val* sadržanih u čvorovima binarnog stabla predstavljenih strukturom *Node*, tako što proces koji poziva ovu funkciju sa argumentom koji ukazuje na koren datog stabla obrađuje taj koren i njegovo levo podstablo, a za obradu desnog podstabla kreira poseban proces koja teče uporedo sa obradom levog podstabla, i tako dalje rekursivno za svako podstablo na isti opisani način. Ignorirati greške.

2. Ako se ova funkcija pozove za koren potpunog balansiranog binarnog stabla sa n nivoa i $2^n - 1$ čvorova, koliko ukupno procesa obrađuje ovo stablo, uključujući i početni koji poziva ovu funkciju za koren ovog stabla? Precizno obrazložiti odgovor.

```
struct Node {
    Node *left, *right;
    int val;
};
int max(Node* nd);
```

Rešenje

```
1.  int max (Node* nd) {
    int m = nd->val, m1;
    if (nd->right)
        if (fork() == 0)
            exit(max(nd->right));
    if (nd->left) {
        m1 = max(nd->left);
        if (m1 > m) m = m1;
    }
    if (nd->right) {
        wait(&m1);
        if (m1 > m) m = m1;
    }
    return m;
};
```

2. Prvi nivo (koren stabla) obrađuje jedan, inicijalni proces. Drugi nivo, osim njega, obrađuje još jedan proces, njegovo dete. Svaki nivo obrađuju svi procesi koji obrađuju prethodni nivo i još toliko novih procesa njihove dece, odnosno dvostruko više (nijedan se ne gasi). Tako n -ti nivo obrađuju svi kreirani procesi, njih $2^n - 1$.

1. zadatak, drugi kolokvijum, maj 2022.

- [Postavka](#)
- [Rešenje](#)

Koristeći samo sistemske pozive *fork*, *wait* i *exit*, kao i funkciju *printf*, napisati C program koji pronalazi maksimalnu vrednost svih elemenata ogromne celobrojne matrice dimenzije M puta N tako što uporedo pronalazi maksimum u svakoj vrsti matrice (maksimum svake vrste pronalazi u po jednom od pokrenutih procesa-dece), a onda pronalazi maksimum tih maksimuma. U slučaju greške, ovaj program treba da ispiše poruku o grešci i vrati status -1, a u slučaju uspeha treba da ispiše pronađeni maksimum i vrati status 0. Pretpostaviti da je matrica već nekako inicijalizovana.

```
const int M = ..., N = ...;
extern int mat[M][N];
```

Rešenje

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
```

```
const int M = ..., N = ...;
extern int mat[M][N];
```

```
int max(int i) {
    int m = mat[i][0];
```

```

    for (int j = 1; j < N; j++) {
        if (mat[i][j] > m) {
            m = mat[i][j];
        }
    }
    return m;
}

int main() {
    for (int i = 0; i < M; i++) {
        pid_t pid = fork();
        if (pid < 0) {
            printf("Error: Cannot create a child process.\n");
            exit(-1);
        }
        if (pid == 0) {
            exit(max(i));
        }
    }
    int max;
    wait(&max);
    for (int i = 1; i < M; i++) {
        int m;
        wait(&m);
        if (m > max) {
            max = m;
        }
    }
    printf("Max: %d\n", max);
    exit(0);
}

```

1. zadatak, drugi kolokvijum, jun 2022.

- [Postavka](#)
- [Rešenje](#)

Koristeći samo sistemske pozive *fork*, *wait/waitpid* i *exit*, kao i funkciju *printf*, napisati C program koji pronalazi maksimalnu vrednost svih elemenata ogromne matrice dimenzije M puta N elemenata tipa `double` tako što uporedo pronalazi maksimum u svakoj vrsti matrice (maksimum svake vrste pronalazi u po jednom od pokrenutih procesa-dece), a onda pronalazi maksimum tih maksimuma. U slučaju greške, ovaj program treba da ispiše poruku o grešci i vrati status -1, a u slučaju uspeha treba da ispiše pronađeni maksimum i vrati status 0.

Pretpostaviti da je matrica već nekako inicijalizovana.

```

const int M = ..., N = ...;
extern double mat[M][N];

```

Rešenje

```

#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>

#define handle_error(msg) do { \
    printf(msg); \
    exit(-1); \
} while(0)

```

```

} while (0)

const int M = ..., N = ...;
extern double mat[M][N];
pid_t pids[M];

int max(int i) {
    double m = mat[i][0];
    int mj = 0;
    for (int j = 1; j < N; j++) {
        if (mat[i][j] > m) {
            m = mat[i][j];
            mj = j;
        }
    }
    return mj;
}

int main() {
    for (int i = 0; i < M; i++) {
        pid_t pid = pids[i] = fork();
        if (pid < 0) {
            handle_error("Error: Cannot create a child process.\n");
        }
        if (pid == 0) {
            exit(max(i));
        }
    }
    double max;
    int mj;
    if (waitpid(pids[0], &mj) < 0) {
        handle_error("Error waiting a child process.\n");
    }
    max = mat[0][mj];
    for (int i = 1; i < M; i++) {
        if (waitpid(pids[i], &mj) < 0) {
            handle_error("Error waiting a child process.\n");
        }
        if (mat[i][mj] > max) {
            max = mat[i][mj];
        }
    }
    printf("Max: %f\n", max);
    exit(0);
}

```

1. zadatak, kolokvijum, oktobar 2020.

- [Postavka](#)
- [Rešenje](#)

U sistemu Windows postoji sistemski poziv

```
int spawnvp (int mode, char *path, const char **argv);
```

koji radi isto što i kombinacija poziva *fork* i *exec* na sistemima nalik sistemu Unix: kreira proces dete pozivajućeg procesa nad programom u izvršivom fajlu zadatom putanjom u argumentu **path**. Parametar **argv** predstavlja niz pokazivača na nizove znakova (*null-terminated strings*) koji predstavljaju argumente poziva programa koji treba

izvršiti. Po konvenciji, prvi argument pokazuje na naziv fajla samog programa koji se izvršava. Ovaj niz pokazivača mora da se završi elementom sa vrednošću NULL. Ovaj sistemski poziv vraća 0 u slučaju uspeha, a vrednost manju od 0 u slučaju greške. Parametar `mode` definiše modalitet kreiranja procesa deteta sledećim simboličkim konstantama:

P_OVERLAY	Overlays the parent process with child, which destroys the parent. This has the same effect as the exec functions.
P_WAIT	Suspends the parent process until the child process has finished executing (synchronous spawn).
P_NOWAIT	Continues to execute the parent process concurrently with child process (asynchronous spawn).
P_DETACH	The child is run in the background without access to the console (asynchronous spawn).

Korišćenjem ovog sistemskog poziva realizovati sledeću funkciju:

```
int multispawn (int number, const char* path, const char* args[]);
```

Ova funkcija treba da kreira zadati broj (`number`) procesa dece, i svaki od tih procesa dece treba da izvršava isti program zadat u fajlu sa stazom `path`, sa samo po jednim argumentom. Argumenti tih procesa dece zadati su redom u prvih `number` elemenata niza `args`. Ova funkcija treba da vrati broj uspešno kreiranih procesa dece i da odmah vrati kontrolu pozivaocu, ne čekajući da se ti procesi deca završe.

Rešenje

```
#include <process.h>

int multispawn (int number, const char* path, const char* args[]){
    int ret = 0;
    // Prepare the arguments for the children:
    const char* childArgs[3];
    childArgs[0] = path;
    childArgs[2] = NULL;
    for (int i=0; i<number; i++) {
        childArgs[1] = args[i];
        // Create a child:
        int status = spawnvp(P_NOWAIT,path,childArgs);
        if (status>=0) ret++;
    }
    return ret;
}
```

1. zadatak, kolokvijum, jul 2020.

- [Postavka](#)
- [Rešenje](#)

Dat je neki veliki jednodimenzioni niz `data` veličine `N`, čiji su elementi tipa `Data` (deklaracije dole). Jedan element niza obrađuje procedura `processData`. Ovaj niz potrebno je obraditi pomoću `n` uporednih niti (procedura `parallelProcessing`), gde je `n` zadati parametar, tako što svaka od tih `n` uporednih niti iterativno obrađuje približno isti broj elemenata ovog velikog niza. Drugim rečima, niz treba particionisati na `n` disjunktnih podnizova, što približnije jednakih, i te particije obrađivati uporedo, kako bi se niz obradio paralelno na višeprocorskom sistemu.

Koristiti sistemski poziv `thread_create` koji kreira nit nad funkcijom na koju ukazuje prvi parametar, i pozivu te funkcije dostavlja drugi parametar ovog sistemskog poziva. Ova funkcija vraća identifikator kreirane niti (pozitivna vrednost), odnosno status greške (negativna vrednost). Ignorirati moguće greške.

```
const int N = ...;
class Data;
Data data[N];
void processData(Data*);
void parallelProcessing (int n);
```

```
int thread_create (void (*thread_body)(void*), void* arg);
```

Rešenje

```
typedef struct ChunkDesc {
    int offset, size;
} ChunkDesc;

void dataProcessor (void* chunk) {
    ChunkDesc* cnk = (ChunkDesc*)chunk;
    int end = cnk->offset+cnk->size;
    for (int i=cnk->offset; i<end; i++)
        processData(&data[i]);
    delete cnk;
}

void parallelProcessing (int n) {
    int chunkSz = N/n;
    int offset = 0;
    for (int i=0; i<n; i++) {
        int myChunkSz = chunkSz;
        if (i<N%n) myChunkSz++;
        ChunkDesc* chunk = new ChunkDesc;
        chunk->offset = offset; chunk->size = myChunkSz;
        offset+= myChunkSz;
        thread_create(dataProcessor, chunk);
    }
}
```

4. zadatak, kolokvijum, avgust 2020.

- [Postavka](#)
- [Rešenje](#)

Na raspolaganju su sledeći standardni POSIX sistemski pozivi:

- `int close (int fd)`: zatvara fajl sa zadatim deskriptorom;
- `int write (int fd, const void *buffer, size_t size)`: upisuje dati sadržaj u fajl sa zadatim deskriptorom; ako je fajl otvoren sa postavljenim `O_APPEND`, onda se pre upisa tekuća pozicija implicitno pomera na kraj fajla i upis uvek vrši iza kraja fajla, proširujući sadržaj;
- `int open(const char *pathname, int flags, mode_t mode)`: otvara fajl sa zadatom putanjom; argument `flags` mora da uključi jedno od sledećih prava pristupa: `O_RDONLY`, `O_WRONLY`, ili `O_RDWR`. Ako je uključen i fleg `O_CREAT`, fajl će biti kreiran ukoliko ne postoji; ako je pritom uključen i `O_EXCL`, a fajl već postoji, funkcija će vratiti grešku (-1), a kod greške postavljen u globalnoj sistemskoj promenljivoj `errno` biće jednak `EEXIST`. Ako je uključen `O_TRUNC`, i ako fajl već postoji, njegov sadržaj se pri otvaranju briše. Argument `mode` definiše prava pristupa za fajl koji se kreira samo u slučaju da je uključen `O_CREAT` (tada je i obavezan), i to na sledeći način:

S_IRWXU	0x0700	user (file owner) has read, write and execute permission
S_IRUSR	0x0400	user has read permission
S_IWUSR	0x0200	user has write permission
S_IXUSR	0x0100	user has execute permission
S_IRWXG	0x0070	group has read, write and execute permission
S_IRGRP	0x0040	group has read permission
S_IWGRP	0x0020	group has write permission
S_IXGRP	0x0010	group has execute permission
S_IRWXO	0x0007	others have read, write and execute permission
S_IROTH	0x0004	others have read permission
S_IWOTH	0x0002	others have write permission
S_IXOTH	0x0001	others have execute permission

U nekom programu koristi se u uređeno binarno stablo sa čvorom tipa strukture `Node` sa celim brojevima kao sadržajem čvora (polje `contents` tipa `int`), tako da su u levom podstablu (polje `left` tipa `Node*`) svakog čvora brojevi koji su manji, a u desnom (polje `right`) oni koji su veći ili jednaki od sadržaja datog čvora. Implementirati funkciju koja u fajl sa datim imenom upisuje niz celih brojeva iz ovakvog stabla sa datim korenom, uređen neopadajuće (funkcija vraća 0 u slučaju uspeha, vrednost manju od 0 neku grešku). Ako fajl ne postoji, treba ga kreirati sa pravima na čitanje i upis za vlasnika, a samo na čitanje za sve ostale.

```
int save (const char* fname, Node* root);
```

Rešenje

```
int write (int fd, Node* root) {
    if (!root) return 0;
    int ret = write(root->left);
    if (ret<0) return ret;
    ret = write(fd,&(root->contents),sizeof(int));
    if (ret<0) return ret;
    ret = write(root->right);
    return ret;
}

int save (const char* fname, Node* root) {
    int fd = open(fname,O_WRONLY|O_CREAT|O_TRUNC,
                  S_IRUSR|S_IWUSR|S_IRGRP|S_IROTH);
    if (fd<0) return fd;
    int ret = write(fd,root);
    close(fd);
    return ret;
}
```

2. zadatak, kolokvijum, septembar 2020.

- [Postavka](#)
- [Rešenje](#)

Neki operativni sistem podržava sledeće sistemske pozive za upravljanje nitima:

- `fork()`: kreira novu nit-dete kao klon roditelja, poput Unix sistemskog poziva *fork* (samo što kreira nit, a ne proces); u kontekstu roditelja vraća ID kreirane niti-deteta, a u kontekstu deteta vraća 0;
- `exit()`: gasi pozivajuću nit;
- `wait(int pid)`: suspenduje pozivajuću nit sve dok se nit dete te niti sa datim ID ne završi (ne ugasi se); argument 0 suspenduje pozivajuću nit sve dok se svi njeni potomci ne završe.

Korišćenjem ovih sistemskih poziva, realizovati operaciju `cobegin()` koja prima tri argumenta: niz pokazivača na funkcije, niz argumenata tipa `void*` i ceo broj *n*, a koja pokreće uporedno izvršavanje *n* niti nad funkcijama iz datog niza, pri čemu funkciju iz *i*-tog elementa niza pokazivača na funkcije poziva sa *i*-tim elementom u nizu argumenata,

a vraća kontrolu pozivaocu tek kada se sve te niti (odnosno funkcije) završe. Obratiti pažnju na to da je nit koja poziva ovu funkciju `cobegin` možda ranije pokrenula neke druge niti koje ne treba čekati pre izlaska iz `cobegin`. Ignorisati sve moguće greške.

```
typedef void (*PF)(void*);

void cobegin (PF f[], void* af[], int n);
```

Rešenje

```
typedef void (*PF)(void*);

void cobegin(PF f[], void* af[], int n) {
    int* ids = new int[n];
    for (int i = 0; i < n; i++)
        if ((ids[i] = fork()) == 0) {
            pf[i](af[i]);
            exit();
        }
    for (int i = 0; i < n; i++)
        wait(ids[i]);
    delete ids;
}
```

3. zadatak, prvi kolokvijum, mart 2019.

- [Postavka](#)
- [Rešenje](#)

U nekom operativnom sistemu postoje sledeći sistemski pozivi:

- `fork()`, `execlp(const char*)`: kao u sistemu Unix i njemu sličnim;
- `int wait(int pid, unsigned timeout)`: suspenduje pozivajući roditeljski proces dok se ne završi proces sa zadatim PID, ali ga suspenduje najduže onoliko koliko je zadato drugim argumentom (vreme čekanja u milisekundama). Ako je vrednost prvog argumenta NULL, pozivajući proces se suspenduje dok se ne završe sva njegova deca (ili ne istekne vreme čekanja); ako je vrednost drugog argumenta 0, poziv odmah vraća rezultat, bez čekanja. Vraćena vrednost 0 označava da su svi procesi koji su se čekali završili (pre isteka vremena čekanja); vraćena vrednost veća od 0 znači da je vreme čekanja isteklo pre nego što je neki od procesa koji su se čekali završili.
- `int kill(int pid)`: gasi proces sa zadatim PID.

Svi ovi sistemski pozivi vraćaju negativan kod greške u slučaju neuspeha.

Na jeziku C napisati program koji se poziva sa jednim argumentom koji predstavlja stazu do *exe* fajla. Ovaj program treba da kreira *N* procesa koji izvršavaju program u *exe* fajlu zadatom argumentom (*N* je konstanta definisana u programu), a potom da čeka da se svi ti procesi-deca završe u roku od 5 sekundi. Sve procese-decu koji se nisu završili u tom roku treba da ugasi. Obraditi sve greške u sistemskim pozivima ispisom odgovarajuće poruke.

Rešenje

```
const int N = ...;
const unsigned timeout = 5000;
int pid[N];

int main (int argc, const* char argv[]) {
    if (argc<2) {
        printf("Error: Missing argument for the program to run.\n");
        exit(-1);
    }
}
```

```

int ret = 0; // Status to return on exit

// Create children:
for (int i=0; i<N; i++) {
    pid[i] = fork();
    if (pid[i] < 0) {
        printf("Error: Failed to create a child process number %d.\n",i);
        ret = -2;
    } else
    if (pid[i] == 0) {
        execlp(argv[1]);
        printf("Error: Failed to execute the program for the child process
number %d.\n",i);
        exit(-1); // Terminate the child, not the parent
    }
}

// Wait for all children:
int stat = wait(NULL,timeout);
if (stat<0) {
    printf("Error: Failed to wait for the children processes.\n");
    ret -= 4;
}

if (stat == 0) {
    printf("All children completed in time.\n",i);
    exit(0);
}

// Kill all incomplete children:
for (int i=0; i<N; i++) {
    stat = wait(pid[i],0);
    if (stat == 0) continue; // Child completed
    if (stat<0) {
        printf("Error: Failed to wait for the child process number %d.\n",i);
        if (ret > -8) ret -= 8;
    }
    stat = kill(pid[i]);
    if (stat<0) {
        printf("Error: Failed to kill the child process number %d.\n",i);
        if (ret > -16) ret -= 16;
    }
}

exit(ret);
}

```

3. zadatak, prvi kolokvijum, maj 2019.

- [Postavka](#)
- [Rešenje](#)

U nekom operativnom sistemu postoje sledeći sistemski pozivi:

- `int fork()`: kao u sistemu Unix i njemu sličnim, samo što ne kreira proces, nego nit; u kontekstu niti-roditelja vraća identifikator niti-deteta (PID), a u kontekstu niti-deteta vraća 0;
- `exit()`: završava pozivajuću nit;
- `wait(int pid)`: suspenduje pozivajuću roditeljsku nit dok se ne završi nit-dete sa zadatim PID.

Svi ovi sistemski pozivi vraćaju negativan kod greške u slučaju neuspeha.

Data je globalna matrica `mat` celih brojeva (`int`), dimenzija $M \times N$ (M i N su konstante). Potrebno je napisati potprogram `par_sum` koji treba da izračuna ukupan zbir svih elemenata matrice `mat`, ali na sledeći način: najpre treba da izračuna zbir svake, i -te vrste ove matrice, uporedo, u M uporednih niti (zbir jedne vrste izračunava jedna nit), i da taj zbir smesti u i -ti element jednog pomoćnog niza; potom treba da sabere ovako izračunate zbirove vrsta i vrati rezultat. Ignorirati eventualne greške i prekoračenje.

```
const int M = ..., N = ...;
int mat[M][N];
int par_sum ();
```

Rešenje

```
const int M = ..., N = ...;
int mat[M][N];
int sums[M];
int pid[M];
```

```
void sum (int row) {
    int s = 0;
    for (int i=0; i<N; i++)
        s += mat[row][i];
    sums[row] = s;
}
```

```
int par_sum () {
    for (int i=0; i<M; i++) {
        pid[i] = fork();
        if (pid[i]==0) {
            sum(i);
            exit();
        }
    }
}
```

```
int s = 0;
for (int i=0; i<M; i++) {
    wait(pid[i]);
    s += sums[i];
}
return s;
}
```

3. zadatak, prvi kolokvijum, jun 2019.

- [Postavka](#)
- [Rešenje](#)

U nekom operativnom sistemu postoje sledeći sistemski pozivi:

- `int thread_create(void*(void*),void*)`: kreira nit nad funkcijom na koju ukazuje prvi argument; ta funkcija prima jedan argument tipa `void*` i ne vraća rezultat; novokreirana nit poziva tu funkciju sa stvarnim argumentom jednakim drugom argumentu ovog sistemskog poziva; sistemski poziv vraća PID kreirane niti;
- `void wait(int pid)`: suspenduje pozivajuću roditeljsku nit dok se ne završi nit-dete sa zadatim PID; ako je vrednost argumenta 0, pozivajuća nit se suspenduje dok se ne završe sve niti-deca.

Korišćenjem bibliotečne funkcije `mempy`, implementirati funkciju `par_mempy` istog potpisa koja će iskoristiti paralelizam na multiprocesorskom sistemu i velike segmente memorije kopirati paralelno. Ova funkcija će to uraditi

tako što će kreirati potreban broj uporednih niti, a svaka nit će kopirati po jedan blok (particiju traženog segmenta memorije) veličine `BLOCK_SIZE` (poslednja nit možda manje od toga). Ignorirati eventualne greške i prekoračenja.

```
void* memcpy (void* destination, const void* source, size_t num);
void* par_memcpy (void* destination, const void* source, size_t num);
size_t BLOCK_SIZE = ...;
```

Rešenje

```
struct copy_task {
    void* dst;
    const void* src;
    size_t num;
};

void copy_block (void* task) {
    copy_task* t = (copy_task*)task;
    memcpy(t->dst, t->src, t->num);
}

void* par_memcpy (void* dest, const void* source, size_t num) {
    if (num<=0) return dest;
    int tail = num%BLOCK_SIZE;
    int numOfBlocks = num/BLOCK_SIZE + (tail?1:0);
    copy_task* tasks = new copy_task[numOfBlocks];
    for (int i=0; i<numOfBlocks; i++) {
        tasks[i].dst = (char*)dest + i*BLOCK_SIZE;
        tasks[i].src = (const char*)source + i*BLOCK_SIZE;
        tasks[i].num = (i<numOfBlocks-1 || tail==0) ? BLOCK_SIZE : tail;
        thread_create(&copy_block, &tasks[i]);
    }
    wait(NULL);
    return dest;
}
```

3. zadatak, prvi kolokvijum, mart 2018.

- [Postavka](#)
- [Rešenje](#)

Korišćenjem sistemskih poziva `fork`, `execlp` i `wait`, napisati program koji implementira jedan krajnje jednostavan interpreter komandne linije (engl. *command line interpreter*). Ovaj interpreter treba da učitava niz stringova razdvojenih belinama (razmacima ili novim redovima) sa standardnog ulaza, sve dok ne učitava string „q“ koji prekida njegov rad. Kada učitava svaki string, interpreter treba da pokrene proces nad programom zadatim stazom u tom stringu, sačeka njegov završetak, i pređe na sledeći string. Ukoliko kreiranje procesa nije uspelo, treba da ispiše poruku o grešci i pređe na sledeći. Pretpostavlja se da svaki string ima najviše 32 znaka. Podsetnik na bibliotečne funkcije i sistemske pozive koje se mogu koristiti:

- `scanf`: učitava sa standardnog ulaza; ukoliko se u formatizacionom specifikatoru, iza znaka `%`, napiše ceo broj, on označava maksimalan broj znakova koji će se učitati sa standardnog ulaza; na primer, `%32s` učitava string, ali ne duži od 32 znaka (i dodaje `'\0'` na kraj);
- `void wait(int pid)`: suspenduje pozivajući roditeljski proces dok se ne završi proces-dete sa zadatim PID; ako je vrednost argumenta 0, pozivajući proces se suspenduje dok se ne završe sve procesi-deca;
- `int strcmp(char*, char*)`: poredi dva data stringa i vraća 0 ako su jednaki.

Rešenje

```
#include <stdio.h>
```

```

char command[33];

void main () {

    while (true) {

        scanf("%32s",&command);
        if (strcmp(command,"q")==0) break;

        int pid = fork();
        if (pid<0)
            printf("Error executing %s.\n",command);
        else
            if (pid>0)
                wait(pid);
            else
                execlp(command);
    }
}

```

3. zadatak, prvi kolokvijum, april 2018.

- [Postavka](#)
- [Rešenje](#)

U nastavku je data donekle izmenjena i pojednostavljena specifikacija sistemskog poziva iz standardnog *POSIX thread* API (*Pthreads*):

`int pthread_create(pthread_t *thread, void *(*routine)(void *), void *arg)`: kreira novu nit nad funkcijom na koju ukazuje routine, dostavljajući joj argument `arg`; identifikator novokreirane niti smešta u lokaciju na koju ukazuje thread; vraća negativnu vrednost u slučaju greške, a 0 u slučaju uspeha.

Dat je potpis funkcije `fun`, pri čemu su `A`, `B`, `C` i `D` neki tipovi:

```
extern D fun (A a, B b, C c);
```

Potrebno je realizovati potprogram `fun_async` čiji će poziv izvršiti asinhroni poziv funkcije `fun`, tj. inicirati izvršavanje funkcije `fun` u posebnoj niti i potom odmah vratiti kontrolu pozivaocu. Po završetku funkcije `fun`, ta nit treba da pozove povratnu funkciju na koju ukazuje argument `cb` (engl. *callback*) i da joj dostavi vraćenu vrednost iz funkcije `fun`. Ignorisi greške.

```
typedef void (*CallbackD)(D);
void fun_async (A a, B b, C c, CallbackD cb);
```

Rešenje

```

extern D fun (A a, B b, C c);
typedef void (*CallbackD)(D);

struct fun_params { A a; B b; C c; CallbackD cb; };

void fun_async (A a, B b, C c, CallbackD cb) {
    fun_params* params = new fun_params;
    params->a = a; params->b = b; params->c = c;
    params->cb = cb;
    pthread_t pid;
    pthread_create(&pid,&fun_wrapper,params);
}

void fun_wrapper (void* params) {

```

```

fun_params* p = (fun_params*)params;
D d = fun(p->a,p->b,p->c);
CallbackD callback = p->cb;
delete p;
callback(d);
}

```

3. zadatak, prvi kolokvijum, jun 2018.

- [Postavka](#)
- [Rešenje](#)

U nastavku je data donekle izmenjena i pojednostavljena specifikacija dva sistemska poziva iz standardnog *POSIX thread* API (*Pthreads*). Obe ove funkcije vraćaju negativnu vrednost u slučaju greške, a 0 u slučaju uspeha.

- `int pthread_create(pthread_t *thread, void *(*start_routine)(void *), void *arg)`: kreira novu nit nad funkcijom na koju ukazuje `start_routine`, dostavljajući joj argument `arg`; identifikator novokreirane niti smešta u lokaciju na koju ukazuje `thread`;
- `int pthread_join(pthread_t thread, void **retval)`: čeka da se nit identifikovana sa `thread` završi (ukoliko se već završila, odmah vraća kontrolu); ako `retval` nije NULL, kopira povratnu vrednost funkcije `start_routine` koju je ta nit izvršavala u lokaciju na koju ukazuje `retval`.

Data je globalna kvadratna matrica `mat` celih brojeva (`int`), dimenzija $N \times N$. Potrebno je napisati potprogram `par_sum` koji treba da izračuna ukupan zbir svih elemenata matrice `mat`, ali na sledeći način: najpre treba da izračuna zbir svake, i -te vrste ove matrice, uporedo, u N uporednih niti (zbir jedne vrste izračunava jedna nit), i da taj zbir smesti u i -ti element jednog pomoćnog niza; potom treba da sabere ovako izračunate zbirove vrsta i vrati rezultat. Ignorirati eventualne greške i prekoračenje.

```

const int N = ...;
int mat[N][N];
int par_sum();

```

Rešenje

```

const int N = ...;
int mat[N][N];
int sums[N];
pthread_t pid[N];
typedef int Row[N];

void sum(Row* row) {
    int s = 0;
    for (int i = 0; i < N; i++)
        s += (*row)[i];
    sums[row - &mat[0]] = s;
}

// Wrapper, for type-casting only:
void* _sum(void* row) {
    sum((Row*)row);
    return NULL;
}

int par_sum() {
    for (int i = 0; i < N; i++)
        pthread_create(&pid[i], &_sum, &mat[i]);
    int s = 0;
    for (int i = 0; i < N; i++) {
        pthread_join(pid[i], NULL);

```

```

        s += sums[i];
    }
    return s;
}

```

3. zadatak, prvi kolokvijum, mart 2017.

- [Postavka](#)
- [Rešenje](#)

U nekom operativnom sistemu postoje sledeći sistemski pozivi:

- `int thread_create(void*(*)(void*),void*)`: kreira nit nad funkcijom na koju ukazuje prvi argument; ta funkcija prima jedan argument tipa `void*` i ne vraća rezultat; novokreirana nit poziva tu funkciju sa stvarnim argumentom jednakim drugom argumentu ovog sistemskog poziva; sistemski poziv vraća PID kreirane niti;
- `void wait(int pid)`: suspenduje pozivajuću roditeljsku nit dok se ne završi nit-dete sa zadatim PID; ako je vrednost argumenta 0, pozivajuća nit se suspenduje dok se ne završe sve niti-deca;
- `char getc(FILE* stream)`: iz fajla na čiji deskriptor ukazuje prvi argument učitava i vraća jedan znak; svaki poziv učitava naredni znak iz fajla; ukoliko je učitavanje stiglo do kraja fajla, funkcija vraća vrednost EOF.

U nekom programu već je otvoreno N ulaznih fajlova na čije deskriptore ukazuju elementi niza `streams`:

```

const int N = ..., M = ...;
FILE* streams [N];
char text [N][M];

```

Na jeziku C napisati deo programa koji u N uporednih niti-dece učitava najviše po M znakova u svaki red tabele `text`; svaka nit učitava po jedan red tabele iz različitog fajla, u i -ti red iz i -tog fajla. Kada se učitavanje u potpunosti završi, program treba da ispiše sve redove tabele `text` na standardni izlaz.

Rešenje

```

#include <stdio.h>
const int N = ..., M = ...;
FILE* streams[N];
char text[N][M];

void read_line (void* ptr) {
    int i = (FILE**)ptr-streams;
    FILE* stream = streams[i];

    char c = getc(stream);
    int j = 0;
    while ((c!=EOF) && (j<M)) {
        text[i][j] = c;
        j++;
        c = getc(stream);
    }

    if (j<M)
        text[i][j] = '\0';
    else
        text[i][M-1] = '\0';
}

void read_text () {
    int i;
    for (i=0; i<N; i++) {
        text[i][0] = '\0';
    }
}

```

```

    thread_create(read_line,&streams[i]);
}

wait(0);

for (i=0; i<N; i++)
    printf("%s\n",text[i]);
}

```

3. zadatak, prvi kolokvijum, mart 2016.

- [Postavka](#)
- [Rešenje](#)

U nastavku je data donekle izmenjena i pojednostavljena specifikacija dva systemska poziva iz standardnog *POSIX thread* API (Pthreads). Obe ove funkcije vraćaju negativnu vrednost u slučaju greške, a 0 u slučaju uspeha.

- `int pthread_create(pthread_t *thread, void *(*start_routine)(void *) ,void *arg)`: kreira novu nit nad funkcijom na koju ukazuje `start_routine`, dostavljaajući joj argument `arg`; identifikator novokreirane niti smešta u lokaciju na koju ukazuje `thread`;
- `int pthread_join(pthread_t thread, void **retval)`: čeka da se nit identifikovana sa `thread` završi (ukoliko se već završila, odmah vraća kontrolu); ako `retval` nije NULL, kopira povratnu vrednost funkcije `start_routine` koju je ta nit izvršavala u lokaciju na koju ukazuje `retval`.

Data je struktura `Node` koja predstavlja jedan čvor binarnog stabla. Korišćenjem datih systemskih poziva implementirati funkciju `createSubtree` koja rekurzivno kreira binarno stablo dubine `n` i vraća pokazivač na njegov koreni čvor, tako što u istoj niti kreira levo podstablo, a u novokreiranoj niti uporedo kreira desno podstablo. Ignorirati eventualne greške.

```

struct Node {
    Node () : leftChild(NULL), rightChild(NULL) {}
    Node *leftChild, *rightChild;
    ...
};

```

```
Node* createSubtree (int n);
```

Rešenje

```

// Wrapper, for type-casting only:
inline void* _createSubtree (void* n) {
    return createSubtree(*(int*)n);
}

Node* createSubtree (int n) {
    if (n<=0) return 0;
    Node* node = new Node();
    if (n<=1) return node;

    int n1 = n-1;
    // Create a new thread to create the right subtree:
    pthread_t pid;
    pthread_create(&pid,&_createSubtree,&n1);
    // Create the left subtree in this thread:
    node->leftChild = createSubtree(n1);
    // Join with the right-subtree child thread:
    pthread_join(pid,&node->rightChild);
    return node;
}

```


3. zadatak, prvi kolokvijum, maj 2016.

- [Postavka](#)
- [Rešenje](#)

U sistemu UNIX i sličnim sistemima sistemski poziv

```
int execvp(const char * filename, const char * const args[]);
```

radi isto što i poziv `execlp`, s tim što drugi argument predstavlja niz pokazivača na nizove znakova (*null-terminated strings*) koji predstavljaju listu argumenata poziva programa koji treba izvršiti. Po konvenciji, prvi argument pokazuje na naziv fajla samog programa koji se izvršava. Ovaj niz pokazivača mora da se završi elementom sa vrednošću `NULL`. Ovaj poziv vraća 0 u slučaju uspeha, a vrednost manju od 0 u slučaju greške.

Korišćenjem ovog sistemskog poziva, kao i sistemskog poziva `fork()`, realizovati sledeću funkciju:

```
int multiexec (int number, const char* filename, const char* const args[]);
```

Ova funkcija treba da kreira zadati broj (`number`) procesa-dece, i svaki od tih procesa-dece treba da izvršava isti program zadat u fajlu sa imenom `filename`, sa samo po jednim argumentom. Argumenti tih procesa-dece zadati su redom u prvim `number` elemenata niza `args`. Ova funkcija treba da vrati broj uspešno kreiranih procesa-dece (bez obzira na to da li su ti procesi uspešno izvršili `execvp`) i da odmah vrati kontrolu pozivaocu, ne čekajući da se ti procesi završe.

Rešenje

```
int multiexec (int number, const char* filename, const char* const args[]){
    int ret = 0;
    // Prepare the arguments for the children:
    const char* childArgs[3];
    childArgs[0] = filename;
    childArgs[2] = NULL;
    for (int i=0; i<number; i++) {
        childArgs[1] = args[i];
        // Create a child:
        int status = fork();
        if (status<0) continue; // fork failed
        if (status==0) // Child's context
            if (execvp(filename,childArgs)<0) exit();
        else // Parent's context
            ret++;
    }
    return ret;
}
```

3. zadatak, prvi kolokvijum, septembar 2015.

- [Postavka](#)
- [Rešenje](#)

Neki operativni sistem podržava sledeće sistemske pozive za upravljanje nitima:

- `fork()`: kreira novu nit-dete kao klon roditelja, poput Unix sistemskog poziva *fork* (samo što kreira nit, a ne proces); u kontekstu roditelja vraća ID kreirane niti-deteta, a u kontekstu deteta vraća 0;
- `exit()`: gasi pozivajuću nit;
- `wait(int pid)`: suspenduje pozivajuću nit sve dok se nit dete te niti sa datim ID ne završi (ne ugasi se); argument 0 suspenduje pozivajuću nit sve dok se sva njena deca ne završe.

Korišćenjem ovih sistemskih poziva, realizovati operaciju `cobegin()` koja prima dva argumenta, pokazivače na dve funkcije, i koja pokreće uporedno izvršavanje dve niti nad te dve funkcije, a vraća kontrolu pozivaocu tek kada se obe te niti (odnosno funkcije) završe.

```
void cobegin (void (*f)(), void (*g)());
```

Rešenje

```
void cobegin (void (*f)(), void (*g)()) {
    int id1 = 0, id2 = 0;
    if (id1 = fork())
        if (id2 = fork()) {
            wait(id1);
            wait(id2);
            return;
        } else {
            g();
            exit(0);
        }
    else {
        f();
        exit(0);
    }
}
```

4. zadatak, prvi kolokvijum, mart 2014.

- [Postavka](#)
- [Rešenje](#)

U sistemu UNIX i sličnim sistemima sistemski poziv

```
int execvp(const char * filename, char * const args[]);
```

radi isto što i poziv `execlp`, s tim što drugi argument predstavlja niz pokazivača na nizove znakova (*null-terminated strings*) koji predstavljaju listu argumenata poziva programa koji treba izvršiti. Po konvenciji, prvi argument pokazuje na naziv fajla samog programa koji se izvršava. Ovaj niz pokazivača mora da se završi elementom sa vrednošću `NULL`.

U nastavku je dat jedan neispravan program. Namera programera je bila da sastavi program koji prima jedan celobrojni argument (po konvenciji, u programu se on vidi kao drugi argument, pored naziva programa), i da, samo ukoliko je vrednost celobrojnog argumenta veća od 0, kreira proces-dete nad istim programom i sa za jedan manjom vrednošću svog celobrojnog argumenta, potom ispiše svoju vrednost argumenta i sačeka da se proces-dete završi, a onda se i sam završi. Proces-dete radi to isto (i tako rekursivno). Napisati ispravnu implementaciju ovog programa.

```
void main (int argc, char* const argv[]){
    if (argc<2) return; // Exception!

    // Get the argument value:
    int myArg = 0;
    sscanf(argv[1], "%d", &myArg);
    if (myArg<=0) return;

    // Prepare the arguments for the child:
    char childArg[10]; // The value of the second argument
    sprintf(childArg, "%d", myArg-1);
    char* childArgs[3];
    childArgs[0] = argv[0];
    childArgs[1] = childArg;
    childArgs[2] = NULL;

    // Create a child:
    execvp(argv[0], childArgs);
```

```
printf("%s\n",myArg);
wait(NULL);
}
```

Rešenje

```
void main (int argc, char* const argv[]){
    if (argc<2) return; // Exception!

    // Get the argument value:
    int myArg = 0;
    sscanf(argv[1], "%d", &myArg);
    if (myArg<=0) return;

    // Prepare the arguments for the child:
    char childArg[10]; // The value of the second argument
    sprintf(childArg, "%d", myArg-1);
    char* childArgs[3];
    childArgs[0] = argv[0];
    childArgs[1] = childArg;
    childArgs[2] = NULL;

    // Create a child:
    int status = fork();
    if (status<0) exit(); // Exception!
    if (status==0) // Child's context
        execvp(argv[0], childArgs);
    else { // Parent's context
        printf("%d\n", myArg);
        wait(NULL);
    }
}
```

4. zadatak, prvi kolokvijum, mart 2013.

- [Postavka](#)
- [Rešenje](#)

U nekom operativnom sistemu postoje sledeći sistemski pozivi:

- `int thread_create(void (*)(int), int)`: kreira nit nad funkcijom na koju ukazuje prvi argument; ta funkcija prima jedan celobrojni argument i ne vraća rezultat; novokreirana nit poziva tu funkciju sa stvarnim argumentom jednakim drugom argumentu ovog sistemskog poziva; sistemski poziv vraća PID kreirane niti;
- `void wait(int pid)`: suspenduje pozivajuću roditeljsku nit dok se ne završi nit-dete sa zadatim PID.

Data je celobrojna konstanta N i funkcija f koja prima dva celobrojna argumenta:

```
int f(int, int);
```

Na jeziku C napisati program koji u $N*N$ uporednih niti-dece izračunava vrednost funkcije $f(i, j)$, $i=0, \dots, N-1$, $j=0, \dots, N-1$, svaku vrednost u po jednoj niti. Kada sve niti-deca završe, program treba da ispiše sve dobijene vrednosti redom na standardni izlaz.

Rešenje

```
const int N = ...;
int f(int, int);
#include <iostream.h>

struct f_params {
```

```

    int i, j, r, pid;
};

f_params params[N*N];

void f_wrapper (int index) {
    params[index].r=f(params[index].i,params[index].j);
}

void main () {
    for (int i=0; i<N; i++)
        for (int j=0; j<N; j++) {
            integer ind = i*N+j;
            params[ind].i=i;
            params[ind].j=j;
            params[ind].pid=thread_create(f_wrapper,ind);
        }
    for (int k=0; k<N*N; k++) {
        wait(params[k].pid);
        cout<<params[k].r<<' ';
    }
}

```

4. zadatak, prvi kolokvijum, april 2013.

- [Postavka](#)
- [Rešenje](#)

U nekom operativnom sistemu postoji sledeći sistemski poziv:

```
int thread_create(void (*)(void*), void*);
```

koji kreira nit nad funkcijom na koju ukazuje prvi argument. Ta funkcija prima jedan netipizirani pokazivač i ne vraća rezultat. Novokreirana nit poziva tu funkciju sa stvarnim argumentom jednakim drugom argumentu ovog sistemskog poziva. Sistemski poziv vraća PID kreirane niti.

Korišćenjem ovog sistemskog poziva realizovati klasu **Thread** poput one u školskom jezgru, sa sledećim interfejsom:

```

class Thread {
public:
    void start ();
    virtual ~Thread ();
protected:
    Thread ();
    virtual void run () {}
};

```

Destruktor ove klase treba da sačeka da se nit predstavljena ovim objektom završi. U tu svrhu postoji sistemski poziv `wait(int pid)` koji suspenduje pozivajuću nit dok se nit sa datim PID ne završi.

Rešenje

```

class Thread {
public:
    void start ();
    virtual ~Thread ();
protected:
    Thread () : myPID(0) {}
    virtual void run () {}
private:

```

```

static void thread (void*);
int myPID;
};

void Thread::thread (void* p) {
    Thread* thr = (Thread*)p;
    if(thr) thr->run();
}

void Thread::start () {
    if (myPID==0) myPID = thread_create(thread,this);
}

Thread::~Thread () {
    if (myPID>0) wait(myPID);
}

```

4. zadatak, prvi kolokvijum, septembar 2013.

- [Postavka](#)
- [Rešenje](#)

U nekom operativnom sistemu postoji sistemski poziv

```
int thread_create(void (*)(void*), void*)
```

koji kreira nit nad funkcijom na koju ukazuje prvi argument. Ta funkcija prima jedan argument tipa `void*` i ne vraća rezultat. Novokreirana nit poziva tu funkciju sa stvarnim argumentom jednakim drugom argumentu ovog sistemskog poziva. Stablo u kome svaki čvor sadrži jedan podatak tipa `Data` i proizvoljno mnogo potomaka realizovano je pomoću sledeće strukture čvora:

```

struct Node {
    Data data;
    Node* children_head;
    Node* next_sibling;
};

extern void process(Data*);

```

Potomci datog čvora uvezani su u jednostruko ulančanu listu na čiji prvi element ukazuje polje `children_head`. Na sledećeg u listi ukazuje polje `next_sibling`. Procedura `process` vrši obradu datog podatka u jednom čvoru.

Na jeziku C napisati proceduru `visit_node` koja, kada se pozove za koren stabla (ili bilo kog podstabla), obilazi to stablo rekursivno na sledeći način: najpre obradi podatak u tom čvoru pozivom procedure `process`, a zatim kreira po jednu nit koja obilazi podstablo svakog podčvora počev od drugog u listi (ako postoji), a u istoj niti obilazi podstablo prvog podčvora (ako postoji).

Rešenje

```

void visit_node (void* n) {
    if (n==0) return;
    Node* nd = (Node*)n;
    process(&nd->data);
    if (nd->children_head) {
        for (Node* p=nd->children_head->next_sibling; p; p=p->next_sibling)
            thread_create(&visit_node,p);
        visit_node(nd->children_head);
    }
}

```

4. zadatak, prvi kolokvijum, mart 2012.

- [Postavka](#)
- [Rešenje](#)

Dat je sledeći kod koji koristi školsko jezgro.

```
class TreeNode {
public:
    TreeNode* getLeftChild();
    TreeNode* getRightChild();
    void process();
    ...
};

class TreeVisitor : public Thread {
public:
    TreeVisitor (TreeNode* root) : myRoot(root) {}
protected:
    virtual void run();
    void visit(TreeNode*);
private:
    TreeNode* myRoot;
};

void TreeVisitor::run () {
    visit(myRoot);
}

void TreeVisitor::visit (TreeNode* node) {
    if (node==0) return;
    TreeNode* rn = node->getRightChild();
    if (rn) (new TreeVisitor(rn))->start();
    node->process();
    visit(node->getLeftChild());
}

void userMain () {
    TreeNode* root = ...;
    Thread* thr = new TreeVisitor(root);
    thr->start();
}
```

Napisati C kod koji radi isto, odnosno obilazi dato stablo rekursivnim kreiranjem niti koje obilaze podstabla, samo korišćenjem nekog sistema u kome se niti kreiraju i odmah pokreću kao parametrizovani pozivi funkcije sledećim sistemskim pozivom:

```
void create_thread (void(*thread_body)(void*), void* param);
```

Klasa `TreeNode` implementirana je sledećim C kodom:

```
struct TreeNode;
TreeNode* getLeftChild (TreeNode*);
TreeNode* getRightChild (TreeNode*);
void process (TreeNode*);
```

Rešenje

```
void visit (void* nd) {
    TreeNode* node = (TreeNode*)nd;
```

```

    if (node==0) return;
    TreeNode* rn = getRightChild(node);
    if (rn) create_thread(&visit,rn);
    process(node);
    visit(getLeftChild(node));
}

void main () {
    TreeNode* root = ...;
    create_thread(&visit,root);
}

```

4. zadatak, prvi kolokvijum, maj 2012.

- [Postavka](#)
- [Rešenje](#)

Klasa Node, čija je delimična definicija data dole, predstavlja čvor binarnog stabla. Funkcije `getLeftChild()` i `getRightChild()` vraćaju levo, odnosno desno podstablo datog čvora (tačnije, njegov levi i desni čvor-potomak).

```

class Node {
public:
    Node* getLeftChild();
    Node* getRightChild();
    ...
};

```

Potrebno je prebrojati čvorove u stablu rekursivnim obilaskom datog binarnog stabla korišćenjem uporednih niti na sledeći način. Ako neka nit trenutno obilazi neki čvor, onda ona treba da napravi novu nit-potomka koja će obići desno podstablo tog čvora, a sama ta nit će nastaviti sa obilaskom levog podstabla tog čvora, i tako rekursivno.

Realizovati globalnu operaciju `size(Node* root)` koja prebrojava čvorove i vraća broj čvorova u stablu sa datim korenim čvorom na opisani način. Niti treba kreirati sistemskim pozivom `fork()` za koga treba pretpostaviti da ima istu sintaksu i značenje povratne vrednosti kao i istoimeni Unix sistemski poziv, samo što umesto procesa kreira nit u istom adresnom prostoru roditelja: novokreirana nit ima istu poziciju u izvršavanju kao i roditeljska nit, deli isti adresni prostor, samo što poseduje sopstveni kontrolni stek koji inicijalno predstavlja identičnu kopiju roditeljskog steka u trenutku poziva `fork()`. Sistemski poziv `wait(null)` suspenduje pozivajuću nit sve dok se ne završe sve niti-potomci, a poziv `exit()` gasi pozivajuću nit.

Rešenje

```

// Helper recursive function: traverse the tree, compute its size,
// and store its size in sz (sz is external to the thread's stack):

```

```

void size_ (Node* node, int& sz) {
    Node* ln = node->getLeftChild();
    int lsz = 0; // left subtree size
    Node* rn = node->getRightChild();
    int rsz = 0; // right subtree size

    if (rn) {
        if (fork()==0) {
            size_(rn,&rsz);
            exit();
        }
    }

    if (ln) size_(ln,&lsz);
}

```

```

    wait(null); // wait for all descendants to complete
    *sz = lsz+rsz+1; // compute the cumulative size and return to the caller
}

int size (Node* node) {
    if (node==0) return 0;
    int sz = 0;
    size_(node,&sz);
    return sz;
}

```

4. zadatak, prvi kolokvijum, septembar 2012.

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu niti se kreiraju sistemskim pozivom `fork()` koji ima istu sintaksu i značenje povratne vrednosti kao i istoimeni Unix sistemski poziv, samo što umesto procesa kreira nit u istom adresnom prostoru roditelja: novokreirana nit ima istu poziciju u izvršavanju kao i roditeljska nit, deli isti adresni prostor, samo što poseduje sopstveni kontrolni stek koji inicijalno predstavlja identičnu kopiju roditeljskog steka u trenutku poziva `fork()`. Sistemski poziv `exit()` gasi pozivajuću nit.

Korišćenjem ovih sistemskih poziva, realizovati funkciju:

```
int create_thread (void (*fun)(void*), void* p);
```

koja kreira nit nad potprogramom na koji ukazuje prvi argument, tako što taj potprogram poziva u kontekstu novokreirane niti sa datim argumentom `p`, i završava kreiranu nit nakon završetka tog potprograma. Ukoliko sistemski poziv `fork()` vrati grešku, isti kod greške treba vratiti i pozivaocu funkcije `create_thread()`. Ukoliko je kreiranje niti uspešno, pozivaocu funkcije `create_thread()` treba vratiti rezultat poziva `fork()` (ID kreirane niti).

Rešenje

```

int create_thread (void (*fun)(void*), void* p) {
    int ret = fork();
    if (ret<0) return ret; // Error
    if (ret>0) return ret; // Parent thread context
    // Child context (ret==0):
    (*fun)(p);
    exit();
}

```

4. zadatak, prvi kolokvijum, maj 2011.

- [Postavka](#)
- [Rešenje](#)

U nekom operativnom sistemu sistemski poziv `clone()` pravi novu nit (*thread*) kao klon roditeljske niti, sa istovetnim kontekstom izvršavanja i u istom adresnom prostoru kao što je i roditeljska nit – isto kao i `fork()`, samo što pravi nit u istom adresnom prostoru, a ne proces. Ovaj poziv vraća 0 u kontekstu niti-deteta, a vrednost `>0` koja predstavlja ID kreirane niti u kontekstu niti-roditelja. Vraćena vrednost `<0` označava neuspešan poziv. Sistemski poziv `terminate(int)` gasi nit sa datim ID. Korišćenjem ovog sistemskog poziva realizovati klasu `Thread` sa istim interfejsom kao u školskom jezgru (kreiranje niti nad virtuelnom funkcijom `run()` i pokretanje niti pozivom funkcije `start()`).

Rešenje

```

class Thread {
public:
    int start ();

```



```
protected:
    Thread () : myID(0) {}
    virtual void run() {}
private:
    int myID;
};

int Thread::start () {
    int id = clone();
    if (id<0) return id; // Failure
    if (id>0) { // Successful start in the parent's context
        myID=id;
        return 0;
    }
    // Child context:
    run();
    while (myID==0); // Busy-wait until the parent writes myID
    terminate(myID);
}
```

4. zadatak, prvi kolokvijum, septembar 2011.

- [Postavka](#)
- [Rešenje](#)

Klasa Node čija je delimična definicija data dole predstavlja čvor binarnog stabla. Funkcije `getLeftChild()` i `getRightChild()` vraćaju levo, odnosno desno podstablo datog čvora (tačnije, njegov levi i desni čvor-potomak). Funkcija `visit()` obavlja nekakvu obradu čvora.

```
class Node {
public:
    Node* getLeftChild();
    Node* getRightChild();
    void visit();
    ...
};
```

Potrebno je obilaziti dato binarno stablo korišćenjem uporednih niti na sledeći način. Ako neka nit trenutno obilazi neki čvor, onda ona treba da napravi novu nit-potomka koja će obići desno podstablo tog čvora, a sama ta nit će obraditi dati čvor i nastaviti sa obilaskom levog podstabla tog čvora, i tako rekursivno.

Realizovati globalnu operaciju `visit(Node* root)` koja vrši opisani obilazak stabla sa datim korenim čvorom. Niti treba kreirati sistemskim pozivom `fork()` za koga treba pretpostaviti da ima istu sintaksu i semantiku kao i istoimeni Unix sistemski poziv, samo što umesto procesa kreira nit u istom adresnom prostoru roditelja.

Rešenje

```
int visit (Node* node) {
    if (node==0) return;

    Node* ln = node->getLeftChild();
    Node* rn = node->getRightChild();
    if (rn) {
        if (fork()==0) {
            visit(rn);
            exit();
            // it is assumed that exit() kills only this thread,
            // not the entire process (all threads)
        }
    }
}
```

```

}

node->visit();

if (ln) visit(ln);
}

```

4. zadatak, prvi kolokvijum, april 2009.

- [Postavka](#)
- [Rešenje](#)

U jezgri nekog operativnog sistema definisani su sledeći makroi, strukture i funkcije:

```

#define saveAll ... // sve registre osim pc i sp čuva na vrhu steka (menja sp)
#define restoreAll ... // sve registre osim pc i sp restaurira sa steka (menja sp)
#define getSP ... // vraća vrednost sp registra
#define setSP(expr) ... // izračunava izraz i upisuje ga u registar sp
void copy(int* dst, int* src, int n); // kopira niz od n reči sa src na dst
struct PCB {
    int sp; // sačuvana vrednost sp registra
    ... // NIJE DOZVOLJENO PRAVITI PRETPOSTAVKE U VEZI SA OSTALIM POLJIMA PCB STRUKTURE
};
PCB* running; // The running process

```

Korišćenjem ovih makroa, funkcije i struktura, napisati kod funkcija `setjump()` i `longjump()` koje će se koristiti u funkciji `dispatch()` na sledeći način:

```

void dispatch() {
    lock();
    if (setjump() == 0) {
        Scheduler::put(running);
        running = Scheduler::get();
        longjump();
    } else unlock();
}

```

Poznato je da se radi o RISC procesoru koji pored SP i PC registara ima još 16 opštenamenskih registara. Stek raste ka nižim adresama i SP pokazuje na poslednju zauzetu lokaciju na vrhu steka. Povratnu vrednost funkcije uvek vraćaju kroz registar R0, a prevodilac generiše kod tako da sadržaj svih ostalih registara ostane neizmenjen i nakon poziva funkcije (pod uslovom da se ne menja stek). Parametri se funkcijama prenose preko steka, a za čuvanje međurezultata dati kompajler koristi najmanji mogući broj registara. Ako taj broj premašuje broj registara, onda za tu namenu koristi i lokacije na steku. Smatrati da su svi podaci i sve adrese veličine tipa `int` i da je adresibilna jedinica jedna reč veličine tipa `int`. U rešenju nije dozvoljeno korišćenje asemblerskog koda.

Rešenje

```

#define saveAll ... //sve registre osim pc i sp čuva na vrhu steka
#define restoreAll ... //sve registre osim pc i sp restaurira sa steka
#define getSP ... //vraća vrednost sp registra
#define setSP(expr) ... //izračunava izraz i upisuje ga u registar sp
void copy(int* dst, int* src, int n); //kopira niz od n reči sa src
                                     //na dst
struct PCB {
    int* sp; // sačuvana vrednost sp registra
    ... // NIJE DOZVOLJENO PRAVITI PRETPOSTAVKE U VEZI SA OSTALIM POLJIMA
        //PCB STRUKTURE
};
PCB* running; // The running process

```

```

int setjump(){ // R0 is used to access fields of PCB pointed by running,
// since its content will be overwritten
// with the return result at the end(return 0)
// compiler will not save it on the stack
    saveAll;
    running->sp = getSP;
    setSP(running->sp-1);
    copy((int*)running->sp-1, (int*)running->sp+16, 1);
    return 0;
}

int longjump(){
    setSP(running->sp);
    restoreAll;
    return 1;
}

```

5. zadatak, prvi kolokvijum, april 2009.

- [Postavka](#)
- [Rešenje](#)

Dat je sledeći program:

```

#include <stdio.h>
#define N 3

void main() {
    int i, f = 0, s = 0;
    for (i = 0; i < N; i++) {
        f = f || fork();
        s += i;
    }
    if (f) exit(0);
    printf("%d ", s);
}

```

Pod pretpostavkom da su svi sistemski pozivi uspeli, da je funkcija `printf` bezbedna za poziv u konkurentnim procesima (svaki poziv se izvršava izolovano i bez interakcije sa pozivima iz drugih procesa, tj., uporedne pozive ove funkcije iz različitih procesa sistem sekvencijalizuje), i da uvek koristi isti standardni izlaz, prikazati ispis koji se dobije pokretanjem datog programa? Odgovor obrazložiti.

Rešenje

```

#include <stdio.h>
#define N 3

void main () {
    int i, f = 0, s = 0;
    for (i=0; i<N; i++) {
        f = f || fork();
        s += i;
    }
    if (f) exit(0);
    printf("%d ",s);
}

```

U tri prolaza petlje kreira se 8 procesa: u prvom prolazu od 1 nastaju 2, u drugom od svakog od ovih nastaju po 2,

što je ukupno 4 i u trećem prolazu od svakog od ova 4 nastaju po 2, što je ukupno 8. Svaki od njih će računati sumu s u svom adresnom prostoru, koja će imati vrednost $0+1+2 = 3$.

Međutim, kako pri pozivu `fork()` samo jedan proces dobije povratnu vrednost 0, jedan od prva dva procesa će u f izračunati 1, a drugi 0. Zbog logičke funkcije „ili“, svi procesi koji nastanu od procesa u kojem je `f = 1` će takođe imati `f = 1` i zbog toga nikada neće stići do ispisa. Onaj koji ima vrednost `f = 0` se na isti način deli na dva procesa, gde f u jednom procesu ima vrednost 1, a u drugom 0. Kao i u prethodnom slučaju, jedini procesi koji imaju šansu da dođu do ispisa su oni koji nastaju od onog koji u f ima vrednost 0. Dalje posmatramo proces koji u f ima vrednost 0. Taj proces se još jednom deli na dva od kojih samo jedan u f ima vrednost 0 i samo taj proces će pri izvršavanju doći do ispisa, što znači da će biti ispisana samo jedna suma, 3.

Rezonovanje bi se moglo nastaviti na isti način dalje, i upotrebom indukcije bi se moglo pokazati da će za proizvoljnu vrednost N biti ispisana samo jedna suma brojeva od 1 do N.

4. zadatak, prvi kolokvijum, april 2008.

- [Postavka](#)
- [Rešenje](#)

Dat je sledeći program:

```
#include <stdio.h>
#define N 3
int pid[N];

void main () {
    int i;
    for (i=0; i<N; i++) pid[i]=0;
    for (i=0; i<N; i++) {
        if (pid[i]==0) pid[i]=fork(); // if1
        if (pid[i]==0)                // if2
            for (j=i+1; j<N; j++) pid[j]=1;
    }
}
```

Pod pretpostavkom da su svi sistemski pozivi uspešni, koliko će ukupno procesa biti kreirano (direktno ili indirektno) od strane jednog procesa početno kreiranog nad ovim programom, uključujući i taj jedan početni proces? Odgovor obrazložiti.

Rešenje

Odgovor: 4

Početni proces kreira tri procesa-potomka, za svaku iteraciju (spoljne) `for` petlje (`i=0, 1, 2`). U svakoj od tih iteracija, naredba `if` označena sa `if1` izvršava svoju *then* granu, jer je `pid[i]` uvek 0 (tako je inicijalizovan). U tom procesu *then* grana naredbe `if2` se ne izvršava, jer je `pid[i]` tada različit od 0 (pošto je `fork()` vratio ID kreiranog procesa). U svakom od kreiranih procesa-potomaka, `fork()` vraća 0, pa je u naredbi `if2` `pid[i]==0`, odnosno izvršava se ugnedžena `for` petlja (po `j`) koja u sve elemente niza `pid` iza `i`-tog upisuje 1. Zbog toga se u ovim procesima-potomcima, u svim narednim iteracijama glavne `for` petlje (po `i`), neće izvršiti `fork()` u naredbi `if1`, pa oni više neće kreirati svoje potomke.

5. zadatak, prvi kolokvijum, april 2007.

- [Postavka](#)
- [Rešenje](#)

Dat je sledeći program:

```
#include <stdio.h>
#define N 3
int pid[N];
```

```
void main () {
    int i;
    for (i=0; i<N; i++) pid[i]=0;
    for (i=0; i<N; i++) pid[i]=fork();
    for (i=0; i<N; i++) printf("%d ",pid[i]);
}
```

Pod pretpostavkom da su svi sistemski pozivi uspešni, koliko ukupno nula ispisuju svi procesi kreirani od strane jednog procesa nad ovim programom? Odgovor obrazložiti.

Rešenje

```
#include <stdio.h>
#define N 3
int pid[N];

void main () {
    int i;
    for (i=0; i<N; i++) pid[i]=0;
    for (i=0; i<N; i++) pid[i]=fork();
    for (i=0; i<N; i++) printf("%d ",pid[i]);
}
```

U tri prolaza petlje u sredini kreiraju se 8 procesa: u prvom prolazu od 1 nastaju 2, u drugom od svakog od ovih nastaju po 2, što je ukupno 4 i u trećem prolazu od svakog od ova 4 nastaju po 2, što je ukupno 8. Svaki od njih će imati sopstveni niz pid i ispisati 3 broja. To je ukupno 24 ispisana broja.

Posle svakog poziva `fork()` ostaju po dva identična procesa (roditelj i novokreirani potomak) koji se razlikuju samo po rezultatu `fork()` funkcije. U prvom prolazu kroz petlju ostaju dva procesa, jedan sa `pid[0]=0` i drugi sa `pid[0]>0`. Pošto su u kontrolnim strukturama ova dva procesa identična, to znači da će od svakog od njih nastati isti broj novih procesa, odnosno da će polovina svih procesa imati `pid[0]=0`, a druga `>0`. Dalje, problem možemo raščlaniti na dva koji imaju za 1 manju dimenziju problema (niz sa elementima `pid[1]` i `pid[2]`). To znači da će, iz istog razloga kao i u prethodnom slučaju, polovina procesa imati `pid[1]=0`, a druga `>0`. Ovakvim razmatranjem ili direktnim brojanjem dolazi se da je polovina ispisanih brojeva jednaka nuli, odnosno $24/2 = 12$ nula.

6. zadatak, prvi kolokvijum, april 2006.

- [Postavka](#)
- [Rešenje](#)

U nekom operativnom sistemu postoji sistemska usluga kreiranja niti koja se iz jezika C/C++ poziva pomoću bibliotečne funkcije `create_thread()`:

```
typedef unsigned long PID; // Process ID
PID create_thread (void (*body)(void*), void* arg);
```

Ovaj sistemski poziv kreira nit nad C funkcijom na koju ukazuje prvi argument `body`, pri čemu se pozivu te funkcije kao argument dostavlja pokazivač `arg`.

Za ovaj operativni sistem realizuje se biblioteka za konkurentno izvršavanje C++ programa koja koncept niti podržava klasom `Thread`. Ova klasa realizuje koncept niti kao u školskom jezgru i ima sledeći interfejs:

```
class Thread {
public:
    void start (); // Starts the thread
protected:
    Thread ();
    virtual void run() {} // The body of the thread
private:
}
```

```
PID pid;  // PID of system thread
};
```

tako da se korisničke niti kreiraju kao u sledećem primeru:

```
class Robot : public Thread {
...
    virtual void run ();
};
```

```
Robot* r = new Robot;
r->start();
```

Preslikavanje niti iz korisničke aplikacije na niti operativnog sistema je jedan-na-jedan. Korišćenjem navedenog sistemskog poziva, realizovati operaciju `Thread::start()` (uz odgovarajuće dodatne delove koda, ako je potrebno).

Rešenje

U `protected` sekciju klase `Thread` potrebno je dodati sledeći metod:

```
class Thread{
...
protected:
    static void starter(void*);
...
}

void Thread::starter(void* toStart){
    Thread* t = (Thread*)toStart;
    if (t) t->run();
}
```

U tom slučaju implementacija metoda `start` izgleda:

```
void Thread::start(){
    pid = create_thread(&starter, this);
}
```

Interfejs niti

2. zadatak, kolokvijum, avgust 2021.

- [Postavka](#)
- [Rešenje](#)

Školsko jezgro proširuje se nestatičkom funkcijom `Thread::join()` koju sme da pozove samo roditeljska nit date niti da bi sačekala da se data nit-dete završi; ukoliko ovu operaciju pozove neka druga nit koja nije roditelj date niti, ova funkcija vraća grešku (-1). Kada jezgro pravi novu nit, poziva funkciju `Thread::wrapper` koja obavlja sve potrebne radnje pre poziva funkcije `Thread::run` (u kontekstu napravljene niti) i nakon povratka iz nje:

```
void Thread::wrapper (Thread* toRun) {
    ...
    unlock();
    toRun->run();
    lock();
    ...
}
```

Precizno navesti sve izmene i dopune koje je potrebno napraviti u klasi `Thread` i implementirati operaciju `Thread::join`. Uzeti u obzir to da nit-roditelj može pozvati `join` više puta, pri čemu se samo pri prvom pozivu eventualno može zaustaviti dok nit-dete ne završi, svi pozivi nakon toga su neblokirajući. Problem gašenja niti i brisanja objekta klase `Thread`, kao i sinhronizacije potrebne za to ne treba rešavati u ovom zadatku.

Rešenje

U klasi `Thread` treba uraditi sledeće izmene:

- Dodati privatan član: `Thread* parent`; ukazuje na roditeljsku nit; inicijalizuje se na `Thread::runningThread` u konstruktoru klase `Thread`;
- Dodati privatne članove tipa `bool`, inicijalizovane na `false` u konstruktoru klase `Thread`: `complete` koji ukazuje na to da je ova nit završila i `isParentWaiting` koji ukazuje na to da roditeljska nit čeka na završetak ove niti;
- U funkciji `Thread::wrapper` uraditi sledeće dopune:

```
void Thread::wrapper (Thread* toRun) {
    ...
    unlock();
    toRun->run();
    lock();
    toRun->complete = true;
    if (toRun->isParentWaiting) {
        toRun->isParentWaiting = false;
        Scheduler::put(toRun->parent);
    }
    ...
}
```

- Dodati javnu funkciju članicu:

```
int Thread::join () {
    if (!this || this->parent!=Thread::runningThread) return -1;
    if (this->complete) return 0;
    lock();
    this->isParentWaiting = true;
    if (setjmp(Thread::runningThread->context)==0) {
        Thread::runningThread = Scheduler::get();
        longjmp(Thread::runningThread->context,1);
    }
}
```

```
unlock();
return 0;
}
```

3. zadatak, prvi kolokvijum, april 2017.

- [Postavka](#)
- [Rešenje](#)

U standardnom jeziku C++ postoje, između ostalog, sledeći elementi podrške za niti:

- `std::thread`: klasa kojom se predstavljaju niti; nova nit se pokreće odmah po kreiranju nekog objekta ove klase;
- `std::thread::thread`: konstruktor klase `std::thread` koji kreira nit nad funkcijom na koju ukazuje prvi argument; ta funkcija prima jedan argument i ne vraća rezultat; novokreirana nit poziva datu funkciju sa stvarnim argumentom jednakim drugom argumentu konstruktora;
- `std::thread::join`: suspenduje pozivajuću nit dok se ne završi nit čija se ova funkcija poziva;
- `thread_local`: specifikator životnog veka i načina alokacije koji se navodi u deklaraciji objekta; objekti ove kategorije životnog veka alociraju se kad god se kreira nova nit i nestaju sa završetkom te niti; svaka kreirana nit, uključujući i „glavnu“ nit kreiranu nad pozivom funkcije `main`, ima svoju sopstvenu kopiju (instancu) ovog objekta, različitu od instanci tog objekta u drugim nitima.

Precizno navesti šta sve može da ispiše sledeći C++ program (izlazne operacije ispisa na standardni izlaz su atomične):

```
thread_local int i=0;
void f (int id) {
    i=id;
    ++i;
    std::cout<<i;
}
void main() {
    i=9;
    std::thread t1(f,1);
    std::thread t2(f,2);
    std::thread t3(f,3);
    t1.join();
    t2.join();
    t3.join();
    std::cout<<i<<std::endl;
}
```

Rešenje

2349, ili 2439, ili 3249, ili 3429, ili 4239, ili 4329 i ništa više osim toga.

3. zadatak, prvi kolokvijum, jun 2017.

- [Postavka](#)
- [Rešenje](#)

Data je klasa `Thread` koja implementira niti kao u školskom jezgru, ali samo sa jednom mogućnošću konstrukcije objekata te klase, kako je dato u njenom interfejsu dole (samo objektno orijentisani API za kreiranje niti).

1. Realizovati klasu `ThreadFnCaller` koja omogućava kreiranje niti nad zadatom globalnom funkcijom koja prima jedan argument tipa `void*` i ne vraća rezultat, tako da ta nit izvršava zadata funkciju sa zadatim argumentom.
2. Napisati fragment koda koji kreira `N` niti nad funkcijom `fn`, pri čemu `i`-ta nit izvršava tu funkciju sa argumentom koji je dat u `i`-tom elementu niza `args`.

```
class Thread {
public:
```



```

Thread ();
void start ();
virtual void run () {}
};

```

Rešenje

1.

```

class ThreadFnCaller : public Thread {
public:
    ThreadFnCaller (void (*fn)(void*), void* arg) : myFn(fn), myArg(arg) {}
    virtual void run () { myFn(myArg); }
private:
    void (*myFn)(void*);
    void* myArg;
};

```

2.

```

for (int i=0; i<N; i++) (new ThreadFnCaller(fn,args[i]))->start();

```

3. zadatak, prvi kolokvijum, septembar 2016.

- [Postavka](#)
- [Rešenje](#)

Dat je neki veliki jednodimenzioni niz `data` veličine `N`, čiji su elementi tipa `Data` (deklaracije dole). Jedan element niza obrađuje procedura `processData`. Ovaj niz potrebno je obraditi pomoću `n` uporednih niti (procedura `parallelProcessing`), gde je `n` zadati parametar, tako što svaka od tih uporednih niti iterativno obrađuje približno isti broj elemenata ovog velikog niza. Drugim rečima, niz treba particionisati na `n` disjunktnih podnizova, što približnije jednakih, i te particije obrađivati uporedo. Implementaciju dati u školskom jezgru.

```

const int N = ...;
class Data;
Data data[N];
void processData(Data*);
void parallelProcessing (int n);

```

Rešenje

```

class DataProcessor : public Thread {
public:
    DataProcessor (int offs, int sz) : offset(offs), size(sz) {}
    virtual void run ();
private:
    int offset, size;
};

void DataProcessor::run () {
    int end = this->offset+this->size;
    for (int i=this->offset; i<end; i++)
        processData(&data[i]);
}

void parallelProcessing (int n) {
    int chunk = N/n;
    int offset = 0;
    for (int i=0; i<n; i++) {
        int myChunk = chunk;

```

```

        if (i<N%n) myChunk++;
        new DataProcessor(offset,myChunk)->start();
        offset+=myChunk;
    }
}

```

3. zadatak, prvi kolokvijum, mart 2015.

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu sistemski poziv `fork()` kreira nit – klon pozivajuće niti, sa iskopiranim celokupnim stekom, slično istoimenom sistemskom pozivu na sistemu Unix (osim što se ovde radi o nitima, a ne procesima).

Dole je dat program čija je zamisao da izvršava dve uporedne niti, jednu koja učitava znak po znak sa stadardnog ulaza i taj znak prenosi kroz promenljivu `c` drugoj niti, koja taj primljeni znak ispisuje na standardni izlaz, i tako neograničeno.

1. Precizno objasniti problem koji ovaj program ima i ispraviti taj problem.
2. Prepraviti samo funkciju `pipe()` tako da se umesto jednog para niti koje vrše razmenu znakova. formiraju dva para takvih niti; svaki par niti predstavlja odvojeni „tok“. pa je potrebno definisati dva para promenljivih `c` i `flag` (npr. `c1`, `c2`, `flag1` i `flag2`).

```

#include <iostream>

void writer (char* c, int* flag) {
    while (1) {
        while (*flag==1);
        cin>>(*c);
        *flag = 1;
    }
}

void reader (char* c, int* flag) {
    while (1) {
        while (*flag==0);
        cout<<(*c);
        *flag = 0;
    }
}

void pipe () {
    char c;
    int flag = 0;
    if (fork())
        writer(&c,&flag);
    else
        reader(&c,&flag);
}

void main () {
    pipe();
}

```

Rešenje

1. Problem je što su promenljive `flag` i `c` koje bi trebalo da budu deljene (zajedničke) između niti, jer preko njih treba da razmenjuju podatke i sinhronizuju se, definisane kao automatske (alociraju se na steku), što znači da zapravo neće biti deljene, već će svaka nit imati svoju instancu ovih promenljivih (na svom steku), pošto

svaka nit ima svoj zaseban stek, pa razmene zapravo neće ni biti. Rešenje je prosto deklarirati ih kao statičke (`static`).

2.

```
void pipe () {
    static char c1, c2;
    static int flag1 = 0, flag2 = 0;

    if (fork())
        writer(&c1,&flag1);
    else
        if (fork())
            reader(&c1,&flag1);
        else
            if (fork())
                writer(&c2,&flag2);
            else
                reader(&c2,&flag2);
}
```

3. zadatak, prvi kolokvijum, maj 2015.

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu sistemski poziv `fork()` kreira nit – klon pozivajuće niti, sa iskopiranim celokupnim stekom, slično istoimenom sistemskom pozivu na sistemu Unix (osim što se ovde radi o nitima, a ne procesima). Dole je dat (neispravan) program čija je zamisao da izvršava dva para uporednih niti, pri čemu u svakom paru jedna nit učitava znak po znak sa standardnog ulaza i taj znak prenosi kroz promenljivu `c[0]/c[1]` drugoj niti u istom paru, koja taj primljeni znak ispisuje na standardni izlaz, i tako neograničeno.

1. Precizno objasniti problem koji ovaj program ima (zašto je neispravan).
2. Prepraviti samo funkciju `pipe()` tako da se umesto dva para niti koje vrše razmenu znakova, formira N parova takvih niti; svaki par niti i predstavlja odvojeni „tok“ koji treba da razmenjuje podatke preko `c[i]` i `flag[i]`.

```
#include <iostream>

void writer (char* c, int* flag) {
    while (1) {
        while (*flag==1);
        cin>>(*c);
        *flag = 1;
    }
}

void reader (char* c, int* flag) {
    while (1) {
        while (*flag==0);
        cout<<(*c);
        *flag = 0;
    }
}

const int N = ...; // N>2
char c[N];
int flag[N];

void main () {
    for (int i=0; i<N; i++) flag[i]=0;
```

```

    pipe();
}

void pipe () {
    if (fork())
        writer(&c[0], &flag[0]);
    else
        reader(&c[0], &flag[0]);
    if (fork())
        writer(&c[1], &flag[1]);
    else
        reader(&c[1], &flag[1]);
}

```

Rešenje

1. Problem je u neispravnim kontrolnim strukturama u telu funkcije pipe. Početna (roditeljska) nit izvršava then granu prve if naredbe, u kojoj se poziva funkcija `writer` i koja se onda neograničeno (beskonačno) izvršava, pa ta prva nit nikada ne izlazi iz ove funkcije. Nit-dete, kreirana u prvom `fork` pozivu, izvršava else granu iste prve if naredbe, u kojoj se poziva funkcija `reader` koja se takođe neograničeno izvršava. Prema tome, ni jedna od ove dve niti neće nikada doći do druge if naredbe, pa se drugi par niti nikada neće kreirati.
- 2.

```

void pipe () {
    for (int i=0; i<N; i++)
        if (fork())
            writer(&c[i], &flag[i]);
        else
            if (fork())
                reader(&c[i], &flag[i]);
}

```

4. zadatak, prvi kolokvijum, septembar 2014.

- [Postavka](#)
- [Rešenje](#)

Korišćenjem niti u školskom jezgru realizovati klasu `Search` koja ima dole dati interfejs. Objekat ove klase obavlja pretragu datog niza celih brojeva a u opsegu indeksa počev od i zaključno sa j, tražeći u njemu vrednost x, ali tako što pretragu obavlja konkurentno i binarno-rekurzivno: jedna već kreirana nit nastavlja pretragu jedne polovine datog niza, a kreira novu nit koja vrši istu takvu pretragu druge polovine niza, i tako dalje rekurzivno. Ako se na nekom elementu n pronade vrednost x, treba ispisati „Found at n!“, a ako se tu ne nađe ta vrednost, treba ispisati „Not found at n!“.

```

class Search : public Thread {
public:
    Search (int a[], int i, int j, int x);
};

```

Primer korišćenja ove klase je sledeći:

```

int array[N];
Thread *t = new Search(array, 0, N-1, 5);
t->start();

```

Rešenje

```

class Search : public Thread {
public:
    Search (int a[], int i, int j, int x) : array(a), ii(i), jj(j), xx(x) {}
}

```

```
protected:
    virtual void run () { find(ii,jj); }
    void find (int i, int j);
private:
    int *array, ii, jj, xx;
};

void Search::find (int i, int j) {
    if (array==0 || i<0 || j<0 || j<i) return;
    if (i==j) {
        if(array[i]==x)
            printf("Found at %d!\n",i);
        else
            printf("Not found at %d!\n",i);
        return;
    }
    int k = (i+j)/2;
    Thread* t = new Search(array,k+1,j,x);
    t->start();
    find(i,k);
}
```

3. zadatak, prvi kolokvijum, septembar 2013.

- [Postavka](#)
- [Rešenje](#)

1. Na assembleru nekog RISC procesora sa *load/store* arhitekturom, poput onog opisanog na predavanjima, napisati prevod sledeće rekurzivne C funkcije koja izračunava najveći zajednički delilac (engl. *greatest common divisor*, GCD) dva data nenegativna cela broja x i y Euklidovim algoritmom, pri čemu je uvek $x \geq y \geq 0$ (pretpostaviti da su argumenti uvek ovakvi i ispravni). Funkcija `remainder(x,y)` je implementirana i vraća ostatak pri deljenju x sa y .

```
unsigned int gcd (unsigned int x, unsigned int y) {
    if (y==0) return x;
    else return gcd(y,remainder(x,y));
}
```

2. Da li je ova funkcija bezbedna za uporedne pozive iz konkurentnih niti? Obrazložiti.

Rešenje

- 1.

```
gcd:      LOAD R0,#x[SP] ; R0:=x
          LOAD R1,#y[SP] ; R1:=y
          CMP R1,#0 ; if (y==0)
          JMPNE gcd_else
gcd_then: POP PC ; return x (already in R0)
gcd_else: PUSH R1 ; push y for call of remainder
          PUSH R0 ; push x for call of remainder
          CALL remainder
          POP R1 ; remove x from the stack
          POP R1 ; remove y from the stack
          PUSH R0 ; push remainder(...) for call of gcd
          PUSH R1 ; push y for call of gcd
          CALL gcd
          POP R1 ; stack cleanup
```

```
POP R1
POP PC ; return (the result is already in R0)
```

- Jeste bezbedna (ovakvi potprogrami se ponekad nazivaju *reentrant*), pošto uopšte ne pristupa statičkim (globalnim) podacima, već sve podatke ima ili na steku, ili u registrima, što je deo konteksta niti.

2. zadatak, drugi kolokvijum, maj 2012.

- [Postavka](#)
- [Rešenje](#)

Data je biblioteka funkcija namenjena podršci optimističkom pristupu međusobnom isključenju bez eksplicitnog zaključavanja:

```
int cmpxchg(void** ptr, void* oldValue, void* newValue);
```

Ova funkcija kao prvi argument (`ptr`) prima adresu lokacije u kojoj se nalazi neki pokazivač bilo kog tipa (`void*`). Ona atomično poredi vrednost pokazivača koji je dostavljen kao drugi argument (`oldValue`) sa vrednošću na lokaciji na koju ukazuje `ptr`, i ako su te vrednosti iste, u lokaciju na koju ukazuje `ptr` upisuje vrednost datu trećim argumentom (`newValue`) i vraća 1; u suprotnom, ako su ove vrednosti različite, ne radi ništa, već samo vraća 0. Atomičnost je obezbeđena implementacijom pomoću odgovarajuće mašinske instrukcije.

Struktura `Record` predstavlja zapis (jedan element) jednostruko ulančane liste. U toj strukturi polje `next` ukazuje na sledeći element u listi.

Korišćenjem date operacije `cmpxchg()` implementirati funkciju:

```
void insert (Record** head, Record* e);
```

Ova funkcija prima argument koji predstavlja adresu pokazivača na prvi element liste (adresu lokacije u kojoj je glava liste), a kao drugi argument dobija pokazivač na novi element u listi koga treba da umetne na početak date liste. Lista je deljena između više procesa, pa ova funkcija treba da bude sigurna za uporedne pozive iz više procesa, s tim da međusobno isključenje treba obezbediti optimističkom strategijom bez eksplicitnog zaključavanja. Zapis na koga ukazuje drugi element (`e`) je privatan samo za pozivajući proces (drugi procesi mu ne pristupaju pre umetanja u listu).

Rešenje

```
void insert (Record** head, Record* e) {
    do {
        Record* oldHead = *head;
        Record* newHead = oldHead;
        e->next = newHead;
        newHead = e;
    } while (cmpxchg(head,oldHead,newHead)==0);
}
```

4. zadatak, prvi kolokvijum, april 2011.

- [Postavka](#)
- [Rešenje](#)

Klasa `Node` čija je delimična definicija data dole predstavlja čvor binarnog stabla. Funkcije `getLeftChild()` i `getRightChild()` vraćaju levo, odnosno desno podstablo datog čvora (tačnije, njegov levi i desni čvor-potomak). Funkcija `visit()` obavlja nekakvu obradu čvora.

```
class Node {
public:
    Node* getLeftChild();
    Node* getRightChild();
    void visit();
```

```
...
};
```

Potrebno je obilaziti dato binarno stablo korišćenjem uporednih niti na sledeći način. Ako neka nit trenutno obilazi neki čvor, onda ona treba da napravi novu nit-potomka koja će obići desno podstablo tog čvora, a sama ta nit će obraditi dati čvor i nastaviti sa obilaskom levog podstabla tog čvora, i tako rekurzivno.

Realizovati klasu `TreeVisitor` izvedenu iz klase `Thread` iz školskog jezgra koja realizuje opisani obilazak binarnog stabla. Ova klasa treba da se koristi na sledeći način:

```
Node* tree = ...; // The root node of a binary tree
TreeVisitor* tv = new TreeVisitor(tree);
tv->start();
```

Rešenje

```
class TreeVisitor : public Thread {
public:
    TreeVisitor (Node* root) : myRoot(root) {}
protected:
    virtual void run();
    void visit(Node*);
private:
    Node* myRoot;
};
```

```
void TreeVisitor::run () {
    visit(myRoot);
}
```

```
void TreeVisitor::visit (Node* node) {
    if (node==0) return;

    Node* rn = node->getRightChild();
    if (rn) (new TreeVisitor(rn))->start();

    node->visit();

    visit(node->getLeftChild());
}
```

5. zadatak, prvi kolokvijum, april 2010.

- [Postavka](#)
- [Rešenje](#)

Na narednoj strani prikazan je najvažniji deo programa koji pronalazi put kroz lavirint (engl. *maze*) poput onog datog na slici dole. Objašnjenje nekih pomoćnih operacija je sledeće:

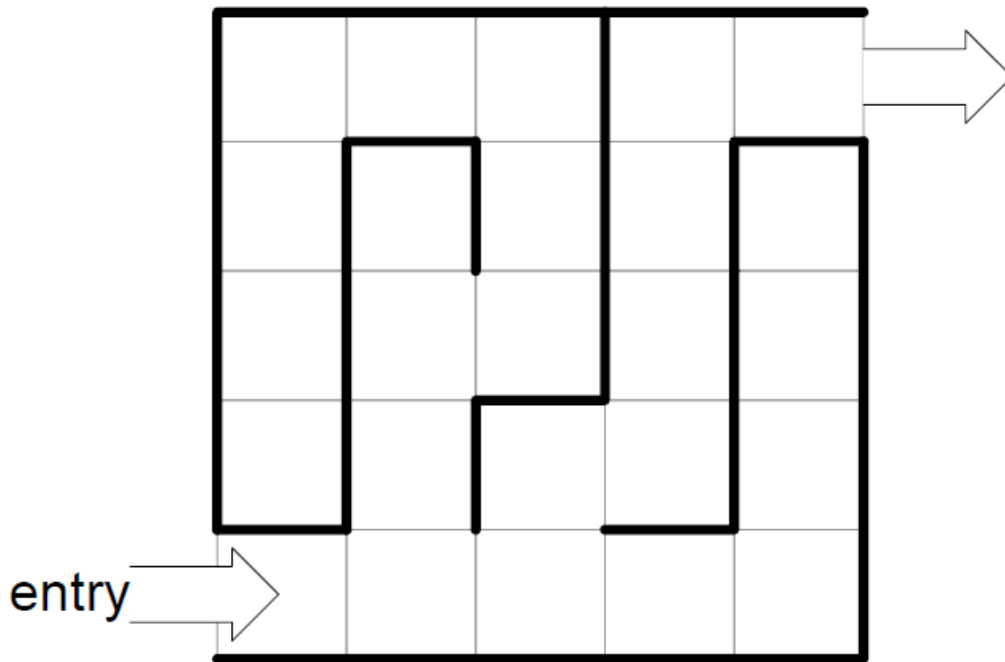
- `Direction operator-(Position pos2, Position pos1)`: Vraća smer u kome se polje označeno pozicijom `pos2` u kvadratnoj koordinatnoj mreži može dostići iz jednog od četiri susedna polja na poziciji `pos1`. Na primer, ako je `pos2` desni sused od `pos1`, vratiće E; ako mu je donji sused, vratiće S; ako mu je levi, vratiće W; ako je gornji, vratiće N.
- `int Maze::isExit(Position)`: Vraća 1 ako je polje na datoj poziciji u lavirintu izlaz, odnosno 0 ako nije.
- `Position* Maze::getFirstOption(Position p, Direction d), getSecondOption, getThirdOption`: Za dato polje `p` u lavirintu u koje se stiglo napredovanjem u datom smeru `d`, vraćaju slobodna susedna polja u koja se može ići bez povratka u polje iz koga se stiglo. Ako nema ni jednog mogućeg smera (slobodnog susednog polja osim onoga iz koga se stiglo u `p`), sve tri operacije vratiće 0; ako je moguće nastaviti samo u

jednom smeru, samo će `getFirstOption` vratiti poziciju susednog slobodnog polja, ostale će vratiti 0; ako je moguće nastaviti u dva smera, prve dve operacije će vratiti pozicije susednih slobodnih polja, treća će vratiti 0.

Tragač za izlazom pokrenut je na sledeći način za lavirint prikazan na donjoj slici:

```
MazeExitSeeker* seeker = new MazeExitSeeker(theMaze,entry,E);
seeker->start();
```

1. Precizno objasniti kako ovaj program pretražuje lavirint.
2. Koliko ukupno niti će biti pokrenuto za ovaj lavirint, računajući i početnu nit (`seeker`) prikazanu gore?
3. Uz pretpostavku da svaka nit ima svoj logički izlazni uređaj `cout` (nema preplitanja ispisa iz različitih niti), koliko puta će ukupno biti ispisana rečenica „I have reached a dead end and I am giving up.”? `entry`



Slika 1: Lavirint

```
enum Direction {N, E, S, W};

class Position {
public:
    friend Direction operator- (Position pos2, Position pos1);
    ...
};

class Maze {
public:
    int isExit(Position);
    Position* getFirstOption(Position,Direction);
    Position* getSecondOption(Position,Direction);
    Position* getThirdOption(Position,Direction);
    ...
};
```



```

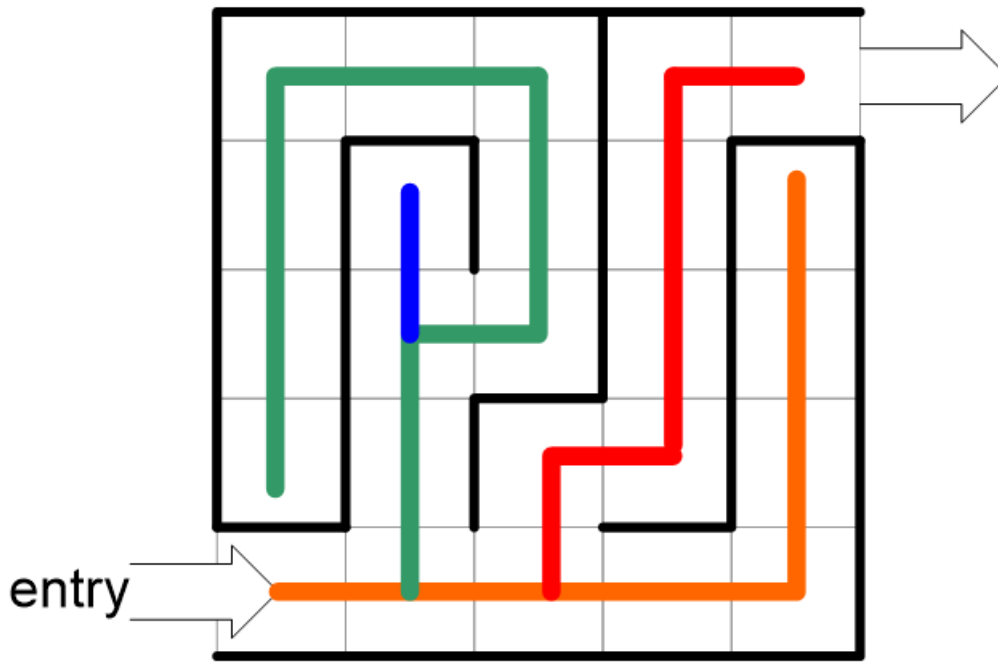
class MazeExitSeeker : public Thread {
public:
    MazeExitSeeker (Maze* m, Position start, Direction dir) :
        myMaze(m), myPos(start), myDir(dir) {}
protected:
    virtual void run();
private:
    Maze* myMaze;
    Position myPos;
    Direction myDir;
};

void MazeExitSeeker::run () {
    cout<<"I'm starting my search from "<<myPos<<" towards "<<myDir<<".\n"
    while (1) {
        if (myMaze->isExit(myPos)) {
            cout<<"I have found the exit!\n"
            return;
        }
        Position* nextPos1 = myMaze->getFirstOption(myPos,myDir);
        Position* nextPos2 = myMaze->getSecondOption(myPos,myDir);
        Position* nextPos3 = myMaze->getThirdOption(myPos,myDir);
        if (nextPos1==0) {
            cout<<"I have reached a dead end and I am giving up.\n"
            return;
        }
        if (nextPos2!=0)
            (new MazeExitSeeker(myMaze,*nextPos2,*nextPos2-myPos))->start();
        if (nextPos3!=0)
            (new MazeExitSeeker(myMaze,*nextPos3,*nextPos3-myPos))->start();
        myDir=*nextPos1-myPos;
        myPos=*nextPos1;
    }
}

```

Rešenje

1. Svaka nit napreduje kroz hodnike lavirinta i u svakom koraku, na datom polju u kvadratnom koordinatnom sistemu, radi sledeće. Najpre pogleda da li je pronašla izlaz i ako jeste, ispisuje to i završava se. Zatim od lavirinta dobije najviše tri susedna slobodna polja u koja može da pređe (osim onoga iz koga je stigla). Ako ni jedno od takvih ne postoji, ta nit je udarila u slepi hodnik, pa ispisuje rečenicu „I have reached a dead end and I am giving up“ i gasi se. Inače, ta nit nastavlja da ide prvom dobijenom opcijom, tako da joj je sledeće polje to susedno polje dobijeno kao prva opcija, u tom smeru. Ako postoje druga i treća opcija, onda ova nit kreira nove niti, po jednu za svaku od tih raspoloživih opcija (drugu i treću), tako da te niti nastavljaju od tih susednih polja, u odgovarajućim smerovima.
2. Četiri niti. Na slici je prikazan jedan mogući raspored puteva kojim tragaju te niti, uz pretpostavku da roditeljska nit nastavlja desnim od mogućih puteva, a potomci ostalim.
3. Tri puta.



Slika 2: Rešenje

Promena konteksta

1. zadatak, kolokvijum, septembar 2020.

- [Postavka](#)
- [Rešenje](#)

Dostupni su sledeći delovi školskog jezgra, kao i funkcije iz standardne biblioteke:

- `Thread::running`: statički član – pokazivač na objekat klase `Thread` koji predstavlja tekuću nit;
- `Thread::context`: nestatički član tipa `jmp_buf` u kom se čuva procesorski kontekst niti;
- `const size_t STACK_SIZE`: veličina prostora za stek niti (u jedinicama `sizeof(char)`);
- `Thread::stack`: pokazivač tipa `void*` koji ukazuje na adresu početka dela memorije u kom je alocirani stek niti;
- `jmp_buf::sp`: polje za sačuvanu vrednost registra SP; stek raste ka nižim adresama, a SP ukazuje na prvu slobodnu lokaciju veličine `sizeof(int)`;
- `Scheduler::put(Thread*)`: statička članica kojom se u red spremnih stavlja data nit;
- `void* malloc(size_t sz)`: standardna funkcija koja alokira prostor veličine `sz` (u jedinicama `sizeof(char)`); funkcija `free(void*)` oslobađa ovako alocirani prostor;
- `memcpy(void* dst, const void* src, size_t size)`: kopira memorijski sadržaj veličine `size` sa mesta na koje ukazuje `src` na mesto na koje ukazuje `dst`;
- `setjmp()`, `longjmp()`: standardne bibliotečne funkcije.

Pomoću ovih elemenata implementirati sistemski poziv školskog jezgra – funkciju:

```
Thread* t_fork();
```

kojom se kreira nova nit kao „klon“ pozivajuće, roditeljske niti, sa istim početnim kontekstom, ali sa sopstvenom kontrolom toka, po uzoru na standardni sistemski poziv `fork()` za procese. U slučaju uspeha, u kontekstu roditeljske niti ova funkcija treba da vrati pokazivač na objekat niti deteta, a u kontekstu niti deteta treba da vrati 0; u slučaju greške, treba da podigne izuzetak tipa `ThreadCreationException`.

Rešenje

```

Thread* t_fork() {
    lock();
    // Allocate a new stack:
    void* stck = malloc(STACK_SIZE);
    if (!stck) throw ThreadCreationException();
    // and copy its contents from the parent's stack:
    memcpy(stck, Thread::running->stack, STACK_SIZE);
    // Create a new Thread object:
    Thread* newThr = new Thread();
    if (!newThr) {
        free(stck);
        throw ThreadCreationException();
    }
    newThr->stack = stck;
    if (setjmp(newThr->context) == 0) {
        // Parent thread:
        // and set its stack pointer:
        newThr->context->sp = newThr->context->sp - Thread::running->stack + stck;
        // Put the new thread to the ready list and return:
        Scheduler::put(newThr);
        unlock();
        return newThr;
    } else {
        // Child thread:
        unlock();
        return 0;
    }
}

```

2. zadatak, prvi kolokvijum, jun 2019.

- [Postavka](#)
- [Rešenje](#)

Raspoređivanje i promena konteksta u školskom jezgru implementirani su na sledeći način:

- Svi objekti klase `Thread` smešteni su u statički vektor `Thread::allThreads` tipa `Thread[NumOfThreads]`.
- Nestatički podatak član `Thread::isRunnable` govori o tome da li je odgovarajuća nit spremna za izvršavanje ili se baš ona izvršava, ili nije (jer je suspendovana).
- Statički podatak član `Thread::running` tipa `int` ukazuje na onaj element niza `allThreads` koji predstavlja nit koja se trenutno izvršava (tekuća nit).
- Za izvršavanje se bira prva sledeća spremna nit u nizu `allThreads` iza one tekuće, i tako u krug.
- Nestatički podatak član `Thread::context` tipa `jmp_buf` čuva procesorski kontekst niti.
- Implementirana je funkcija `yield(jmp_buf oldC, jmp_buf newC)` koja čuva kontekst procesora u prvi argument i restaurira kontekst iz drugog argumenta.

Korišćenjem date funkcije `yield`, implementirati funkciju `dispatch` koja treba da preda procesor niti koja je data kao argument, pod uslovom da je taj argument dat i da je ta nit spremna. Ako je data nit spremna, funkcija treba da vrati 0; u suprotnom, tekuća nit treba da nastavi izvršavanje, a ova funkcija da vrati -1. Ako nije zadata nit, ova funkcija treba da preda procesor sledećoj spremnoj niti i da vrati 0.

```
int dispatch (Thread* newT = 0);
```

Rešenje

```

int dispatch (Thread* newT = 0) {
    lock();

```

```

if (newT && !newT->isRunnable) {
    unlock();
    return -1;
}
int oldR = Thread::running;
jmp_buf oldC = Thread::allThreads[oldR].context;
if (newT)
    Thread::running = newT - Thread::allThreads;
else
    do
        Thread::running = (Thread::running+1)%NumOfThreads;
        while (!Thread::allThreads[Thread::running].isRunnable &&
            Thread::running!=oldR);
    if (Thread::running!=oldR) {
        jmp_buf newC = Thread::allThreads[Thread::running].context;
        yield(oldC,newC);
    }
unlock();
return 0;
}

```

2. zadatak, prvi kolokvijum, april 2017.

- [Postavka](#)
- [Rešenje](#)

U nekom asimetričnom multiprocesorskom operativnom sistemu jedan od procesora posvećen je samo obavljanju ulazno-izlaznih operacija koje su zahtevali korisnički procesi. Operacije sa svakim pojedinačnim ulazno-izlaznim uređajem obavlja po jedna interna kernel nit predstavljena objektom klase `IOThread`. Ove niti izvršavaju se samo na ovom posvećenom procesoru i nemaju nikakve veze sa ostalim nitima kernela niti korisničkim procesima (osim preuzimanja zahteva koje su oni postavili); na ovom procesoru izvršavaju se samo ove niti.

Svaka od tih niti ima sledeći generički oblik: ona uzme jedan zahtev za ulazno-izlaznom operacijom iz reda zahteva postavljenih za taj uređaj, pokrene operaciju sa uređajem na način specifičan za taj uređaj, a onda se suspenduje pozivom operacije `IOThread::suspend` dok uređaj ne signalizira spremnost za novu operaciju spoljašnjim prekidom. Prekidne rutine svih tih uređaja ne vrše promenu konteksta (preotimanje procesora), već samo postavljaju polje `IOThread::isReady` svoje niti na 1, čime ta nit ponovno postaje spremna i može da zada novu operaciju. Zbog svega ovoga nije potrebno raditi nikakvo maskiranje prekida niti međusobno isključenje sa ostalim procesorima.

Sve ove niti predstavljene su objektima klase `IOThread` u statičkom nizu `IOThread::allThreads`, a njihov broj je konstantan i iznosi `IOThread::NumberOfThreads`. Polje `IOThread::running` je pokazivač na nit koja se trenutno izvršava na ovom posvećenom procesoru. Na raspolaganju je funkcija:

```
void IOThread::yield (IOThread* oldThread, IOThread* newThread);
```

koja čuva kontekst tekuće niti u za to predviđeno polje objekta na koga ukazuje prvi argument i restaurira kontekst niti iz polja objekta na koga ukazuje drugi argument. Implementirati operaciju `IOThread::suspend`. Za izvršavanje je dovoljno uzeti prvu (ili bilo koju drugu) spremnu nit iz niza `IOThread::allThreads`. Obratiti pažnju na to da je moguće da nijedna nit nije spremna za izvršavanje, jer su sve suspendovane i čekaju na završetak svojih operacija i prekide od svojih uređaja koji će ih ponovo učiniti spremnim; u tom slučaju procesor treba da uposlono čeka dok neka nit ne postane spremna.

Rešenje

```

void IOThread::suspend () {
    IOThread::running->isReady = 0;
    int newRunning = -1;
    while (newRunning== -1) {
        for (int i=0; i<IOThread::NumberOfThreads; i++)

```

```

        if (IOThread::allThreads[i].isReady) {
            newRunning = i;
            break;
        }
    }
    IOThread* oldThread = IOThread::running;
    IOThread* newThread = &IOThread::allThreads[newRunning];
    IOThread::running = newThread;
    yield(oldThread, newThread);
}

```

2. zadatak, prvi kolokvijum, jun 2017.

- [Postavka](#)
- [Rešenje](#)

U školskom jezgri promena konteksta implementirana je korišćenjem date funkcije `yield()`, u sistemskom pozivu `dispatch()` i na svim ostalim mestima na sličan način kao što je dato.

Potrebno je implementirati sistemski poziv (statičku operaciju):

```
void Thread::wait(Thread* forChild=0);
```

kojim pozivajuća nit čeka (suspenduje se ako je potrebno) dok se ne završi nit-dete na koje ukazuje argument, odnosno sve niti-deca koje je ova pozivajuća nit do tada kreirala, ako je ovaj argument *null*. Za te potrebe treba implementirati i sledeće nestatičke funkcije-članice:

- `void Thread::created(Thread* parent)`: poziva je jezgro interno za datu novokreiranu nit (`this`), kada je ta nit kreirana, sa argumentom `parent` koji ukazuje na roditeljsku nit u čijem kontekstu je ova nova nit-dete kreirana;
- `void Thread::completed()`: poziva je jezgro za datu nit (`this`), kada se ta nit završila.

Ukoliko proširujete klasu `Thread` novim članovima, precizno navedite kako.

```

void yield (jmp_buf oldBuf, jmp_buf newBuf) {
    if (setjmp(oldBuf)==0) longjmp(newBuf,1);
}

```

```

void Thread::dispatch () {
    lock();
    jmp_buf oldBuf = Thread::running->context;
    Scheduler::put(Thread::running);
    Thread::running = Scheduler::get();
    jmp_buf newBuf = Thread::running->context;
    yield(oldBuf, newBuf);
    unlock();
}

```

Rešenje

U klasu `Thread` dodati su sledeći privatni, nestatički podaci-članovi sa datim inicijalnim vrednostima:

```

Thread* Thread::parent = 0;
bool Thread::isActive = false;
bool Thread::isWaitingForAllChildren = false;
unsigned long Thread::activeChildrenCounter = 0;
Thread* Thread::isWaitingForChild = 0;

void Thread::created (Thread* par) {
    this->isActive = true;
    this->parent = par;
}

```

```

    if (par) this->parent->activeChildrenCounter++;
}

void Thread::completed () {
    this->isActive = false;
    if (!this->parent) return;
    this->parent->activeChildrenCounter--;
    if ((this->parent->isWaitingForAllChildren &&
        this->parent->activeChildrenCounter==0) ||
        this->parent->isWaitingForChild==this) {
        this->parent->isWaitingForAllChildren = false;
        this->parent->isWaitingForChild = 0;
        Scheduler::put(this->parent);
    }
}

void Thread::wait (Thread* forChild=0) {
    lock();
    jmp_buf old = Thread::running->context;

    if (forChild==0)
        if (Thread::running->activeChildrenCounter>0)
            Thread::running->isWaitingForAllChildren = true;
        else
            Scheduler::put(Thread::running);
    else
        if (forChild->parent==Thread::running && forChild->isActive)
            Thread::running->isWaitingForChild = forChild;
        else
            Scheduler::put(Thread::running);

    Thread::running = Scheduler::get();
    jmp_buf new = Thread::running->context;
    yield(old,new);
    unlock();
}

```

2. zadatak, prvi kolokvijum, maj 2016.

- [Postavka](#)
- [Rešenje](#)

U školskom jezgri promena konteksta implementirana je korišćenjem date funkcije `yield()`, u sistemskom pozivu `dispatch()` i na svim ostalim mestima na sličan način kao što je dato.

Potrebno je implementirati sistemski poziv (statičku operaciju) `Thread::wait()` kojim pozivajuća nit čeka (suspenduje se ako je potrebno) dok se ne završe sve niti-deca koje je ova pozivajuća nit do tada kreirala. Za te potrebe treba implementirati i sledeće nestatičke funkcije-članice:

- `void Thread::created(Thread* parent)`: poziva je jezgro interno za datu novokreiranu nit (`this`), kada je ta nit kreirana, sa argumentom `parent` koji ukazuje na roditeljsku nit u čijem kontekstu je ova nova nit-dete kreirana;
- `void Thread::completed()`: poziva je jezgro za datu nit (`this`), kada se ta nit završila.

Ukoliko proširujete klasu `Thread` novim članovima, precizno navedite kako.

```

void yield (jmp_buf old, jmp_buf new) {
    if (setjmp(old)==0) longjmp(new,1);
}

```

```

void Thread::dispatch () {
    lock();
    jmp_buf old = Thread::running->context;
    Scheduler::put(Thread::running);
    Thread::running = Scheduler::get();
    jmp_buf new = Thread::running->context;
    yield(old,new);
    unlock();
}

```

Rešenje

U klasu Thread dodati su sledeći privatni, nestatički podaci-članovi sa datim inicijalnim vrednostima:

```

Thread* Thread::parent = 0;
bool Thread::isWaitingForChildren = false;
unsigned long Thread::activeChildrenCounter = 0;

void Thread::created (Thread* parent) {
    this->parent = parent;
    if (parent) parent->activeChildrenCounter++;
}

void Thread::completed () {
    if (this->myParent && --this->myParent->activeChildrenCounter==0 &&
        this->myParent->isWaitingForChildren) {
        this->myParent->isWaitingForChildren = false;
        Scheduler::put(this->myParent);
    }
}

void Thread::wait () {
    lock();
    jmp_buf old = Thread::running->context;
    if (Thread::running->activeChildrenCounter>0)
        Thread::running->isWaitingForChildren = true;
    else
        Scheduler::put(Thread::running);
    Thread::running = Scheduler::get();
    jmp_buf new = Thread::running->context;
    yield(old,new);
    unlock();
}

```

2. zadatak, prvi kolokvijum, septembar 2015.

- [Postavka](#)
- [Rešenje](#)

Školsko jezgro proširuje se konceptom tzv. asinhronih signala, koji podržavaju mnogi operativni sistemi, sa sledećim značenjem.

Kernel može „poslati“ signal nekoj korisničkoj niti. Signal je celobrojna konstanta u opsegu 1..SIGS-1. Čim ta nit ponovo dobije procesor, umesto da odmah nastavi izvršavanje tamo gde je bila prekinuta, odnosno izgubila procesor, najpre će skočiti u proceduru za obradu tog signala, tzv. *signal handler*, pa kada se iz te procedure vrati, nastaviće dalje izvršavanje tamo gde je ono bilo prekinuto.

Na procedure za obradu signala pokazuju pokazivači u tabeli koja se nalazi u PCB svake niti, poput vektor tabele, u

nizu `Thread::sigHandlers[SIGS]`. Za svaku nit, signalu n odgovara procedura za obradu na koju ukazuje pokazivač u ulazu `Thread::sigHandlers[n]` (ulaz 0 se ne koristi). Kada „šalje“ signal nekoj niti, kernel u PCB te niti postavi vrednost signala n u polje `Thread::signal`; vrednost različita od 0 u ovom polju ukazuje na postojanje poslatog signala, dok 0 znači da signala nema.

Dole je data implementacija sistemskog poziva `dispatch()` u školskom jezgru. Modifikovati ovu implementaciju tako da podrži obradu poslatog signala datoj niti. Smatrati da su na isti ovakav način realizovani i svi ostali sistemski pozivi i preuzimanja procesora.

```
void yield (jmp_buf old, jmp_buf new) {
    if (setjmp(old)==0)
        longjmp(new,1);
}

void dispatch () {
    lock();
    jmp_buf old = Thread::running->context;
    Scheduler::put(Thread::running);
    Thread::running = Scheduler::get();
    jmp_buf new = Thread::running->context;
    yield(old,new);
    unlock();
}
```

Rešenje

```
void yield (jmp_buf old, jmp_buf new) {
    if (setjmp(old)==0)
        longjmp(new,1);
}

void dispatch () {
    lock();
    jmp_buf old = Thread::running->context;
    Scheduler::put(Thread::running);
    Thread::running = Scheduler::get();
    jmp_buf new = Thread::running->context;
    yield(old,new);
    unlock();
    if (Thread::running->signal) {
        Thread::running->sigHandlers[Thread::running->signal]();
        Thread::running->signal = 0;
    }
}
```

4. zadatak, prvi kolokvijum, april 2014.

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu svi sistemski pozivi vrše se softverskim prekidom broj 44h, pri čemu u registru `r0` sistem očekuje identifikator sistemskog poziva. U ovom sistemu postoji sistemski poziv za kreiranje i pokretanje niti nad potprogramom koji prihvata jedan argument tipa pokazivača. Ovaj poziv u registrima očekuje sledeće argumente:

- u `r0` treba da bude identifikator ovog poziva, 0 u ovom slučaju;
- u `r1` treba da bude adresa potprograma nad kojim se kreira nit;
- u `r2` treba da bude argument potprograma nad kojim se kreira nit.

Svaki sistemski poziv vraća rezultat u registru `r0`, a za ovaj sistemski poziv rezultat je identifikator niti u jezgru (ceo

broj veći od 0), odnosno kod greške (ceo broj manji od 0). U asemblerskim blokovima unutar C/C++ koda može se koristiti simbolička konstanta sa nazivom formalnog argumenta funkcije unutar koje se nalazi dati asemblerski kod; ova simbolička konstanta ima vrednost odgovarajućeg pomeraja lokacije u kojoj se nalazi dati formalni argument u odnosu na vrh steka (kao što je pokazano u primerima na predavanjima). C/C++ funkcije vraćaju vrednost u registru r0 ukoliko je tip povratne vrednosti odgovarajući.

1. Implementirati funkciju

```
int create_thread (void (*f)(void*), void* arg);
```

koja vrši opisani sistemski poziv (kreira nit nad funkcijom `f` sa argumentom `arg`) i koja može da se poziva iz C koda sa zadatim argumentima. Ova funkcija vraća identifikator kreirane niti koji se može koristiti u ostalim sistemskim pozivima da identifikuje tu nit u jezgru, odnosno kod greške.

2. Korišćenjem prethodne funkcije, implementirati funkciju `start` klase `Thread`. Ova klasa obezbeđuje koncept niti u objektnom duhu, poput onog u školskom jezgru (kreira nit nad virtuelnom funkcijom `run`).

```
class Thread {
public:
    Thread () : pid(0) {}
    int start ();
    virtual void run () {}
private:
    int pid; // Thread ID
};
```

Rešenje

```
int create_thread (void (*f)(void*), void* arg) {
    asm {
        load r0,#0
        load r1,#f[sp]
        load r2,#arg[sp]
        int 44h
    }
}

void wrapper (void* t) {
    if(t)((Thread*)t)->run();
}

int Thread::start () {
    if(pid)
        return pid;
    else
        return pid = create_thread(&wrapper,this);
}
```

3. zadatak, prvi kolokvijum, maj 2012.

- [Postavka](#)
- [Rešenje](#)

Korišćenjem standardnih bibliotečnih funkcija `setjmp()` i `longjmp()`, realizovati operaciju

```
yield(jmp_buf old, jmp_buf new);
```

koja čuva čuva kontekst niti čiji je `jmp_buf` dat kao prvi argument, oduzima joj procesor i restaurira kontekst niti čiji je `jmp_buf` dat kao drugi argument, kojoj predaje procesor.

Koristeći ovu operaciju `yield()`, realizovati operaciju `dispatch()` koja ima isti efekat kao i ona data u školskom jezgru.

Rešenje

```
void yield (jmp_buf old, jmp_buf new) {
    if (setjmp(old)==0)
        longjmp(new,1);
}

void dispatch () {
    lock();
    jmp_buf old = Thread::running->context;
    Scheduler::put(Thread::running);
    Thread::running = Scheduler::get();
    jmp_buf new = Thread::running->context;
    yield(old,new);
    unlock();
}
```

3. zadatak, prvi kolokvijum, septembar 2012.

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu realizovana je operacija

```
void yield (jmp_buf old, jmp_buf new);
```

koja čuva kontekst niti čiji je `jmp_buf` dat kao prvi argument, oduzima joj procesor i restaurira kontekst niti čiji je `jmp_buf` dat kao drugi argument, kojoj predaje procesor.

Koristeći ovu operaciju `yield()`, realizovati operacije:

- `Thread::suspend()`: (statička) suspenduje (blokira) izvršavanje pozivajuće niti sve dok je neka druga nit ne „probudi“ pomoću `resume()`;
- `Thread::resume()`: (nestatička) „budi“ (deblokira) nit za koju je pozvana; vrši i promenu konteksta predajući procesor niti koja je na redu za izvršavanje.

Pretpostaviti da je lista spremnih procesa uvek neprazna prilikom suspenzije niti i da je nit za koju se poziva `resume` sigurno suspendovana (ignorirati mogućnost greške). Klase `Thread` i `Scheduler` su u preostalim delovima implementirane kao u školskom jezgru.

Rešenje

```
void Thread::suspend () {
    lock();
    jmp_buf old = Thread::running->context;
    Thread::running = Scheduler::get();
    jmp_buf new = Thread::running->context;
    yield(old,new);
    unlock();
}

void Thread::resume () {
    lock();
    Scheduler::put(this);
    jmp_buf old = Thread::running->context;
    Scheduler::put(Thread::running);
    Thread::running = Scheduler::get();
}
```

```

jmp_buf new = Thread::running->context;
yield(old,new);
unlock();
}

```

3. zadatak, prvi kolokvijum, april 2010.

- [Postavka](#)
- [Rešenje](#)

U donju tabelu upisati “Da” ukoliko operativni sistem datu operaciju treba, a “Ne” ukoliko ne treba da izvrši prilikom promene konteksta procesa, odnosno niti.

Operacija	Promena konteksta procesa	Promena konteksta niti
Restauracija programski dostupnih registara procesora za podatke		
Invalidacija TLB-a čiji ključevi ne sadrže identifikaciju procesa		
Invalidacija TLB-a čiji ključevi sadrže identifikaciju procesa		
Restauracija registra koji čuva identifikaciju korisničkog procesa		
Restauracija pokazivača na tabelu preslikavanja stranica (PMTP)		
Restauracija procesorske statusne reči (PSW)		
Invalidacija procesorskog keša koji kao ključeve čuva virtuelne adrese		
Invalidacija procesorskog keša koji kao ključeve čuva fizičke adrese		
Zatvaranje otvorenih fajlova		
Restauracija pokazivača steka (SP)		

Rešenje

Operacija	Promena konteksta procesa	Promena konteksta niti
Restauracija programski dostupnih registara procesora za podatke	Da	Da
Invalidacija TLB-a čiji ključevi ne sadrže identifikaciju procesa	Da	Ne
Invalidacija TLB-a čiji ključevi sadrže identifikaciju procesa	Ne	Ne
Restauracija registra koji čuva identifikaciju korisničkog procesa	Da	Ne
Restauracija pokazivača na tabelu preslikavanja stranica (PMTP)	Da	Ne
Restauracija procesorske statusne reči (PSW)	Da	Da
Invalidacija procesorskog keša koji kao ključeve čuva virtuelne adrese	Da	Ne
Invalidacija procesorskog keša koji kao ključeve čuva fizičke adrese	Ne	Ne
Zatvaranje otvorenih fajlova	Ne	Ne
Restauracija pokazivača steka (SP)	Da	Da

5. zadatak, prvi kolokvijum, april 2008.

- [Postavka](#)
- [Rešenje](#)

U jezgru nekog jednoprocorskog operativnog sistema definisani su sledeći elementi:

```

const int IVTSize = ...; // Number of interrupt entries
typedef int InerruptID; // IVT Entry Number

class Thread {
public: ...
    jmp_buf context;
    Thread* next; // Pointer to the next thread in a (waiting or ready) list
};

Thread* running; // The running thread

```

```
Thread* readyFirst; // The head of the ready list
Thread* readyLast; // The tail of the ready list
```

Implementirane su i standardne funkcije `setjmp()` i `longjmp()` koje čuvaju, odnosno restauriraju sve programski dostupne registre (uključujući i PC i SP). Pretpostavlja se da lista spremnih niti nikad nije prazna – uvek je u njoj bar jedna podrazumevana nit koja izvršava praznu petlju i nikad se ne suspenduje i ne gasi, tzv. *besposlena* (engl. *idle*) nit koja u svakoj svojoj iteraciji vrši eksplicitno preuzimanje (koje ne treba realizovati). Korišćenjem ovih elemenata, implementirati sledeće funkcije:

- `void suspend(InterruptID)`: suspenduje (blokira) izvršavanje tekuće niti sve dok ga ne „probudi“ prekid sa datim brojem;
- `void resume(InterruptID)`: „budi“ (deblokira) izvršavanje jedne niti suspendovane na prekidu sa datim brojem.

Pretpostavlja se da su svi ulazi IVT tabele u opsegu `0..IVTSize-1` već inicijalizovani tako da ukazuju na po jednu jednostavnu prekidnu rutinu, pri čemu sve one prosto pozivaju operaciju `resume()` sa argumentom koji predstavlja broj tog prekida i ne rade ništa više, a razlikuju se samo po tom broju. Prekidi se mogu maskirati i demaskirati operacijama `lock()` i `unlock()`, respektivno.

Rešenje

```
// Auxiliary list operations:
```

```
void put(Thread* t, Thread*& head, Thread*& tail) {
    if (t==0) return; // Error
    t->next=0;
    if (tail) tail->next=t;
    else head=t;
    tail=t;
};
```

```
Thread* get(Thread*& head, Thread*& tail) {
    Thread* t = head;
    if (t==0) return;
    t->next=0;
    head=head->next;
    if (head==0) tail=0;
}
```

```
class InterruptWaitingQueue {
public:
    static InterruptWaitingQueue* Instance();
    void put(InterruptID, Thread*);
    Thread* get(InterruptID);
private:
    struct InterruptWaitingQueueEntry {
        Thread* head;
        Thread* tail;
        InterruptWaitingQueueEntry():head(0),tail(0) {}
    };
    InterruptWaitingQueueEntry queue[IVTSize];
};
```

```
InterruptWaitingQueue* InterruptWaitingQueue::Instance() {
    static InterruptWaitingQueue instance;
    return &instance;
}

void InterruptWaitingQueue::put(InterruptID intID, Thread* t) {
    put(t, queue[intID].head, queue[intID].tail);
}
```

```

}

Thread* InterruptWaitingQueue::get(InterruptID intID) {
    return get(queue[intID].head, queue[intID].tail);
}

void suspend (InterruptID intID) {
    if (intID>=IVTSize) return; // Error;
    lock();
    if (setjmp(running->context)==0) {
        // Suspend the running thread:
        InterruptWaitingQueue::Instance()->put(running);
        // Get a new running thread:
        running=get(readyFirst, readyLast);
        longjmp(running->context);
    };
    unlock();
}

void resume (InterruptID intID) {
    if (intID>=IVTSize) return; // Error;
    Thread* t = InterruptWaitingQueue::Instance()->get(intID);
    if (t==0) return; // No effect
    put(t, readyFirst, readyLast);
}

```

4. zadatak, prvi kolokvijum, april 2007.

- [Postavka](#)
- [Rešenje](#)

U jezgru nekog operativnog sistema definisane su sledeće strukture podataka:

```

typedef unsigned int PID;
#define MaxProc ... // Max number of processes
struct PCB {
    int sp; // Saved stack pointer
    PID next; // Next PCB in the Ready or a waiting list
    ...
};
PCB* allProcesses[MaxProc]; // Maps a PID to a PCB*
PID running; // The running process
PID ready; // Head of Ready list

```

Korišćenjem ovih struktura, po uzoru na funkciju datu na predavanjima, na jeziku C i assembleru zamišljenog procesora napisati telo funkcije `yield(PCB* current, PCB* next)` koja vrši promenu konteksta sa procesa `current` na proces `next`; ova funkcija služi samo za realizaciju drugih operacija jezgra i poziva se samo iz koda jezgra. Korišćenjem te funkcije, na jeziku C napisati tela sledećih funkcija:

- `suspend()`: suspenduje (blokira) izvršavanje pozivajućeg procesa sve dok ga neki drugi proces ne „probudi“ pomoću `resume()`;
- `resume(PID pid)`: „budi“ (deblokira) izvršavanje procesa koji je zadat argumentom.

Rešenje

```

typedef unsigned int PID;
#define MaxProc ... // Max number of processes
struct PCB {
    int sp; // Saved stack pointer

```

```

    PID next;           // Next PCB in the Ready or a waiting list
    ...
};
PCB* allProcesses[MaxProc]; // Maps a PID to a PCB*
PID running; // The running process
PID ready;   // Head of Ready list

void yield(PCB* current, PCB* next){
    asm{
        push R0
        ...
        push Rn

        load R0, #current[BP]
        load #sp[R0], SP // #sp je pomjeraj odgovarajuceg polja strukture
        load R0, #next[BP] // i u ovom slucaju je #sp=0
        load SP, #sp[R0]

        pop Rn
        ...
        pop R0
    }
    return;
}

```

Drugi način za yield (Neki kompajleri posjeduju registarske pseudovarijable, pa je registrima moguće pristupiti i direktno iz C odnosno C++ koda. Jedan od takvih je TC.):

```

void yield(PCB* current, PCB* next){
    asm{
        push R0
        ...
        push Rn
    }
    current->sp = _SP; // _SP je pseudovarijabla za SP
    _SP = next->sp;

    asm{
        pop Rn
        ...
        pop R0
    }
    return;
}

void suspend(){
    PCB* old = allProcesses[running];
    running = ready;
    ready = allProcesses[ready]->next;
    PCB* new = allProcesses[running];
    allProcesses[running]->next=0;
    yield(old, new);
    return;
}

resume(PID pid){

```

```

int ind=(running != pid);
for(PID i = 0;i < MaxProc;i++) if (allProcesses[i].next == pid) ind=0;
if (ind){
    allProcesses[pid]->next = ready;
    ready = pid;
}
}

```

5. zadatak, prvi kolokvijum, april 2006.

- [Postavka](#)
- [Rešenje](#)

U jezgri nekog operativnog sistema definisane su sledeće strukture podataka:

```

typedef unsigned int PID;
#define MaxProc ... // Max number of processes
struct PCB {
    PID pid; // Process ID
    jmp_buf context;
    PCB* next; // Next PCB in the Ready or a waiting list
    ...
};
PCB* allProcesses[MaxProc]; // Maps a PID to a PCB*
PCB* running; // The running process
PCB* ready; // Head of Ready list

```

Korišćenjem bibliotečnih funkcija `setjmp()` i `longjmp()`, na jeziku C napisati tela sledećih funkcija:

- `suspend()`: suspenduje (blokira) izvršavanje pozivajućeg procesa sve dok ga neki drugi proces ne „probudi“ pomoću `resume()`;
- `resume(PID pid)`: „budi“ (deblokira) izvršavanje procesa koji je zadat argumentom.

Rešenje

Potrebno je dodati sledeću deklaraciju, koja će predstavljati pokazivač na listu suspendovanih (blokiranih) procesa.

```
PCB* blocked;
```

Implementacije traženih funkcija:

```

void suspend () {
    lock();
    if (setjmp(running->context)==0) {
        // stavlja running proces u listu blokiranih
        running->next = blocked;
        blocked = running;

        // uzima prvi spreman proces i dodeljuje mu procesor
        running = ready;
        ready = ready->next;

        longjmp(running->context,1);
    } else {
        unlock ();
        return;
    }
}

void resume(PID pid){

```

```

if (blocked == 0) return;

lock();

// pronalazi PCB procesa koji treba deblokirati
PCB *prev = 0, *curr = blocked;
while (curr != 0 && curr->pid != pid){
    prev = curr;
    curr = curr->next;
}

if (curr == 0) {
    unlock();
    return;
}

// vraća nađeni PCB u listu spremnih
if (prev == 0) blocked = blocked->next;
else prev->next = curr->next;

curr->next = ready;
ready = curr;

unlock();
}

```

1. zadatak, drugi kolokvijum, maj 2006.

- [Postavka](#)
- [Rešenje](#)

U nekom računarskom sistemu postoje spoljašnji maskirajući prekidi koje generišu ulazno/izlazni uređaji. Ovi prekidi maskiraju se operacijom `int_mask()`, a demaskiraju operacijom `int_unmask()`. Po prihvatanju prekida, procesor automatski maskira prekide. Potrebno je realizovati koncept spoljašnjeg događaja (*external event*) koji se manifestuje pojavom spoljašnjeg maskirajućeg prekida. Ovaj koncept treba realizovati klasom `ExternalEvent`, po uzoru na realizaciju semafora pomoću operacija `setjmp()` i `longjmp()` datu na predavanjima. Pretpostaviti da operacije `setjmp()` i `longjmp()` čuvaju celokupan kontekst izvršavanja, tj. sve programski dostupne registre. Interfejs klase `ExternalEvent` treba da izgleda ovako:

```

const int NumOfInterruptEntries = ... ; // Number of interrupt entries

class ExternalEvent {
public:
    ExternalEvent(int interruptNo);
    void wait();

    static void interruptOccurrence(int interruptNo);
};

```

Konstruktor ove klase kreira jednu instancu događaja koji se signalizira prekidom sa brojem ulaza zadatim parametrom konstruktora. Operacija `wait()` blokira pozivajuću nit sve dok se prekid ne dogodi. Pojava prekida na ulazu *N* deblokira jednu nit blokiranu na događaju koji je vezan za prekid sa tim brojem ulaza *N*. Pretpostaviti da u sistemu već postoje prekidne rutine vezane za spoljašnje maskirajuće prekide tako da prekidna rutina za prekid na ulazu *N* poziva funkciju `ExternalEvent::interruptOccurrence(int)` sa argumentom koji predstavlja taj broj ulaza.

Rešenje

```

// ExternalEvent.h
void int_mask();
void int_unmask();

const int NumOfInterruptEntries = ... ; // Number of interrupt entries

class ExternalEvent {
public:
    ExternalEvent(int interruptNo);
    void wait();

    static void interruptOccurrence(int interruptNo);

protected:
    void signal ();

    void block ();
    void deblock ();

    ~ExternalEvent();
    static ExternalEvent* events[NumOfInterruptEntries];

private:
    int val, intNo;
    Queue blocked;
};

// ExternalEvent.cpp

ExternalEvent::ExternalEvent(int interruptNo) : val(0),
intNo(interruptNo){
    if (intNo>=0 && intNo<NumOfInterruptEntries) {
        // Razmisliti zašto je ovo neophodno.
        // Uputstvo: šta ako je pokazivač veličine više reči koje se upisuju neatomično u narednoj naredbi?
        int_mask();
        // add the event to the event list
        events[intNo]=this;
        int_unmask();
    }
}

ExternalEvent::~ExternalEvent(){
    // Isto kao i gore!
    int_mask();
    events[intNo]=0;
    int_unmask();
}

void ExternalEvent::interruptOccurrence(int interruptNo){
    if (interruptNo intNo>=0 && interruptNo intNo<NumOfInterruptEntries
    && events[interruptNo])
        events[interruptNo]->signal();
}

void ExternalEvent::block () {

```

```
    if (setjmp(Thread::runningThread->context)==0) {
        // Blocking:
        blocked->put(Thread::runningThread);
        Thread::runningThread = Scheduler::get();
        longjmp(Thread::runningThread->context);
    } else return;
}

void ExternalEvent::deblock () {
    // Deblocking:
    Thread* t = blocked->get();
    Scheduler::put(t);
}

void ExternalEvent::wait () {
    int_mask();
    if (--val<0) block();
    int_unmask();
}

void ExternalEvent::signal () {
    if (val++<0) deblock();
}
```

Sinhronizacija procesa (implementacija)

2. zadatak, drugi kolokvijum, april 2024.

- [Postavka](#)
- [Rešenje](#)

U školskom jezgri promena konteksta implementirana je korišćenjem date funkcije `yield()`, u sistemskom pozivu `dispatch()` i na svim ostalim mestima na sličan način kao što je dato.

Potrebno je implementirati sistemski poziv (nestatičku funkciju članicu):

```
int Thread::join(int* status = nullptr);
```

kojim pozivajuća nit čeka (suspenduje se ako je potrebno) dok se ne završi nit dete na koje ukazuje `this` i potom vraća 0; ako `this` ukazuje na nit koja nije dete pozivajuće niti, pozivalac ne čeka ništa i ova funkcija vraća -1. Ukoliko je parametar `status` različit od `null`, u dati izlazni parametar treba upisati povratni status završene niti deteta.

Za te potrebe treba implementirati i sledeće nestatičke funkcije članice:

- `void Thread::created()`: poziva je jezgro interno za datu novokreiranu nit (`this`), kada je ta nit kreirana;
- `void Thread::completed(int status)`: poziva je jezgro za datu nit (`this`), kada se ta nit završila; celobrojni argument je dostavila završena nit kao svoj povratni status.

Pretpostaviti da se objekti klase `Thread` nikada ne uništavaju (ili barem ne uništavaju dok se ne završe i unište roditelji). Ukoliko proširujete klasu `Thread` novim članovima, precizno navedite kako.

```
inline void yield (Thread* oldThr, Thread* newThr) {
    if (setjmp(oldThr->context) == 0)
        longjmp(newThr->context, 1);
}

void Thread::dispatch() {
    lock();
    Thread* oldThr = Thread::running;
    Scheduler::put(oldThr);
    Thread* newThr = Thread::running = Scheduler::get();
    yield(oldThr, newThr);
    unlock();
}
```

Rešenje

```
Thread* Thread::parent = nullptr;
bool Thread::isActive = false;
Thread* Thread::awaitedChild = nullptr;
int Thread::status = 0;

void Thread::created() {
    this->parent = Thread::running;
    this->isActive = true;
    this->awaitedChild = nullptr;
    this->status = 0;
}

void Thread::completed(int status) {
    this->isActive = false;
    this->status = status;
    if (this->parent && this->parent->awaitedChild == this) {
        this->parent->awaitedChild = nullptr;
        Scheduler::put(this->parent);
    }
}
```

```

    }
}

int Thread::join(int* status = nullptr) {
    lock();
    int ret = 0;
    Thread* thisThr = Thread::running;
    if (this->parent == thisThr)
        if (this->isActive)
            Thread::running->awaitedChild = child;
        else
            Scheduler::put(Thread::running);
    else {
        ret = -1;
        Scheduler::put(Thread::running);
    }
    Thread* newThr = Thread::running = Scheduler::get();
    yield(thisThr, newThr);
    if (ret == 0 && status)
        *status = this->status;
    unlock();
    return ret;
}

```

3. zadatak, drugi kolokvijum, maj 2023.

- [Postavka](#)
- [Rešenje](#)

U školskom jezgru promenu konteksta obavlja operacija `yield` data dole. Korišćenjem ove funkcije za promenu konteksta implementirati binarni semafor za međusobno isključenje (*mutex*) klasom `Mutex` sa sledećim ograničenjima:

- Operaciju `wait` ne sme da pozove nit koja je zaključala mutex.
- Operaciju `signal` sme da pozove samo nit koja je zaključala mutex.

Ukoliko neko od navedenih ograničenja nije zadovoljeno treba baciti celobrojni izuzetak `ERR_MUTEX`.

```

void yield() {
    Thread* oldt = Thread::running;
    Thread* newt = Thread::running = Scheduler::get();
    if (setjmp(oldt->context) == 0) {
        longjmp(newt->context, 1);
    }
}

```

Rešenje

```

class Mutex {
public:
    Mutex() : holder(0) {}
    void wait();
    void signal();
private:
    Queue blocked;
    Thread* holder;
};

void Mutex::wait() {
    lock();

```

```

    if (Thread::running == holder) {
        unlock();
        throw ERR_MUTEX;
    }
    if (holder == 0) {
        holder = Thread::running;
        Scheduler::put(holder);
    } else {
        blocked.put(Thread::running);
    }
    yield();
    unlock();
}

void Mutex::signal() {
    lock();
    if (Thread::running != holder) {
        unlock();
        throw ERR_MUTEX;
    }
    Thread* t = blocked.get();
    holder = t;
    if (t) {
        Scheduler::put(t);
    }
    yield();
    unlock();
}

```

2. zadatak, drugi kolokvijum, avgust 2023.

- [Postavka](#)
- [Rešenje](#)

- Korišćenjem standardnih funkcija `setjmp` i `longjmp` u školskom jezgru implementirati sledeće dve sistemske usluge (precizno navesti sva eventualna potrebna proširenja i napisati implementacije ovih funkcija):
 - `void Thread::suspend()`: statička funkcija članica klase `Thread` koja bezuslovno suspenduje pozivajuću (tekuću) nit.
 - `void Thread::resume()`: nestatička funkcija članica klase `Thread` koja reaktivira iz suspenzije nit za koju je pozvana, ukoliko je ta nit suspendovana sa `suspend()`; u suprotnom nema nikakvog efekta.
- Opisane usluge iskorišćene su za uslovnu sinhronizaciju dve niti na sledeći način:
 - nit na čiji objekat klase `Thread` ukazuje pokazivač `t` i koja čeka na uslov izvršava:


```
if (!condition) Thread::suspend();
```
 - nit koja signalizira ispunjenje uslova izvršava:


```
t->resume();
```

Da li je navedena upotreba ovih usluga ispravna? Precizno obrazložiti odgovor.

Rešenje

- U klasu `Thread` dodaje se privatni podatak član `isSuspended` tipa `bool` inicijalizovan na `false`.

```

void Thread::suspend() {
    lock();
    runningThread->isSuspended = true;
    if (setjmp(runningThread->context) == 0) {
        runningThread = Scheduler::get();
        longjmp(runningThread->context, 1);
    }
}

```

```

    }
    unlock();
}

void Thread::resume() {
    lock();
    if (this->isSuspended) {
        this->isSuspended = false;
        Scheduler::put(this);
    }
    unlock();
}

```

- Opisano rešenje ima problem utrkiivanja (*race condition*) koje se može dogoditi na sledeći način: uslov nije ispunjen; nit koja treba da čeka na uslov ispituje uslov i zaključuje da treba da se suspenduje, ali pre nego što to uradi nit koja signalizira uslov uradi *resume* bez efekta, pa se nakon toga prva nit bespotrebno suspenduje potencijalno neograničeno.

3. zadatak, drugi kolokvijum, avgust 2023.

- [Postavka](#)
- [Rešenje](#)

Klasa `Data` predstavlja neku strukturu podataka i poseduje konstruktor kopije. Objekti ove klase mogu se praviti operatorom `new` i brisati operatorom `delete`. Pokazivač `sharedData` je deljen između uporednih niti i ukazuje na deljeni objekat ove klase. Klasa `OptimisticCCTRL` implementira optimistički pristup kontroli konkurentnosti nad objektom na koga ukazuje deljeni pokazivač i namenjena je da se koristi na dole dat način. Svaki pokušaj transakcije izmene deljenog objekta mora da se započne pozivom operacije `startTrans` kojoj se dostavlja adresa pokazivača na deljeni objekat klase `Data`. Ova operacija vraća pokazivač na kopiju objekta nad kojim transakcija može da radi izmene (upis). Na kraju treba pozvati operaciju `commit` koja vraća `true` ako je uspela, `false` ako je detektovan konflikt; u ovom drugom slučaju transakcija mora da se pokuša iznova sve dok konačno ne uspe.

```

Data* sharedData = ...
OptimisticCCTRL* ctrl = new OptimisticCCTRL();

bool committed = false;
do {
    Data* myCopy = ctrl->startTrans(&sharedData);
    myCopy->write(...); // Write to *myCopy
    committed = ctrl->commit();
} while (!committed);

```

Implementirati klasu `OptimisticCCTRL` čiji je interfejs dat dole. Na raspolaganju je sistemska funkcija `cmp_and_swap` koja radi atomičnu proveru jednakosti vrednosti pokazivača `*shared` i `read` i ako su oni isti, u `*shared` upisuje vrednost `copy`.

```

class OptimisticCCTRL {
public:
    OptimisticCCTRL();
    Data* startTrans(Data** shared);
    bool commit();
};

bool cmp_and_swap(void** shared, void* read, void* copy);

```

Rešenje

```

class OptimisticCCTRL {
public:
    OptimisticCCTRL() {}

```

```

    Data* startTrans(Data** original);
    bool commit();
private:
    Data **shared = 0, *read = 0, *copy = 0;
};

Data* OptimisticCCtrl::startTrans(Data** original) {
    shared = original;
    read = *original;
    return copy = new Data(*read);
}

bool OptimisticCCtrl::commit() {
    bool ret = cmp_and_swap(shared, read, copy);
    if (!ret) delete copy;
    return ret;
}

```

2. zadatak, drugi kolokvijum, jun 2022.

- [Postavka](#)
- [Rešenje](#)

U školsko jezgro dodaje se koncept *uslova* (engl. *condition*) za uslovnu sinhronizaciju, podržan klasom `Condition` čiji je interfejs dat dole. Uslov može biti ispunjen ili neispunjen; inicijalna vrednost zadaje se konstruktorom. Niti koje smeju da nastave izvršavanje iza neke tačke samo ako je uslov ispunjen treba da pozovu operaciju `wait`, koja ih suspenduje ako uslov nije ispunjen. Bilo koja nit koja ispuni uslov to objavljuje pozivom operacije `set`; sve niti koje čekaju na taj uslov tada nastavljaju izvršavanje. Kada neka nit promeni uslov tako da on više nije ispunjen, treba da pozove operaciju `clear`. Implementirati klasu `Condition`.

```

class Condition {
public:
    Condition(bool init = false);
    void set();
    void clear();
    void wait();
};

```

Rešenje

```

class Condition {
public:
    Condition(bool init = false) : cond(init) {}
    void set();
    void clear() {
        lock();
        cond = false;
        unlock();
    }
    void wait();
private:
    bool cond;
    Queue blocked;
};

void Condition::wait() {
    lock();
    if (!cond) {
        if (setjmp(Thread::runningThread->context) == 0) {

```

```

        blocked.put(Thread::runningThread);
        Thread::runningThread = Scheduler::get();
        longjmp(Thread::runningThread->context, 1);
    }
}
unlock();
}
void Condition::set() {
    lock();
    cond = true;
    for (Thread* t = blocked.get(); t; t = blocked.get()) {
        Scheduler::put(t);
    }
    unlock();
}

```

2. zadatak, kolokvijum, jun 2021.

- [Postavka](#)
- [Rešenje](#)

U Školskom jezgru implementirana je statička operacija klase `Thread`

```
void Thread::yield (Thread* oldRunning, Thread* newRunning);
```

koja obavlja promenu konteksta oduzimajući procesor (tekućoj) niti na koju pokazuje parametar `oldRunning` i predajući procesor niti na koju pokazuje parametar `newRunning`. Korišćenjem ove operacije implementirati operacije `wait` i `signal` klase `Semaphore`, s tim da se uvek, pa i kod neblokirajućih poziva obavlja promena konteksta ukoliko raspoređivač odabere neku drugu nit za izvršavanje. U redu spremnih uvek se nalazi barem neka nit, makar nit *idle* koja ne radi ništa korisno (samo troši procesorsko vreme instrukcijama bez efekta).

Rešenje

```

void Semaphore::wait() {
    lock(lck);
    Thread* oldRunning = Thread::runningThread;
    if (--val < 0)
        this->blocked.put(oldRunning);
    else
        Scheduler::put(oldRunning);
    Thread* newRunning = Thread::runningThread = Scheduler::get();
    if (oldRunning != newRunning)
        Thread::yield(oldRunning, newRunning);
    unlock(lck);
}

void Semaphore::signal() {
    lock(lck);
    if (++val <= 0)
        Scheduler::put(this->blocked.get());
    Thread* oldRunning = Thread::runningThread;
    Scheduler::put(oldRunning);
    Thread* newRunning = Thread::runningThread = Scheduler::get();
    if (oldRunning != newRunning)
        Thread::yield(oldRunning, newRunning);
    unlock(lck);
}

```


1. zadatak, kolokvijum, avgust 2020.

- [Postavka](#)
- [Rešenje](#)

Školsko jezgro proširuje se konceptom koji služi za alokaciju i dealokaciju nedeljivog resursa u sistemu, ponaša se kao binarni semafor, a implementiran je klasom `Resource` čiji je interfejs dat u nastavku. Kada želi da zauzme neki deljeni resurs, nit treba da pozove operaciju `acquire` objekta ove klase koji služi za međusobno isključenje pristupa tom resursu, a kada ga oslobodi, treba da pozove operaciju `release`. Da bi se sprečila pojava mrtve blokade (*deadlock*), koristi se algoritam „čekaj ili umri“ (*wait-die*) koji funkcioniše na sledeći način. Svakoj niti pri samom kreiranju pridružuje se jedinstvena „vremenska marka“ (ceo broj dostupan u atributu `Thread::timestamp`), tako da starije niti imaju manju vrednost ove marke. Kada nit T_a zahteva resurs koji drži zauzeta nit T_b , onda:

- Ako je T_a starija nego T_b ($T_a < T_b$), nit T_a čeka suspendovana dok ne dobije resurs i tada operacija `acquire` vraća 1 (istu vrednost ova operacija vraća i kada je resurs slobodan, onda ga nit odmah dobija);
- Ako je T_a mlađa nego T_b ($T_a > T_b$), niti T_a se odbija zahtev tako što operacija `acquire` vraća 0, a nit onda tu situaciju obrađuje na odgovarajući način (pokušava ponovo ili odustaje).

Implementirati u potpunosti klasu `Resource`.

```
class Resource {
public:
    Resource ();
    int acquire();
    void release();
};
```

Rešenje

Pokazana je jedna jednostavna implementacija koja zadovoljava uslove, koristi jedostavan FIFO red zahteva, ali koja ima značajan stepen odbijanja zahteva (red zahteva je opadajući po vremenskoj marki). Moguće su i složenije varijante sa boljim učinkom (manje odbijenih zahteva), ali treba paziti na moguće izgladnjivanje.

```
typedef unsigned long Time;
const Time MAXTIME = (Time)-1;

class Resource {
public:
    Resource () : holder(0), last(MAXTIME) {}
    int acquire();
    void release();
protected:
    void block (); // Implementacija ista kao i za Semaphore
    void deblock (); // Implementacija ista kao i za Semaphore
private:
    Thread* holder;
    Time last;
    ThreadQueue blocked;
};

void Resource::acquire () {
    lock();
    if (!holder) {
        holder = Thread::running;
        last = holder->timestamp;
        unlock();
        return 1;
    } else
        if (Thread::running->timestamp >= last) {
            unlock();
```

```

    return 0;
} else {
    last = Thread::running->timestamp;
    block();
    unlock();
    return 1;
}
}

void Resource::release () {
    lock();
    Thread* thr = blocked.first();
    if (thr) {
        holder = thr;
        deblock();
    } else {
        holder = 0; last = MAXTIME;
    }
    unlock();
}

```

1. zadatak, drugi kolokvijum, maj 2019.

- [Postavka](#)
- [Rešenje](#)

Školsko jezgro proširuje se konceptom *mutex* koji predstavlja binarni semafor namenjen za međusobno isključenje pristupa kritičnim sekcijama ili deljenim resursima, ali uz mogućnost da nit može ugnežđeno „zauzimati“ (operacija *wait*) i „oslobađati“ (operacija *signal*) isti *mutex* na sledeći način:

- operaciju *signal* može da pozove samo nit koja je već zauzela *mutex* operacijom *wait*; u suprotnom, operacija *signal* vraća grešku;
- nit koja je već zauzela *mutex* operacijom *wait*, može ponovo izvršiti *wait* na njemu više puta, ali bez mrtve blokade sama sa sobom (samo prvi poziv *wait* slobodnog *mutex*-a ima efekta, ostali pozivi iz iste niti nemaju efekta); ova ista nit mora pozvati operaciju *signal* isti broj puta (upareno), tako da samo (poslednji) poziv operacije *signal* koji je uparen sa prvim efektivnim pozivom *wait* ima efekat oslobađanja *mutex*-a; u slučaju neuparenog poziva *signal* vratiti grešku.

Operacije *wait* i *signal* vraćaju celobrojnu vrednost, 0 u slučaju uspeha, negativnu vrednost u slučaju greške. Prikazati implementaciju klase *Mutex*, po uzoru na klasu *Semaphore* prikazanu na predavanjima (ne treba implementirati red čekanja niti, pretpostaviti da ta klasa postoji).

Rešenje

```

class Mutex {
public:
    Mutex () : lck(0), holder(0), nestingCnt(0) {}
    int wait();
    int signal();

protected:
    void block (); // Implementacija ista kao i za Semaphore
    void deblock (); // Implementacija ista kao i za Semaphore
private:
    unsigned lck, nestingCnt;
    ThreadQueue blocked;
    Thread* holder;
};

```

```

int Mutex::wait () {
    if (holder==Thread::running) {
        nestingCnt++;
        return 0;
    }
    lock(&lck);
    if (holder) block();
    holder = Thread::running;
    nestingCnt++;
    unlock(&lck);
    return 0;
}

int Mutex::signal () {
    if (holder!=Thread::running || nestingCnt==0) return -1; // Error
    lock(&lck);
    if (--nestingCnt==0) {
        holder = 0;
        if (!blocked.isEmpty()) deblock();
    }
    unlock(&lck);
    return 0;
}

```

1. zadatak, drugi kolokvijum, april 2017.

- [Postavka](#)
- [Rešenje](#)

Neki multiprocesorski operativni sistem sa preotimanjem (*preemptive*) dozvoljava preuzimanje tokom izvršavanja koda kernela, osim u kritičnim sekcijama, kada se pristupa deljenim strukturama podataka jezgra kojima mogu da pristupaju različiti procesori. Svaku takvu deljenu strukturu „štiti“ jedna celobrojna promenljiva koja služi za međusobno isključenje pristupa sa različitih procesora pomoću tehnike *spin lock*, dok se zabrana preuzimanja na datom procesoru obezbeđuje maskiranjem prekida. Međusobno isključenje pristupa kritičnoj sekciji, odnosno deljenoj strukturi, obavlja se pozivima operacija čiji je argument pokazivač na celobrojnu promenljivu koja „štiti“ datu deljenu strukturu:

```

void lock (short* lck);
void unlock (short* lck);

```

na ulazu, odnosno izlazu iz kritične sekcije. Pošto postoji potreba da kernel kod istovremeno pristupa različitim deljenim strukturama, kritične sekcije se mogu ugnježdavati. Zato demaskiranje prekida treba uraditi tek kada se izađe iz krajnje spoljašnje kritične sekcije (a ne pri svakom pozivu `unlock`).

Podaci koji su svojstveni svakom procesoru (svaki procesor ima svoju instancu ovih podataka) grupisani su u strukturu tipa `ProcessorPrivate`. (Ovu strukturu možete proširivati po potrebi.) Na raspolaganju su i sledeće funkcije ili makroi:

- `disable_interrupts()`: onemogućava spoljašnje prekide na ovom procesoru;
- `enable_interrupts()`: dozvoljava spoljašnje prekide na ovom procesoru;
- `test_and_set(short* lck)`: „obavlja“ instrukciju procesora tipa test-and-set;
- `this_processor()`: vraća celobrojni identifikator tekućeg procesora (na kome se kod izvršava);
- `ProcessorPrivate processor_private[NumOfProcessors]`: niz struktura sa „privatnim“ podacima svakog procesora.

Implementirati operacije `lock` i `unlock`.

Rešenje

```

struct ProcessorPrivate {
    int lock_count; // Counts the number of locks (nested critical sections)
    ...
    ProcessorPrivate () : lock_count(0),... {...}
};

void lock (short* lck) {
    disable_interrupts(); // Multiple calls of disable_intterupt are harmless
    processor_private[this_processor()].lock_count++;
    while (test_and_set(lck));
}

void unlock (short* lck) {
    *lck=0;
    if (--processor_private[this_processor()].lock_count==0)
        enable_interrupts();
}

```

1. zadatak, drugi kolokvijum, jun 2017.

- [Postavka](#)
- [Rešenje](#)

Školsko jezgro treba proširiti konceptom *barijere* („ograda“, „rampa“, engl. *barrier*) čiji je interfejs dat dole. Semantika ovog koncepta i operacija nad njim je sledeća:

- Barijera može biti u jednom od dva stanja: *otvorena* ili *zatvorena*. Barijera se inicijalizuje zadatim stanjem (argument konstruktora). Nit u čijem kontekstu se kreira barijera (tj. izvršava konstruktor) je njen *vlasnik*.
- `void Barrier::pass()`: ukoliko pozivajuća nit nije vlasnik ove barijere, ova operacija nema efekta – nit nastavlja svoje izvršavanje; ukoliko je pozivajuća nit vlasnik barijere, a barijera zatvorena, nit se suspenduje (blokira) sve dok se barijera ne otvori; ako je barijera otvorena, nit nastavlja izvršavanje bez blokade;
- `void Barrier::close()`: zatvara barijeru;
- `void Barrier::open()`: otvara barijeru i eventualno deblokira nit koja je na barijeri blokirana.

Implementirati ovu klasu `Barrier`. Ukoliko proširujete klasu `Thread`, precizno napisati kojim članovima.

```

class Barrier {
public:
    Barrier (int open=1);
    void open();
    void close();
    void pass();
};

```

Rešenje

U klasu `Thread` treba dodati sledeći član:

```
Barrier* blockedOnBarrier(0);
```

```

class Barrier {
public:
    Barrier (int open=1) : isOpen(open), owner(Thread::running) {}
    void open();
    void close();
    void pass();
private:
    int isOpen;
}

```

```

    Thread* owner;
};

void Barrier::pass () {
    if (this->owner!=Thread::running) return;
    lock();
    if (!this->isOpen) {
        Thread::running->blockedOnBarrier = this;
        if (setjmp(Thread::running->context)==0) {
            Thread::running = Scheduler->get();
            longjmp(Thread::running->context,1);
        }
    }
    unlock();
}

void Barrier::open () {
    lock();
    this->isOpen = 1;
    if (this->owner->blockedOnBarrier==this) {
        this->owner->blockedOnBarrier = 0;
        Scheduler::put(this->owner);
    }
    unlock();
}

inline void Barrier::close () {
    this->isOpen = 0;
}

```

1. zadatak, drugi kolokvijum, maj 2016.

- [Postavka](#)
- [Rešenje](#)

Školsko jezgro proširuje se konceptom *mutex* koji predstavlja binarni semafor, poput događaja (*event*), sa istom semantikom operacija *wait* i *signal* kao kod događaja, ali sa sledećim dodatnim ograničenjima koja su u skladu sa namenom upotrebe samo za međusobno isključenje kritične sekcije:

- inicijalna vrednost je uvek 1;
- operaciju *signal* može da pozove samo nit koja je zatvorila ulaz u kritičnu sekciju, odnosno koja je pozvala operaciju *wait*; u suprotnom, operacija *signal* vraća grešku;
- nit koja je već zatvorila *mutex* operacijom *wait*, ne može ponovo izvršiti *wait* na njemu.

Operacije *wait* i *signal* vraćaju celobrojnu vrednost, 0 u slučaju uspeha, negativnu vrednost u slučaju greške. Prikazati implementaciju klase *Mutex*, po uzoru na klasu *Semaphore* prikazanu na predavanjima (ne treba implementirati red čekanja niti, pretpostaviti da ta klasa postoji).

Rešenje

```

class Mutex {
public:
    Mutex () : val(1), lck(0), holder(0) {}
    int wait();
    int signal();

protected:
    void block (); // Implementacija ista kao i za Semaphore

```

```

    void deblock (); // Implementacija ista kao i za Semaphore
private:
    int val, lck;
    ThreadQueue blocked;
    Thread* holder;
};

int Mutex::wait () {
    if (holder==Thread::running) return -1; // Error
    lock(&lck);
    if (--val<0) block();
    holder = Thread::running;
    unlock(&lck);
    return 0;
}

int Mutex::signal () {
    if (holder!=Thread::running) return -1; // Error
    lock(&lck);
    holder = 0;
    if (val<0) {
        val++;
        deblock();
    } else // No blocked threads, val>=0
        val = 1;
    unlock(&lck);
    return 0;
}

```

2. zadatak, prvi kolokvijum, septembar 2016.

- [Postavka](#)
- [Rešenje](#)

Školsko jezgro proširuje se konceptom tzv. *tačke spajanja* ili *susreta* (engl. *join*, *rendez-vous*), kao jednostavne sinhronizacione primitive sa sledećim značenjem.

Program može kreirati objekat klase `Join`, čiji je interfejs dat dole, zadajući mu kao parametre konstruktora (pokazivače na) dve niti koje će se sinhronizovati (susretati, „spajati“) na ovom objektu. Iz konteksta ove dve niti (i samo njih) može se pozvati operacija `wait` ovog objekta. Tom operacijom se ove dve niti „spajaju“ (sinhronizuju, susreću), odnosno međusobno sačekuju, tako da ni jedna od njih neće nastaviti svoje izvršavanje dalje ako ona druga nije stigla do iste te tačke spajanja (tj. pozvala ovu operaciju `wait` na istom ovom objektu klase `Join`). Drugim rečima, niti nastavljaju izvršavanje iza poziva `wait` samo kada su obe stigle do te tačke. Dole je data implementacija sistemskog poziva `dispatch()` u školskom jezgru, kao primer kako su implementirana sva mesta promene konteksta. Implementirati u celini klasu `Join`, bez ikakve izmene ili dopune klase `Thread`. Operacija `wait` treba da vrati sledeće:

- -1 u slučaju da je ovu operaciju pozvala neka druga nit, osim one dve koje su definisane konstruktorom (ili bilo kakve druge greške);
- 0 u slučaju da je pozivajuća nit prva stigla do tačke spajanja, a 1 u suprotnom.

```

void dispatch () {
    lock();
    jmp_buf old = Thread::running->context;
    Scheduler::put(Thread::running);
    Thread::running = Scheduler::get();
    jmp_buf new = Thread::running->context;
    if (setjmp(old)==0) longjmp(new,1);
    unlock();
}

```

```
class Join {
public:
    Join (Thread* thread1, Thread* thread2);
    int wait ();
};
```

Rešenje

```
class Join {
public:
    Join (Thread* thread1, Thread* thread2);
    int wait ();
private:
    Thread* thread[2];
    int arrived[2];
};

Join::Join (Thread* t1, Thread* t2) {
    this->thread[0] = t1; this->thread[1] = t2;
    this->arrived[0] = this->arrived[1] = 0;
}

int Join::wait () {
    if (Thread::running!=this->thread[0] && Thread::running!=this->thread[1])
        return -1; // Exception
    lock();
    int ret;
    int myself = (Thread::running==this->thread[0])?0:1;
    int other = 1-myself;
    this->arrived[myself] = 1;
    if (this->arrived[other]) {
        ret = 1;
        Scheduler::put(this->thread[other]);
        this->arrived[0] = this->arrived[1] = 0;
    } else {
        ret = 0;
        jmp_buf old = Thread::running->context;
        Thread::running = Scheduler::get();
        jmp_buf new = Thread::running->context;
        if (setjmp(old)==0) longjmp(new,1);
    }
    unlock();
    return ret;
}
```

1. zadatak, drugi kolokvijum, septembar 2015.

- [Postavka](#)
- [Rešenje](#)

U školskom jezgru modifikuje se (uopštava) koncept brojačkog semafora podržan klasom **Semaphore** na sledeći način. Operacija `wait()` prima jedan nenegativan celobrojni argument n sa podrazumevanom vrednošću 1 i sa sledećim značenjem. Trenutna nenegativna vrednost semafora v predstavlja broj raspoloživih „žetona“. Operacijom `wait` pozivajuća nit „traži“ n „žetona“ (kod standardnog brojačkog semafora n je uvek podrazumevano 1). Ako je trenutna vrednost v semafora veća ili jednaka argumentu n operacije `wait`, ta vrednost v će biti umanjena za n , a nit će nastaviti izvršavanje bez blokade, jer je „dobila“ svih traženih n „žetona“. U suprotnom, ova nit će „uzeti“ v

„žetona“, i čekaće blokirana na semaforu dok se operacijama *signal* ne pojavi još $n-v$ žetona; kada se to dogodi, nit može nastaviti sa izvršavanjem (jer je dobila svih traženih n „žetona“). Operacija *signal* „obezbeđuje“ uvek jedan „žeton“, kao i kod standardnog brojačkog semafora.

```
class Semaphore {
public:
    Semaphore (unsigned int init=1);
    void wait(unsigned int n=1);
    void signal();
    int val ();
};
```

Za podršku ovoj implementaciji, u klasi *Thread* postoji nenegativno celobrojno polje *waiting* koje pokazuje na koliko još preostalih „žetona“ čeka data nit, ukoliko je blokirana na nekom semaforu. Osim toga, klasa *Queue* kojom se implementira FIFO red čekanja na semaforu ima operaciju *first()* koja vraća prvu nit u tom redu, ako je ima (0 ako je red prazan), bez izbacivanja te niti iz reda. Pomoćne operacije *block()* i *deblock()* klase *Semaphore* su iste kao i u postojećoj implementaciji školskog jezgra.

Dati izmenjenu implementaciju operacija *wait()* i *signal()*.

Rešenje

```
void Semaphore::wait (unsigned int n = 1) {
    lock(lck);
    val -= n;
    if (val<0) {
        Thread::running->waiting = -val;
        val = 0;
        block();
    }
    unlock(lck);
}

void Semaphore::signal () {
    lock(lck);
    Thread* thr = blocked.first();
    if (thr) { // There are blocked threads, val==0
        thr->waiting--;
        if (thr->waiting==0)
            deblock();
    } else // No blocked threads, val>=0
        val++;
    unlock(lck);
}
```

1. zadatak, drugi kolokvijum, april 2014.

- [Postavka](#)
- [Rešenje](#)

Modifikovati operaciju *wait* klase *Semaphore* u školskom jezgru tako da ima izmenjenu deklaraciju datu dole sa sledećim dodatnim mogućnostima i izmenjenim ponašanjem:

- ukoliko je vrednost argumenta *toBlock* različita od 0, operacija se ponaša na standardan način i vraća 1 ako se pozivajuća nit blokirala (suspendovala), a 0 ako nije;
- ukoliko je vrednost argumenta *toBlock* jednaka 0, ako je vrednost semafora nula (ili manja od 0), pozivajuća nit se neće blokirati, vrednost semafora se neće promeniti, a operacija će odmah vratiti -1; inače, ukoliko je vrednost semafora veća od nule, operacija se ponaša na standardan način (i vraća 0 pošto se nit nije blokirala).

```
int Semaphore::wait (int toBlock);
```


Rešenje

```

int Semaphore::wait (int toBlock) {
    lock(lck);
    int ret = 0;
    if (!toBlock && val<=0)
        ret = -1;
    else
        if (--val<0) {
            ret = 1;
            block();
        }
    unlock(lck);
    return ret;
}

```

1. zadatak, drugi kolokvijum, septembar 2014.

- [Postavka](#)
- [Rešenje](#)

U nekom operativnom sistemu postoje samo standardni brojački semafori podržani klasom **Semaphore** čiji je interfejs isti kao u školskom jezgru:

```

class Semaphore {
public:
    Semaphore (int init=1);
    void wait();
    void signal();
    int val ();
};

```

Pomoću ovih semafora implementirati koncept događaja (binarnog semafora), podržanog klasom **Event** sa sledećim interfejsom:

```

class Event {
public:
    Event ();
    void wait();
    void signal();
    int val ();
};

```

Rešenje

```

class Event {
public:
    Event () : mySem(0), mutex(1) {}
    void wait() { mySem.wait(); }
    void signal();
    int val () { return mySem.val(); }
private:
    Semaphore mySem, mutex;
};

inline void Event::signal () {
    mutex.wait();
    if (mySem.val()<1) mySem.signal();
}

```

```
mutex.signal();
}
```

2. zadatak, drugi kolokvijum, april 2013.

- [Postavka](#)
- [Rešenje](#)

Data je sledeća implementacija operacije `Semaphore::lock()` koja obezbeđuje zaključavanje semafora (međusobno isključenje operacija na semaforu) u kodu školskog jezgra za višeprocorski sistem. Operacija `swap()` implementirana je pomoću odgovarajuće atomične instrukcije procesora. Objasniti zašto ova implementacija operacije `lock()` nije dobra, a onda je popraviti.

```
extern void swap (int*, int*);

void Semaphore::lock() {
    int zero = 0;
    mask_interrupts();
    while (!this->lck) {}
    swap(&zero, &(this->lck));
}
```

Rešenje

Data implementacija nije dobra jer je moguće utrkiavanje (*race condition*) između izvršavanja istog ovog koda za isti semafor na dva (ili više) procesora. I jedan i drugi procesor mogu da izvrše petlju `while` i iz nje izađu, uporedno pročitavši iz atributa `lck` vrednost 1, a potom izvrše `swap` i izađu iz operacije `lock`, odnosno uđu u operaciju semafora. Ispravljena verzija je sledeća (ima i nešto malo efikasnijih, kako je prikazano na predavanjima):

```
void Semaphore::lock() {
    int zero = 0;
    mask_interrupts();
    while (!zero) swap(&zero, &(this->lck));
}
```

2. zadatak, drugi kolokvijum, septembar 2013.

- [Postavka](#)
- [Rešenje](#)

U nekom operativnom sistemu svi sistemski pozivi izvršavaju se kao softverski prekid koji skače na prekidnu rutinu označenu kao `sys_call`, dok se sama identifikacija sistemskog poziva i njegovi parametri prenose kroz registre procesora. Jezgro tog operativnog sistema je višenitno – poseduje više internih kernel niti koje obavljaju različite poslove: izvršavaju uporedne I/O operacije, vrše druge interne poslove jezgra itd. Prilikom obrade sistemskog poziva `sys_call`, prema tome, prekidna rutina samo oduzima procesor tekućem korisničkom procesu uz čuvanje njegovog konteksta i dodeljuje procesor tekućoj kernel niti.

Jedna od tih niti zadužena je za samu obradu zahteva koju je korisnički proces postavio u sistemskom pozivu. Ona preuzima parametre poziva iz sačuvanih registara i, na osnovu vrste sistemskog poziva, poziva odgovarajuću proceduru kernela za taj sistemski poziv.

Ovde se posmatraju sistemski pozivi za operacije na binarnim semaforima (događajima). Unutar kernela, binarni semafori su realizovani klasom `Event`, poput klase `Semaphore` date u školskom jezgru. Realizovati operacije `wait` i `signal` ove klase, koje poziva gore pomenuta kernel nit kada obrađuje te sistemske pozive. Na raspolaganju su operacije `lock()` i `unlock()` koje obezbeđuju međusobno isključenje unutar jezgra, kao i klasa `Scheduler` koja implementira raspoređivanje, kao u školskom jezgru. Na korisnički proces koji je izvršio sistemski poziv ukazuje pokazivač `runningUserThread` unutar kernela.

Rešenje

```

void Event::wait () {
    lock(lck);
    if (--val<0) {
        blocked.put(runningUserThread);
        runningUserThread = Scheduler::get();
    }
    unlock(lck);
}

void Event::signal () {
    lock(lck);
    if (val++<0)
        Scheduler::put(blocked.get());
    else val = 1;
    unlock(lck);
}

```

2. zadatak, drugi kolokvijum, septembar 2012.

- [Postavka](#)
- [Rešenje](#)

U nekom višeprocorskom sistemu ne postoji stanje suspendovanih (blokiranih) niti, već su sve aktivne niti uvek spremne, dok se operacije čekanja na semaforu i drugim sinhronizacionim primitivama realizuju uposlenim čekanjem. Jezgro sistema povremeno (na prekid od tajmera) jednostavno preotima neki procesor od tekuće niti i predaje ga nekoj drugoj aktivnoj niti. U sistemu su implementirane operacije

```

void lock (int lck);
void unlock (int lck);

```

koje realizuju međusobno isključenje nad deljenom strukturom podataka zaštićenom celobrojnim ključem `lck` maskiranjem prekida i mehanizmom *spin-lock* za višeprocorski pristup.

Realizovati klasu `Semaphore` koja apstrahuje standardni brojački semafor, sa uposlenim čekanjem.

Rešenje

```

class Semaphore {
public:
    Semaphore (int init=1) : v(init), lck(0) {}
    void wait ();
    void signal ();
private:
    int v, lck;
};

void Semaphore::wait () {
    int done = 0;
    while (!done) {
        lock(lck);
        if (v>0) v--, done=1;
        unlock(lck);
    }
}

void Semaphore::signal () {
    lock(lck);

```

```

v++;
unlock(lck);
}

```

2. zadatak, drugi kolokvijum, maj 2011.

- [Postavka](#)
- [Rešenje](#)

U klasi `Semaphore` postoji privatni podatak član `isLocked` tipa `int` koji služi da obezbedi međusobno isključenje pristupa strukturi semafora u višeprocorskom operativnom sistemu. Ako je njegova vrednost 1, kod koji se izvršava na nekom procesoru je zaključao semafor za svoj isključivi pristup; ako je vrednost 0, pristup semaforu je slobodan. Korišćenjem operacije `test_and_set()` koja je implementirana korišćenjem odgovarajuće atomične instrukcije procesora, realizovati operaciju `Semaphore::lock()` koja treba da obezbedi međusobno isključenje pristupa strukturi semafora u višeprocorskom operativnom sistemu, ali tako da bude efikasnija od dole date implementacije tako što ne poziva operaciju `test_and_set` ako je semafor već zaključan od strane drugog procesora, pošto ta operacija ima veće režijske troškove na magistrali računara.

```

void Semaphore::lock () {
    while (test_and_set(this->isLocked));
}

```

Rešenje

```

void Semaphore::lock () {
    for (int acquired = 0; !acquired;) {
        while (this->isLocked);
        acquired = !test_and_set(this->isLocked);
    }
}

```

2. zadatak, drugi kolokvijum, septembar 2011.

- [Postavka](#)
- [Rešenje](#)

Izmeniti datu implementaciju operacije `wait` na semaforu u školskom jezru tako da, pre nego što odmah blokira pozivajuću nit ukoliko je semafor zatvoren, najpre pokuša da uposlano sačeka, ali ograničeno, ponavljajući petlju čekanja najviše `SemWaitLimit` puta. Ostatak klase `Semaphore` se ne menja.

```

void Semaphore::wait () {
    lock(lck);
    if (--val<0)
        block();
    unlock(lck);
}

```

Rešenje

```

void Semaphore::wait () {
    for (int i=SemWaitLimit; val<=0 && i>0; i--);
    lock(lck);
    if (--val<0)
        block();
    unlock(lck);
}

```

1. zadatak, drugi kolokvijum, maj 2010.

- [Postavka](#)

- [Rešenje](#)

U nekom operativnom sistemu svi sistemski pozivi izvršavaju se kao softverski prekid koji skače na prekidnu rutinu označenu kao `sys_call`, dok se sama identifikacija sistemskog poziva i njegovi parametri prenose kroz registre procesora. Jezgro tog operativnog sistema je višenitno – poseduje više internih kernel niti koje obavljaju različite poslove: izvršavaju uporedne I/O operacije, vrše druge interne poslove jezgra itd. Prema tome, sve potrebne radnje prilikom promene konteksta korisničkih procesa (osim samog čuvanja i restauracije konteksta procesora), kao što su smeštanje PCB korisničkog procesa koji je do tada bio tekući u odgovarajući red (spremnih ili blokiranih, u zavisnosti od situacije), izbor novog tekućeg procesa iz skupa spremnih, promena memorijskog konteksta, obrada samog konkretnog sistemskog poziva, itd. obavljaju se u kontekstu internih kernel niti. Prilikom obrade sistemskog poziva u prekidnoj rutini `sys_call`, prema tome, samo se oduzima procesor tekućem korisničkom procesu i dodeljuje se tekućoj kernel niti.

Na PCB tekućeg korisničkog procesa ukazuje globalni pokazivač `runningUserProcess`, a na PCB tekuće interne niti jezgra ukazuje globalni pokazivač `runningKernelThread`. I interne niti jezgra vrše promenu konteksta između sebe na isti način, pozivom softverskog prekida koji ima istu internu strukturu kao i rutina `sys_call`.

Posebna interna kernel nit zadužena je za obradu sistemskog poziva izdatog od strane korisničkog procesa. Ova nit najpre određuje o kom sistemskom pozivu se radi i preuzima njegove parametre, a onda poziva odgovarajuće operacije jezgra koje obavljaju zahtevanu sistemsku uslugu, recimo operacije `wait` ili `signal` na semaforu.

Po uzoru na implementaciju klase `Semaphore` u školskom jezgru, implementirati ovu klasu za potrebe opisanog jezgra. Međusobno isključenje koda operacija `wait` i `signal` obezbeđuju uobičajene operacije `lock` i `unlock`. Operacije `lock` i `unlock` su implementirane (ne treba ih implementirati).

Rešenje

```
void Semaphore::wait () {
    lock(lck);
    if (--val<0) {
        blocked.put(runningUserProcess);
        runningUserProcess = UserProcessScheduler::get();
    }
    unlock(lck);
}

void Semaphore::signal () {
    lock(lck);
    if (val++<0)
        UserProcessScheduler::put(blocked.get());
    unlock(lck);
}
```

Pomoćne operacije `block()` i `unblock()` više nisu potrebne (izbacuju se). Ostatak definicije klase `Semaphore` ostaje isti.

1. zadatak, drugi kolokvijum, maj 2009.

- [Postavka](#)
- [Rešenje](#)

Realizaciju semafora pomoću C++ klase `Semaphore` date na predavanjima modifikovati tako da se promena konteksta može dogoditi i kod neblokirajućih poziva `wait` i `signal` na semaforu (tj. pri svakom pozivu `signal`, kao i pri pozivu `wait` i kada ne treba blokirati pozivajući proces).

Rešenje

```
void Semaphore::wait () {
    lock();
    if (setjmp(Thread::runningThread->context)==0) {
```

```

        if (--val<0)
            blocked.put(Thread::runningThread);
        else
            Scheduler::put(Thread::runningThread);
        Thread::runningThread = Scheduler::get();
        longjmp(Thread::runningThread->context,1);
    }
    unlock();
}

void Semaphore::signal () {
    lock();
    if (val++<0)
        Scheduler::put(blocked.get());
    if (setjmp(Thread::runningThread->context)==0) {
        Scheduler::put(Thread::runningThread);
        Thread::runningThread = Scheduler::get();
        longjmp(Thread::runningThread->context,1);
    }
    unlock();
}

```

Pomoćne operacije `block()` i `unblock()` više nisu potrebne (izbacuju se). Ostatak definicije klase `Semaphore` ostaje isti.

1. zadatak, drugi kolokvijum, maj 2008.

- [Postavka](#)
- [Rešenje](#)

U jezgru nekog multiprocesorskog operativnog sistema realizovane su sledeće operacije koje se koriste za obezbeđenje atomičnosti operacija nad semaforom:

```

void Semaphore::lock (int* lck) {
    disable_interrupts();
    while test_and_set(lck);
}

void Semaphore::unlock (int* lck) {
    *lck=0;
    enable_interrupts();
}

```

Koncept semafora realizovan je klasom `Semaphore`, čiji su fragmenti prikazani u nastavku:

```

class Semaphore {
...
private:
    static int commonLock; // initialized to 0
    int myLock; // initialized to 0
...
};

```

Komentarisati razliku između sledeće dve realizacije operacija nad semaforom, odnosno njihovog ulaznog i izlaznog protokola sa ciljem obezbeđenja atomičnosti:

```

lock(&commonLock);          lock(&myLock);
...                          ...
unlock(&commonLock);        unlock(&myLock);

```

Posebno komentarisati sledeće aspekte:

- da li su obe varijante sasvim korektne i pod kojim uslovima?
- stepen paralelizma koji ove varijante omogućavaju.

Rešenje

Prva varijanta zaključava semafor (kao deljeni resurs) maskiranjem prekida i uposlenim čekanjem (operacijom tipa *test and set*) nad jednom zajedničkom varijablom (`commonLock`) koja se koristi za sve semafore (instance klase `Semaphore`), dok druga varijanta radi to isto samo korišćenjem posebne varijable za svaki semafor (`myLock`). Prva varijanta ne dozvoljava ulaz u kod operacije nad semaforom nekom procesoru ako je bilo koji drugi procesor ušao u kod operacije nad bilo kojim drugim semaforom, dok su kod druge varijante moguća paralelna izvršavanja koda operacija nad različitim semaforima. Odatle slede zaključci:

- Ukoliko kod operacija nad semaforom ne uzrokuje konflikte na deljenim strukturama koje se koriste u izvršavanjima na različitim procesorima, onda su obe varijante korektne. U suprotnom, ukoliko ovaj kod uzrokuje ovakve konflikte, npr. korišćenjem istog reda spremnih procesa za različite procesore bez obezbeđenja međusobnog isključenja nad tom strukturom, onda je samo prva varijanta korektna (pod uslovom da ne pravi konflikte sa drugim uslugama operativnog sistema), a druge ne.
- Druga varijanta omogućava znatno veći stepen paralelizma, jer ne zaustavlja napredovanje operacije nad jednim semaforom na jednom procesoru zbog toga što je drugi procesor ušao u operaciju nad drugim semaforom, kako to čini prva varijanta.

1. zadatak, drugi kolokvijum, maj 2007.

- [Postavka](#)
- [Rešenje](#)

Na raspolaganju su sledeće funkcije:

- `swap(int*,int*)` koja, korišćenjem odgovarajuće mašinske instrukcije, atomično zamenjuje vrednosti dve memorijske reči na adresama zadatim argumentima (C/C++ tip `int` je uvek veličine mašinske reči).
- `set_interrupts(int)` koja maskira sve nemaskirajuće prekide ako je argument jednak 0, odnosno demaskira ih ako je argument jednak 1.

Korišćenjem ovih funkcija realizovati standardni brojački semafor (klasu `Semaphore`) sa operacijama `wait()` i `signal()`, pri čemu se koristi uposleno čekanje umesto suspenzije (blokiranja) pozivajućeg procesa. Realizacija treba da bude prilagođena multiprocesorskom sistemu sa preuzimanjem (*preemptive*).

Rešenje

```
class Semaphore {
public:
    Semaphore (unsigned int value=1);
    void wait ();
    void signal();
protected:
    inline void lock();    // inline nije semantički bitan
    inline void unlock(); // inline nije semantički bitan
private:
    int lock;
    unsigned int val;
};

Semaphore::Semaphore (unsigned int v) : lock(0), val(v) {}

void Semaphore::lock () {
    set_interrupts(0);
    for (int lck=1; lck;) swap(&lck,&lock);
}
```

```
void Semaphore::unlock () {
    lock=0;
    set_interrupts(1);
}

void Semaphore::wait () {
    for (int done==0; !done; ) {
        while(val==0);
        lock();
        if (val==0) { unlock(); continue; }
        done=1;
        val--;
        unlock();
    }
}

void Semaphore::signal () {
    lock();
    val++;
    unlock();
}
```


Sinhronizacija procesa (interfejs)

3. zadatak, drugi kolokvijum, april 2024.

- [Postavka](#)
- [Rešenje](#)

Korišćenjem školskog jezgra pravi se softver za kontrolu ulaza u neku javnu garažu. Kontrolu obavlja klasa `GarageController` čiji je interfejs dat dole. Objekat ove klase inicijalizuje se parametrom koji zadaje broj slobodnih mesta za parkiranje unutar garaže. Kada neki automobil dođe ispred rampe na ulazu garaže, odgovarajuća nit koja upravlja tom rampom poziva operaciju `entry` objekta ove klase. Ukoliko u garaži trenutno nema slobodnih mesta, ova operacija blokira pozivajuću nit dok se u garaži ne pojavi slobodno mesto. Ukoliko slobodnih mesta ima, ova operacija ažurira broj slobodnih mesta u garaži smanjujući ga za jedan i vraća kontrolu pozivaocu. Kada automobil prođe kroz izlaznu rampu, nit koja kontroliše tu rampu poziva operaciju `exit` objekta ove klase, čime se broj slobodnih mesta uvećava za jedan. Operacija `getFreeSlots` vraća broj trenutno slobodnih mesta u garaži.

Implementirati klasu `GarageController` u potpunosti korišćenjem semafora školskog jezgra.

```
class GarageController {
public:
    GarageController(unsigned slots);
    ~GarageController();
    void entry();
    void exit();
    unsigned getFreeSlots() const;
};
```

Rešenje

```
class GarageController {
public:
    GarageController(unsigned slots) {
        mySem = new Semaphore(slots);
    }
    ~GarageController() { delete mySem; }
    void entry() { mySem->wait(); }
    void exit() { mySem->signal(); }
    unsigned getFreeSlots() const {
        int n = mySem->val();
        return (n >= 0) ? n : 0;
    }
private:
    Semaphore* mySem;
};
```

3. zadatak, drugi kolokvijum, jun 2022.

- [Postavka](#)
- [Rešenje](#)

Više uporednih niti-pisaca upisuje izračunate celobrojne dvodimenzionalne koordinate (x, y) na koje treba pomeriti robota u deljeni objekat klase `SharedCoord` čiji je interfejs dat dole; svaka ovakva nit nezavisno upisuje svoj par izračunatih koordinata pozivom operacije `write` ove klase. Jedna nit-čitalac periodično očitava par koordinata iz tog deljenog objekta i pomera robota na očitane koordinate; ova nit to radi pozivom operacije `read` ove klase, nezavisno od pisaca, svojim tempom, tako da svaki put čita poslednje upisane koordinate (može pročitati više puta isti par koordinata ili neke izračunate koordinate i preskočiti).

Implementirati klasu `SharedCoord` uz neophodnu sinhronizaciju korišćenjem najmanjeg broja semafora školskog jezgra.

```
class SharedCoord {
public:
    SharedCoord();
    void read(int& x, int& y);
    void write(int x, int y);
};
```

Rešenje

```
class SharedCoord {
public:
    SharedCoord();
    void read(int& x, int& y);
    void write(int x, int y);
private:
    Semaphore mutex;
    int x, y;
};

inline SharedCoord() : mutex(1) {}
inline void SharedCoord::read(int& x_, int& y_) {
    mutex.wait();
    x_ = this->x;
    y_ = this->y;
    mutex.signal();
}

inline void SharedCoord::write(int x_, int y_) {
    mutex.wait();
    this->x = x_;
    this->y = y_;
    mutex.signal();
}
```

2. zadatak, kolokvijum, jul 2021.

- [Postavka](#)
- [Rešenje](#)

Korišćenjem sistemskog poziva *fork* i POSIX semafora za sinhronizaciju napisati program koji, kad se nad njim pokrene proces, pokrene jedno svoje dete, a potom ta dva procesa strogo naizmenično na svoj standardni izlaz ispisuju po jedno „A“ (proces-roditelj), odnosno „B“ (proces-dete) tačno po 10 puta, nakon čega se završavaju. Za sistemski poziv *fork* pretpostavlja se sledeće:

- proces-dete nasleđuje standardni izlaz od roditelja, a operativni sistem obezbeđuje međusobno isključenje ispisa na isti standardni izlaz;
- proces-dete nasleđuje deskriptore semafora koje je proces-roditelj otvorio, tako da isti deskriptori ukazuju na isti semafor u sistemu koji je deljen između roditelja i deteta; oba procesa moraju da zatvore semafor kada im više nije potreban.

Rešenje

```
#include <fcntl.h>
#include <sys/stat.h>
#include <semaphore.h>
#include <unistd.h> // for fork()
#include <stdio.h> // for printf

sem_t *sem1, *sem2;
```

```

int main() {
    sem1 = sem_open("/sem1", O_CREAT | O_EXCL, O_RDWR, 1);
    if (sem1 == NULL) return -1;
    sem2 = sem_open("/sem2", O_CREAT | O_EXCL, O_RDWR, 0);
    if (sem2 == NULL) {
        sem_close(sem1);
        return -1;
    }
    pid_t pid = fork();
    if (pid < 0) return -1;
    for (int i = 0; i < 10; i++) {
        if (pid > 0) {
            if (sem_wait(sem1) < 0) break;
            printf("A");
            fflush(stdout);
            if (sem_post(sem2) < 0) break;
        } else {
            if (sem_wait(sem2) < 0) break;
            printf("B");
            fflush(stdout);
            if (sem_post(sem1) < 0) break;
        }
    }
    sem_close(sem1);
    sem_unlink("/sem1");
    sem_close(sem2);
    sem_unlink("/sem2");
    return 0;
}

```

2. zadatak, kolokvijum, jun 2020.

- [Postavka](#)
- [Rešenje](#)

Korišćenjem školskog jezgra i date klase `RowAdder`, implementirati funkciju `mat_add` koja sabira elemente svake vrste matrice `mat` i rezultat tog sabiranja smešta u odgovarajući element niza `res`, i to radi uporedo tako da kontrolu vraća pozivaocu tek kada su uporedna sabiranja svih vrsta matrice završena.

```

class RowAdder : public Thread {
public:
    RowAdder (int arr[], int cnt, int* res, Semaphore* sem)
        : a(arr), n(cnt), r(res), s(sem) {}
protected:
    virtual void run ();
private:
    int *a, n, *r;
    Semaphore* s;
};

void RowAdder::run () {
    *r = 0;
    for (int i=0; i<n; i++) *r += a[i];
    if (s) s->signal();
}

const int M = ..., N = ...;
int mat[M][N];

```

```
int res[M];

void mat_add ();
```

Rešenje

```
void mat_add () {
    Thread* thr[M];
    Semaphore* sem[M];

    for (int i=0; i<M; i++) {
        sem[i] = new Semaphore(0);
        thr[i] = new RowAdder(mat[i],N,&res[i],sem[i]);
        thr[i]->start();
    }

    for (int i=0; i<M; i++) {
        sem[i]->wait();
        delete sem[i];
        delete thr[i];
    }
}
```

1. zadatak, drugi kolokvijum, jun 2019.

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu postoje sledeći sistemski pozivi:

- `sem_t* sem_create(unsigned init_value)`: kreira semafor sa zadatom inicijalnom vrednošću;
- `int sem_wait(sem_t* semaphore)`
`int sem_signal(sem_t* semaphore)`: standardne operacije *wait* i *signal* na zadatom semaforu;
- `int thread_create(void*(void*),void*)`: kreira nit nad funkcijom na koju ukazuje prvi argument; ta funkcija prima jedan argument tipa `void*` i ne vraća rezultat; novokreirana nit poziva tu funkciju sa stvarnim argumentom jednakim drugom argumentu ovog sistemskog poziva; sistemski poziv vraća PID kreirane niti.

Sve funkcije vraćaju negativnu vrednost ili *null* pokazivač u slučaju greške. Korišćenjem ovih sistemskih poziva, uz pomoću semafora za sinhronizaciju, napisati program koji kreira *N* niti koje će pozivati funkciju `process` bez argumenata jedna nakon druge, strogo u poretku numeracije tih niti (nit *i*+1 treba da pozove funkciju `process` kada nit *i* završi sa tim pozivom). Ignorirati sve greške.

Rešenje

```
const int N = ...;
struct sem_t;
sem_t* semaphores[N];

void initSems () {
    semaphores[0] = sem_create(1);
    for (int i=1; i<N; i++)
        semaphores[i] = sem_create(0);
}

void threadBody (void* ps) {
    sem_t** psem = (sem_t**)ps;
    sem_wait(*psem++);
```

```

process();
if (psem < semaphores+N)
    sem_signal(*psem);
}

int main () {
    initSems();
    for (int i=0; i<N; i++)
        thread_create(threadBody,semaphores+i);
}

```

1. zadatak, drugi kolokvijum, april 2018.

- Postavka
- Rešenje

U nastavku je dat parcijalni opis neznatno izmenjenog (pojednostavljenog) POSIX API.

```

#include <fcntl.h>
/* For O_* constants */
#include <sys/stat.h>
sem_t *sem_open(const char *name, int oflag, unsigned int value);
#include <semaphore.h>
int sem_post(sem_t *sem);
int sem_wait(sem_t *sem);
int sem_close(sem_t *sem);

```

sem_open() creates a new POSIX semaphore or opens an existing semaphore. The semaphore is identified by name of the form */somenam*; that is, a null-terminated string of up to *NAME_MAX-4* (i.e., 251) characters consisting of an initial slash, followed by one or more characters, none of which are slashes. Two processes can operate on the same named semaphore by passing the same name to *sem_open()*.

The *oflag* argument specifies flags that control the operation of the call. (Definitions of the flags values can be obtained by including *<fcntl.h>*.) If *O_CREAT* is specified in *oflag*, then the semaphore is created if it does not already exist. If both *O_CREAT* and *O_EXCL* are specified in *oflag*, then an error is returned if a semaphore with the given name already exists. If *O_CREAT* is specified in *oflag*, then the value argument must be specified. The value argument specifies the initial value for the new semaphore.

If *O_CREAT* is specified, and a semaphore with the given name already exists, then value is ignored. After the semaphore has been opened, it can be operated on using *sem_post()* and *sem_wait()*. When a process has finished using the semaphore, it can use *sem_close()* to close the semaphore.

On success, *sem_open()* returns the address of the new semaphore; this address is used when calling other semaphore-related functions.

sem_post() increments (unlocks) the semaphore pointed to by *sem*. If the semaphore's value consequently becomes greater than zero, then another process or thread blocked in a *sem_wait()* call will be woken up and proceed to lock the semaphore.

sem_wait() decrements (locks) the semaphore pointed to by *sem*. If the semaphore's value is greater than zero, then the decrement proceeds, and the function returns, immediately. If the semaphore currently has the value zero, then the call blocks until either it becomes possible to perform the decrement (i.e., the semaphore value rises above zero).

```

#include <fcntl.h>
pid_t getpid(void);

```

getpid() shall return the process ID of the calling process.

Korišćenjem ovih sistemskih poziva, implementirati apstrakciju **Mutex** koja predstavlja sinhronizacionu primitivu za međusobno isključenje, koja se može koristiti samo u te svrhe, tj. uz ograničenje da samo proces koji je zaključao kritičnu sekciju može da je otključa: ukoliko to nije zadovoljeno, operacije **exit** treba da vrati grešku (negativnu vrednost). Ova apstrakcija treba da ima sledeći objektno orijentisani interfejs:

```
class Mutex {
public:
    Mutex (const char *name);
    int entry ();
    int exit();
};
```

Rešenje

```
#include <fcntl.h>
#include <sys/stat.h>
#include <semaphore.h>

class Mutex {
public:
    Mutex (const char *name);
    ~Mutex ();
    int entry ();
    int exit();
private:
    sem_t* mutex;
    pid_t pid;
};

Mutex::Mutex (const char *name) {
    this->mutex = sem_open(name, O_CREAT, 1);
}

int Mutex::entry () {
    if (!this->mutex) return -1;
    int ret = sem_wait(this->mutex);
    if (ret<0) return ret;
    this->pid = getpid(); // Must not be done before sem_wait
    return 0;
}

int Mutex::exit () {
    if (!this->mutex || this->pid!=getpid()) return -1; // error
    return sem_post(this->mutex);
}

Mutex::~~Mutex () {
    sem_close(this->mutex);
}
```

1. zadatak, drugi kolokvijum, jun 2018.

- [Postavka](#)
- [Rešenje](#)

U klasu Semaphore u školskom jezgru dodata je statička funkcija članica

```
int Semaphore::waitOr(Semaphore*, Semaphore*);
```

koja čeka na dva semafora po *ili* uslovu, tj. dok bilo koji od dva semafora ne postane veći od 0. Ukoliko je taj uslov ispunjen, ova operacija smanjuje samo jedan od njih za 1 (nikada oba, čak i ako su oba veća od 0), i vraća 1 ili 2, ukazujući na to koji od njih je umanjila za 1 (prvi ili drugi).

Potrebno je implementirati ograničeni bafer čiji je interfejs dat dole. Ovom baferu dostavljaju se uporedo, od strane

dve vrste proizvođača, elementi tipa `int` operacijom `put1`, odnosno elementi tipa `double` operacijom `put2`. Ove elemente treba smeštati u dva različita interna bafera (za svaki tip elementa po jedan). Sa druge strane, potrošač operacijom `get` treba da dobije bilo koji od dve vrste elemenata, koji god je na raspolaganju. Ova operacija s treba da postavi uzetu vrednost u jedan od dva objekta na koji ukazuju argumenti, i da vrati vrednost 1 ili 2, u zavisnosti koji od ta dva objekta je postavio vrednost.

```
class BoundedBuffer {
public:
    BoundedBuffer ();

    void put1 (int);
    void put2 (double);
    int get (int*, double*);
};
```

Rešenje

```
const int N = ...; // Capacity of the buffer
```

```
class BoundedBuffer {
public:

    BoundedBuffer ();

    void put1 (int);
    void put2 (double);
    int get (int*, double*);

private:
    Semaphore mutex;
    Semaphore spaceAvailable1, itemAvailable1;
    Semaphore spaceAvailable2, itemAvailable2;

    int buffer1[N];
    int head1, tail1;

    double buffer2[N];
    int head2, tail2;
};
```

```
BoundedBuffer::BoundedBuffer () :
    mutex(1),
    spaceAvailable1(N), itemAvailable1(0),
    head1(0), tail1(0),
    spaceAvailable2(N), itemAvailable2(0),
    head2(0), tail2(0) {}
```

```
void BoundedBuffer::put1 (int d) {
    spaceAvailable1.wait();
    mutex.wait();
    buffer1[tail1] = d;
    tail1 = (tail1+1)%N;
    mutex.signal();
    itemAvailable1.signal();
}
```

```
// Similar for put2
```

```
int BoundedBuffer::get (int* pi, double* pd) {
    int s = Semaphore::waitOr(&itemAvailable1,&itemAvailable2);
    mutex.wait();
    if (s==1) {
        *pi = buffer1[head1];
        head1 = (head1+1)%N;
        mutex.signal();
        spaceAvailable1.signal();
    } else {
        *pd = buffer2[head2];
        head2 = (head2+1)%N;
        mutex.signal();
        spaceAvailable2.signal();
    }
    return s;
}
```

1. zadatak, treći kolokvijum, jun 2016.

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu izvršena je adaptacija interfejsa znakovno orijentisanog sekvencijalnog izlaznog uređaja na blokovski orijentisani sekvencijalni uređaj pomoću ograničenog bafera. Proizvođači, koji su uporedne niti, upisuju u bafer sekvencijalno bajt po bajt, operacijom `put`. Potrošači, takođe uporedne niti, uzimaju po ceo blok podataka veličine `BlockSize` iz bafera operacijom `read` i prenose ih dalje na izlazni uređaj.

Implementirati klasu `Buffer` čiji je interfejs dat dole. Sinhronizaciju vršiti pomoću semafora školskog jezgra, koji su, radi pogodnosti upotrebe za ovu namenu, prošireni operacijom `signal(unsigned)` koja atomično inkrementira vrednost semafora za vrednost zadatog argumenta (koji može biti proizvoljan prirodan broj, a ne samo jedan, kako je podrazumevano).

```
typedef unsigned short Byte;
const int BlockSize = ...;
const int NumOfBlocks = ...;
const int BufferSize = NumOfBlocks*BlockSize;

class Buffer {
public:
    Buffer ();
    void put (Byte b);
    void read (Byte block[]);
};
```

Rešenje

```
typedef unsigned short Byte;
const int BlockSize = ...;
const int NumOfBlocks = ...;
const int BufferSize = NumOfBlocks*BlockSize;
class Buffer {
public:
    Buffer ();
    void put (Byte b);
    void read (Byte block[]);
private:
    Byte buffer[BufferSize];
```



```

    int rdCursor, wrCursor;
    Semaphore mutex, spaceAvailable, itemAvailable;
};
Buffer::Buffer () : rdCursor(0), wrCursor(0),
    mutex(1), spaceAvailable(BufferSize), itemAvailable(0) {}
void Buffer::put (Byte b) {
    spaceAvailable.wait();
    mutex.wait();
    this->buffer[this->wrCursor] = b;
    this->wrCursor = (this->wrCursor+1)%BufferSize;
    bool toSignal = (this->wrCursor%BlockSize == 0);
    mutex.signal();
    if (toSignal) itemAvailable.signal();
}
void Buffer::read (Byte block[]) {
    itemAvailable.wait();
    mutex.wait();
    for (int i=0; i<BlockSize; i++)
        block[i] = this->buffer[this->rdCursor+i];
    this->rdCursor = (this->rdCursor+BlockSize)%BufferSize;
    mutex.signal();
    spaceAvailable.signal(BlockSize);
}

```

1. zadatak, drugi kolokvijum, septembar 2016.

- [Postavka](#)
- [Rešenje](#)

Školsko jezgro treba proširiti podrškom za slanje i prijem poruka između niti, implementacijom sledeće dve operacije klase Thread:

- `void Thread::send(char* message)`: pozivajuća nit šalje poruku datoj niti (`this`); ukoliko je ovoj niti već stigla neka poruka koju ona nije preuzela, pozivajuća nit se suspenduje dok se prethodna poruka ne preuzme i tek onda ostavlja poruku i nastavlja izvršavanje;
- `static char* Thread::receive()`: pozivajuća nit preuzima poruku koja joj je poslata; ukoliko poruke nema, pozivajuća nit se suspenduje dok poruka ne stigne.

Rešenje

U klasu Thread treba dodati sledeće članove:

```

char* Thread::message(0);
Semaphore msgEmpty(1), msgAvailable(0);
void Thread::send (char* msg) {
    this->msgEmpty.wait();
    this->message = msg;
    this->msgAvailable.signal();
}
char* Thread::receive () {
    Thread::running->msgAvailable.wait();
    char* msg = Thread::running->message;
    Thread::running->msgEmpty.signal();
    return msg;
}

```

1. zadatak, drugi kolokvijum, maj 2015.

- Postavka
- Rešenje

U nastavku je dat parcijalni opis neznatno izmenjenog (pojednostavljenog) POSIX API za semafore. Napisati delove koda dva procesa koji međusobno isključuju izvršavanja svojih kritičnih sekcija korišćenjem ovih semafora.

```
#include <fcntl.h>
/* For O_* constants */
#include <sys/stat.h>
sem_t *sem_open(const char *name, int oflag, unsigned int value);
#include <semaphore.h>
int sem_post(sem_t *sem);
int sem_wait(sem_t *sem);
int sem_close(sem_t *sem);
```

sem_open() creates a new POSIX semaphore or opens an existing semaphore. The semaphore is identified by name of the form */somenam*; that is, a null-terminated string of up to *NAME_MAX-4* (i.e., 251) characters consisting of an initial slash, followed by one or more characters, none of which are slashes. Two processes can operate on the same named semaphore by passing the same name to *sem_open()*.

The *oflag* argument specifies flags that control the operation of the call. (Definitions of the flags values can be obtained by including *<fcntl.h>*.) If *O_CREAT* is specified in *oflag*, then the semaphore is created if it does not already exist. If both *O_CREAT* and *O_EXCL* are specified in *oflag*, then an error is returned if a semaphore with the given name already exists.

If *O_CREAT* is specified in *oflag*, then the *value* argument must be specified. The *value* argument specifies the initial value for the new semaphore. If *O_CREAT* is specified, and a semaphore with the given name already exists, then *value* is ignored. After the semaphore has been opened, it can be operated on using *sem_post()* and *sem_wait()*. When a process has finished using the semaphore, it can use *sem_close()* to close the semaphore.

On success, *sem_open()* returns the address of the new semaphore; this address is used when calling other semaphore-related functions.

sem_post() increments (unlocks) the semaphore pointed to by *sem*. If the semaphore's value consequently becomes greater than zero, then another process or thread blocked in a *sem_wait()* call will be woken up and proceed to lock the semaphore.

sem_wait() decrements (locks) the semaphore pointed to by *sem*. If the semaphore's value is greater than zero, then the decrement proceeds, and the function returns, immediately. If the semaphore currently has the value zero, then the call blocks until either it becomes possible to perform the decrement (i.e., the semaphore value rises above zero).

Rešenje

Deo koda oba procesa izgleda isto:

```
#include <fcntl.h>
#include <sys/stat.h>
#include <semaphore.h>

// Initialization:
const char* mutexName = "/myprogram_mutex";
sem_t* mutex = sem_open(mutexName, O_CREAT, 1);

...
// Use for mutual exclusion
sem_wait(mutex);
// Critical section
sem_post(mutex);
...
```

```
// Release the semaphore when it is no longer needed:
sem_close(mutex);
```

1. zadatak, drugi kolokvijum, april 2013.

- [Postavka](#)
- [Rešenje](#)

U nastavku je dat kod uporednih procesa koji simuliraju policijski helikopter i automobil u poteri za beguncem, objašnjen na predavanjima. Korišćenjem standardnih brojačkih semafora obezbediti potrebnu sinhronizaciju koja eliminiše problem utrivanja (*race condition*) koji postoji u ovom kodu, objašnjen na predavanjima, ali tako da se tokom izvršavanja potprograma `moveTo` ne sprečava napredovanje procesa `Helicopter`.

```
type Coord = record {
  x : integer;
  y : integer;
};

var sharedCoord : Coord;

process Helicopter
var nextCoord : Coord;
begin
  loop
    computeNextCoord(nextCoord);
    sharedCoord := nextCoord;
  end;
end;

process PoliceCar
begin
  loop
    moveTo(sharedCoord);
  end;
end;
```

Rešenje

```
type Coord = record {
  x : integer;
  y : integer;
};

var sharedCoord : Coord;
    mutex : Semaphore = 1;

process Helicopter
var nextCoord : Coord;
begin
  loop
    computeNextCoord(nextCoord);
    mutex.wait();
    sharedCoord := nextCoord;
    mutex.signal();
  end;
end;

process PoliceCar
```

```

var nextCoord : Coord;
begin
    loop
        mutex.wait();
        nextCoord := sharedCoord;
        mutex.signal();
        moveTo(nextCoord);
    end;
end;

```

1. zadatak, drugi kolokvijum, septembar 2013.

- [Postavka](#)
- [Rešenje](#)

U nekom operativnom sistemu postoje sledeći sistemski pozivi koji se odnose na klasične brojačke semafore za sinhronizaciju između uporednih procesa:

- `sem_t* sem_open(const char* name, unsigned int value)`: Otvara semafor sa datim simboličkim imenom `name` za korišćenje u pozivajućem procesu, ukoliko je semafor sa tim imenom već kreiran (od strane istog ili nekog drugog procesa). Ukoliko nije, kreira takav semafor sa datom početnom vrednošću `value`. Različiti procesi tako mogu da dele isti semafor. Struktura `sem_t` predstavlja ručku do semafora (deskriptor). U slučaju greške, vraća 0.
- `int sem_wait(sem_t* sem)`: Operacija *wait* na datom semaforu. Vraća 0 u slučaju uspeha, -1 u slučaju greške.
- `int sem_post(sem_t* sem)`: Operacija *signal* na datom semaforu. Vraća 0 u slučaju uspeha, -1 u slučaju greške.
- `int sem_unlink(sem_t* sem)`: Oslobađa semafor od strane pozivajućeg procesa. Semafor se dealocira kada ga oslobode svi procesi koji ga koriste. Vraća 0 u slučaju uspeha, -1 u slučaju greške.

Sve navedene deklaracije nalaze se u `semaphore.h`.

Napisati program koji (u funkciji `main`) obezbeđuje međusobno isključenje kritične sekcije u odnosu na druge procese koji su na isti način konstruisani.

Rešenje

```

#include <semaphore.h>

void main () {
    sem_t* mutex = sem_open("mutex",1);
    if (mutex==0) return; // Exception!

    <noncritical_section>

    int status = sem_wait(mutex);
    if (status<0) ... // Exception!
    <critical_section>
    int status = sem_post(mutex);
    if (status<0) ... // Exception!

    <noncritical_section>
    sem_unlink(mutex);
}

```

1. zadatak, drugi kolokvijum, maj 2012.

- [Postavka](#)
- [Rešenje](#)

Dva uporedna kooperativna procesa A i B razmenjuju podatke tako što proces A izračunava vrednost deljene promenljive a pozivom svoje privatne procedure `compute_a`. Tu vrednost promenljive a koristi proces B tako što na osnovu nje izračunava vrednost deljene promenljive b pozivom svoje privatne procedure `compute_b`. Tu vrednost promenljive b potom koristi proces A za izračunavanje nove vrednosti promenljive a i tako ciklično. Implementirati ove procese korišćenjem uposlenog čekanja za uslovnu sinhronizaciju. Inicijalno, proces A može odmah da izračuna početnu vrednost za a na osnovu statički inicijalizovane vrednosti za b, dok proces B ne može da izračuna prvu vrednost b dok ne dobije prvu izračunatu vrednost a.

Rešenje

```
shared var
  a, b : integer := 0;
  sa, sb : boolean := 0;

process a;
begin
  loop
    compute_a(b);
    sa:=1;
    while (sb==0) do null; sb:=0;
  end;
end;

process b;
begin
  loop
    while (sa==0) do null; sa:=0;
    compute_b(a);
    sb:=1;
  end;
end;
```

1. zadatak, drugi kolokvijum, septembar 2012.

- [Postavka](#)
- [Rešenje](#)

Dva uporedna kooperativna procesa A i B treba da ulaze u kritičnu sekciju strogo naizmenično. Korišćenjem standardnih brojačkih semafora prikazati potrebnu sinhronizaciju.

Rešenje

```
shared var
  sa : Semaphore:=1,
  sb : Semaphore:=0;

process A;
begin
  loop
    wait(sa);
    <critical-section>
    signal(sb);
    <non-critical-section>
  end;
end;

process B;
begin
```

```

loop
  wait(sb);
  <critical-section>
  signal(sa);
  <non-critical-section>
end;
end;

```

1. zadatak, drugi kolokvijum, maj 2011.

- [Postavka](#)
- [Rešenje](#)

Dva uporedna kooperativna procesa A i B razmenjuju podatke tako što proces A izračunava vrednost deljene promenljive **a** pozivom svoje privatne procedure `compute_a`. Tu vrednost promenljive **a** koristi proces B tako što na osnovu nje izračunava vrednost deljene promenljive **b** pozivom svoje privatne procedure `compute_b`. Tu vrednost promenljive **b** potom koristi proces A za izračunavanje nove vrednosti promenljive **a** i tako ciklično. Korišćenjem standardnih brojačkih semafora napisati kod procesa A i B sa svim neophodnim deklaracijama. Inicijalno, proces A može odmah da izračuna početnu vrednost za **a** na osnovu statički inicijalizovane vrednosti za **b**, dok proces B ne može da izračuna prvu vrednost **b** dok ne dobije prvu izračunatu vrednost **a**.

Rešenje

```

shared var
  a, b : integer := 0;
  sa, sb : semaphore := 0;

process a;
begin
  loop
    compute_a(b);
    sa.signal();
    sb.wait();
  end;
end;

process b;
begin
  loop
    sa.wait();
    compute_b(a);
    sb.signal();
  end;
end;

```

1. zadatak, drugi kolokvijum, septembar 2011.

- [Postavka](#)
- [Rešenje](#)

Korišćenjem standardnih brojačkih semafora rešiti problem utrivanja (engl. *race condition*) u kodu datog programa koji simulira rad policijskog helikoptera i automobila, a koji je objašnjen na predavanjima.

```

type Coord = record {
  x : integer;
  y : integer;
};

var sharedCoord : Coord;

```

```

process Helicopter
var nextCoord : Coord;
begin
  loop
    computeNextCoord(nextCoord);
    sharedCoord := nextCoord;
  end;
end;

process PoliceCar
begin
  loop
    moveTo(sharedCoord);
  end;
end;

```

Rešenje

```

type Coord = record {
  x : integer;
  y : integer;
};

var sharedCoord : Coord;
    readyToWrite : Semaphore = 1;
    readyToRead : Semaphore = 0;

process Helicopter
var nextCoord : Coord;
begin
  loop
    computeNextCoord(nextCoord);
    readyToWrite.wait;
    sharedCoord := nextCoord;
    readyToRead.signal;
  end;
end;

process PoliceCar
var nextCoord : Coord;
begin
  loop
    readyToRead.wait;
    nextCoord := sharedCoord;
    readyToWrite.signal;
    moveTo(nextCoord);
  end;
end;

```

2. zadatak, drugi kolokvijum, maj 2010.

- [Postavka](#)
- [Rešenje](#)

Kreirano je više procesa istog tipa P koji imaju istu sledeću strukturu. U kritičnoj sekciji A nalaze se ugneždene dve kritične sekcije B i C , s tim da se sekcije B i C izvršavaju jedna posle druge (nisu ugneždene). Potrebno je obezbediti

sledeću sinhronizaciju: u kritičnoj sekciji A može se u jednom trenutku nalaziti najviše N ovih procesa tipa P , dok se u sekciji B , odnosno C može nalaziti najviše jedan proces. (Svaka od sekcija B i C je međusobno isključiva, ali se B i C ne isključuju međusobno – jedan proces može biti u B dok je drugi u C .) Korišćenjem standardnih brojačkih semafora obezbediti ovu sinhronizaciju: prikazati strukturu tipa procesa P , uz odgovarajuće definicije i inicijalizacije potrebnih semafora.

Rešenje

```
shared var mutexA : semaphore:=N;
        mutexB, mutexC : semaphore:=1;
```

```
type P = process begin
    ...
    wait(mutexA);
    <critical section A>
    ...
    wait(mutexB);
    <critical section B>
    signal(mutexB);
    ...
    wait(mutexC);
    <critical section C>
    signal(mutexC);
    ...
    signal(mutexA);
    ...
end;
```

2. zadatak, drugi kolokvijum, maj 2009.

- [Postavka](#)
- [Rešenje](#)

Dat je kod dva tipa procesa A i B:

```
shared var count : integer := 0;
        mutex : semaphore:=1;
        gate : semaphore:=1;
```

```
type A = process begin
    ...
    wait(mutex);
    count:=count+1;
    if (count=1) then wait(gate);
    signal(mutex);
    ... (* Critical section A *)
    wait(mutex);
    count:=count-1;
    if (count=0) then signal(gate);
    signal(mutex);
    ...
end;
```

```
type B = process begin
    ...
    wait(gate);
    ... (* Critical section B *)
    signal(gate);
```



```
...
end;
```

Ukoliko postoji proizvoljno, ali konačno mnogo aktivnih procesa tipa *A* i procesa tipa *B*, precizno odgovoriti i objasniti koliko *istovremeno* može biti procesa u označenim kritičnim sekcijama.

Rešenje

Ili proizvoljno mnogo procesa tipa *A* u svojim kritičnim sekcijama i ni jedan proces tipa *B* u svojoj, ili samo jedan proces tipa *B* i ni jedan proces tipa *A*.

Semafor `mutex` služi da obezbedi međusobno isključenje pristupa deljenoj promenljivoj `count` od strane uporednih procesa tipa *A*. Promenljiva `count` je brojač procesa tipa *A* koji su ušli u svoju kritičnu sekciju. Kada prvi ovakav proces ulazi u svoju kritičnu sekciju, izvršiće `wait` na semaforu `gate` i ili proći taj semafor (i zatvoriti ga), ili se blokirati na njemu. U prvom slučaju, svi naredni procesi tipa *A* slobodno ulaze u svoju kritičnu sekciju. U drugom slučaju, svi naredni procesi tipa *A* koji žele da uđu u svoju kritičnu sekciju će se blokirati na semaforu `mutex`. Poslednji proces tipa *A* koji izlazi iz svoje kritične sekcije izvršiće `signal` na semaforu `gate`. Kako samo jedan proces može proći operaciju `wait` na semaforu `gate` bez blokiranja, to znači da će ili prvi proces tipa *A* ili samo jedan proces tipa *B* to moći da uradi i tako uđe u svoju kritičnu sekciju. Odatle sledi dati odgovor.

2. zadatak, drugi kolokvijum, maj 2006.

- [Postavka](#)
- [Rešenje](#)

Korišćenjem koncepta spoljašnjeg događaja iz zadatka 1, realizovati telo niti koja ciklično izvršava sledeći posao:

- uzima jedan zahtev iz liste postavljenih zahteva za ulaznom operacijom prenosa bloka podataka sa nekog ulaznog uređaja u memoriju pomoću DMA kontrolera, odnosno čeka blokirana ako takvog zahteva nema
- pokreće operaciju prenosa bloka podataka na zadato mesto u memoriji pomoću DMA kontrolera
- čeka (suspendovana) da DMA kontroler završi prenos, što DMA kontroler signalizira prekidom na ulazu *N*
- kada se završi prenos, označava zahtev ispunjenim i signalizira semafor koji je zadat u zahtevu, kako bi se nit koja je postavila zahtev deblokirala.

Zahtev za ulaznom operacijom i red zahteva izgledaju ovako:

```
struct IORequest {
    IORequest* next; // Next in the request queue
    void* buffer;    // Input buffer in memory
    long size;       // Size of the data block
    int isCompleted; // Is this request completed?
    Semaphore* toSignal; // The semaphore to signal on completion
};

extern IORequest* requestQueue;
extern Semaphore mutex;
extern Semaphore requestQueueSize;
```

Za pristup do reda zahteva treba obezbediti međusobno isključenje pomoću klasičnog brojačkog semafora `mutex`. Brojački semafor `requestQueueSize` ima vrednost koja je jednaka broju zahteva u redu zahteva `requestQueue`. Prenosa bloka podataka sa DMA kontrolera pokreće se operacijom `void startDMA(void* buffer, long size)`.

Rešenje

```
ExternalEvent dmaEvent(N);
```

```
while (1) {
    requestQueueSize.wait();
    mutex.wait();
    IORequest* r = requestQueue;
    requestQueue = requestQueue->next;
```

```
mutex.signal();

startDMA(r->buffer, r->size);
dmaEvent.wait();

r ->isCompleted = 1;
r ->toSignal->signal();
}
```

Baferi, proizvođač/potrošač

3. zadatak, drugi kolokvijum, maj 2022.

- [Postavka](#)
- [Rešenje](#)

Više procesa proizvođača i potrošača međusobno razmenjuju znakove (`char`) preko ograničenog bafera tako što svi ti procesi dele jedan zajednički logički segment memorije veličine `sizeof(bbuffer)` koji su alocirali sistemskim pozivom `mmap` kao `bss` segment (inicijalizovan nulama pri alokaciji). Svaki od tih procesa adresu tog segmenta konvertuje u pokazivač na tip `struct bbuffer` i dalje radi operacije sa ograničenim baferom pozivajući sledeće operacije iz biblioteke `bbuf` čije su deklaracije (`bbuf.h`) date dole:

- `bbuf_init`: svaki proces koji želi da koristi bafer mora najpre da pozove ovu operaciju kako bi ona otvorila potrebne semafore; ukoliko neki od sistemskih poziva vezanih za semafore nije uspeo, sve one već otvorene semafore treba osloboditi i vratiti negativnu vrednost (greška); ukoliko je inicijalizacija uspešna, treba vratiti 0;
- `bbuf_close`: svaki proces koji koristi bafer treba da pozove ovu operaciju kada završi sa korišćenjem bafera;
- `bbuf_append`, `bbuf_take`: operacije stavljanja i uzimanja elementa (`char`) u bafer.

Sve ove operacije podrazumevaju da je argument ispravan pokazivač (ne treba ga proveravati). Sve operacije osim `bbuf_init` takođe podrazumevaju da je inicijalizacija pre toga uspešno završena – proces ih ne sme pozivati ako nije, pa ne treba proveravati ispravnost stanja bafera.

Implementirati biblioteku `bbuf` (`bbuf.cc`): dati definiciju strukture `bbuffer` i implementirati sve ove operacije korišćenjem POSIX semafora.

```
struct bbuffer;
int bbuf_init(struct bbuffer*);
void bbuf_append(struct bbuffer*, char);
char bbuf_take(struct bbuffer*);
void bbuf_close(struct bbuffer*);
```

Rešenje

```
#include <fcntl.h>
#include <sys/stat.h>
#include <semaphore.h>

const size_t BBSIZE = ...;
static sem_t *mutex, *space_available, *item_available;

struct bbuffer {
    char buf[BBSIZE];
    int head, tail;
};

int bbuf_init(struct bbuffer* bb) {
    mutex = sem_open("/bounded_buf_mx", O_CREAT, O_RDWR, 1);
    space_available = sem_open("/bounded_buf_sa", O_CREAT, O_RDWR, BBSIZE);
    item_available = sem_open("/bounded_buf_ia", O_CREAT, O_RDWR, 0);
    if (!mutex || !space_available || !item_available) {
        if (mutex) {
            sem_close(mutex);
        }
        if (space_available) {
            sem_close(space_available);
        }
        if (item_available) {
            sem_close(item_available);
        }
    }
}
```

```

        return -1;
    }
    return 0;
}

void bbuf_close(struct bbuffer* bb) {
    if (mutex) {
        sem_close(mutex);
    }
    if (space_available) {
        sem_close(space_available);
    }
    if (item_available) {
        sem_close(item_available);
    }
}

void bbuf_append(struct bbuffer* bb, char c) {
    sem_wait(space_available);
    sem_wait(mutex);
    bb->buf[bb->tail] = c;
    bb->tail = (bb->tail+1)%BBSIZE;
    sem_post(mutex);
    sem_post(item_available);
}

char bbuf_take(struct bbuffer* bb) {
    sem_wait(item_available);
    sem_wait(mutex);
    char c = bb->buf[bb->head];
    bb->head = (bb->head+1)%BBSIZE;
    sem_post(mutex);
    sem_post(space_available);
    return c;
}

```

3. zadatak, kolokvijum, jul 2020.

- [Postavka](#)
- [Rešenje](#)

Realizovati u potpunosti klasu `DoubleBuffer` čiji je interfejs dat. Ova klasa implementira dvostruki bafer. Proizvođač stavlja u „izlazni“ bafer blokove veličine `chunkSizeProd` znakova pozivom operacije `put()`; potrošač uzima iz „ulaznog“ bafera blokove veličine `chunkSizeCons` znakova pozivom operacije `get()`. Kada obojica završe sa svojim baferom, baferi zamenjuju uloge. Proizvođač i potrošač su uporedne niti (ne treba ih realizovati), dok je sva potrebna sinhronizacija unutar klase `DoubleBuffer`. Pretpostaviti da je zadata veličina oba bafera u znakovima (argument `size` konstruktora) celobrojan umnožak obe zadate veličine bloka (argumenti `chunkSize`). Za sinhronizaciju koristiti semafore školskog jezgra.

```

class DoubleBuffer {
public:
    DoubleBuffer (size_t size, size_t chunkSizeProd, size_t chunkSizeCons);
    void put (const char* buffer);
    void get (char* buffer);
private:
    ...
};

```

Rešenje

```

class DoubleBuffer {
public:
    DoubleBuffer (size_t size, size_t chunkSizeProd, size_t chunkSizeCons);
    void put (const char* buffer);
    void get (char* buffer);

private:
    Semaphore inputBufReady, outputBufReady;
    char* buffer[2];
    size_t size, chunkP, chunkC, head, tail, slots, items;
    int inputBuf, outputBuf;
};

DoubleBuffer::DoubleBuffer (size_t sz, size_t cp, size_t cc)
: inputBufReady(1), outputBufReady(0) {
    buffer[0] = new char[sz];
    buffer[1] = new char[sz];
    size = sz;
    chunkP = ((cp>0)?cp:1);
    chunkC = ((cc>0)?cc:1);
    head = tail = 0;
    slots = size; items = 0;
    inputBuf = 0; outputBuf = 1;
}

void DoubleBuffer::put (const char* buf) {
    if (slots==0) {
        inputBufReady.wait();
        outputBuf = !outputBuf;
        slots = size;
        tail = 0;
    }
    for (size_t i=0; i<chunkP; i++) {
        buffer[outputBuf][tail++] = buf[i++];
        slots--;
    }
    if (slots==0)
        outputBufReady.signal();
}

void DoubleBuffer::get (char* buf) {
    if (items==0) {
        outputBufReady.wait();
        inputBuf = !inputBuf;
        items = size;
        head = 0;
    }
    for (size_t i=0; i<chunkC; i++) {
        buf[i++] = buffer[inputBuf][head++];
        items--;
    }
    if (items==0)
        inputBufReady.signal();
}

```

1. zadatak, treći kolokvijum, septembar 2013.

- [Postavka](#)

- Rešenje

Realizovati u potpunosti klasu `DoubleBuffer` čiji je interfejs dat. Ova klasa implementira dvostruki bafer. Proizvođač stavlja u bafer blokove veličine `chunkSize` znakova pozivom operacije `put()`; znakovi se uzimaju iz bafera pozivaoca na koji ukazuje argument `buffer`, a stavljaju u trenutni „izlazni“ bafer od dva interna bafera veličine `size` znakova. Potrošač uzima znak po znak iz trenutnog „ulaznog“ bafera pozivom operacije `get()`. Kada obojica završe sa svojim baferom, baferi zamenjuju uloge. Proizvođač i potrošač su uporedne niti (ne treba ih realizovati), dok je sva potrebna sinhronizacija unutar klase `DoubleBuffer`. Pretpostaviti da je zadata veličina bafera u znakovima (argument `size` konstruktora) celobrojan umnožak zadate veličine bloka (argument `chunkSize`). Za sinhronizaciju koristiti semafore.

```
class DoubleBuffer {
public:
    DoubleBuffer (int size, int chunkSize);
    void put (char* buffer);
    char get ();
private:
    ...
};
```

Rešenje

```
class DoubleBuffer {
public:
    DoubleBuffer (int size, int chunkSize);
    void put (char* buffer);
    char get ();
private:
    Semaphore inputBufReady, outputBufReady;
    char* buffer[2];
    int size, chunk, head, tail, slots, items, inputBuf, outputBuf;
};
```

```
DoubleBuffer::DoubleBuffer (int sz, int cs)
: inputBufReady(1), outputBufReady(0) {
    buffer[0] = new char[sz];
    buffer[1] = new char[sz];
    size = sz;
    chunk = ((cs>0)?cs:1);
    head = tail = 0;
    slots = size; items = 0;
    inputBuf = 0; outputBuf = 1;
}
```

```
void DoubleBuffer::put (char* buf) {
    if (slots==0) {
        inputBufReady.wait();
        outputBuf = !outputBuf;
        slots = size;
        tail = 0;
    }
    for (int i=0; i<chunk; i++) {
        buffer[outputBuf][tail++] = buf[i++];
        slots--;
    }
    if (slots==0)
        outputBufReady.signal();
}
```

```

char DoubleBuffer::get () {
    if (items==0) {
        outputBufReady.wait();
        inputBuf = !inputBuf;
        items = size;
        head = 0;
    }
    char ret = buffer[inputBuf][head++];
    items--;
    if (items==0)
        inputBufReady.signal();
    return ret;
}

```

1. zadatak, treći kolokvijum, septembar 2012.

- [Postavka](#)
- [Rešenje](#)

Realizovati u potpunosti klasu `DoubleBuffer` čiji je interfejs dat. Ova klasa implementira dvostruki bafer. Proizvođač stavlja u bafer znak po znak pozivom operacije `put()`; znak se stavlja u trenutni „izlazni“ bafer od dva interna bafera veličine `size` znakova. Potrošač uzima blokove veličine `chunkSize` znakova iz trenutnog „ulaznog“ bafera pozivom operacije `get()`; znakovi se prepisuju u bafer pozivaoca na koji ukazuje argument `buffer`. Kada obojica završe sa svojim baferom, baferi zamenjuju uloge. Proizvođač i potrošač su uporedne niti (ne treba ih realizovati), dok je sva potrebna sinhronizacija unutar klase `DoubleBuffer`. Pretpostaviti da je zadata veličina bafera u znakovima (argument `size` konstruktora) celobrojan umnožak zadate veličine bloka (argument `chunkSize`). Za sinhronizaciju koristiti semafore.

```

class DoubleBuffer {
public:
    DoubleBuffer (int size, int chunkSize);
    void put (char);
    void get (char* buffer);
private:
    ...
};

```

Rešenje

```

class DoubleBuffer {
public:
    DoubleBuffer (int size, int chunkSize);
    void put (char);
    void get (char* buffer);
private:
    Semaphore inputBufReady, outputBufReady;
    char* buffer[2];
    int size, chunk, head, tail, slots, items, inputBuf, outputBuf;
};

```

```

DoubleBuffer::DoubleBuffer (int sz, int cs)
: inputBufReady(1), outputBufReady(0) {
    buffer[0] = new char[sz];
    buffer[1] = new char[sz];
    size = sz;
    chunk = ((cs>0)?cs:1);
    head=tail=0;
    slots=size; items=0;
}

```

```

    inputBuf=0; outputBuf=1;
}

void DoubleBuffer::put (char c) {
    if (slots==0) {
        inputBufReady.wait();
        outputBuf=!outputBuf;
        slots=size;
        tail=0;
    }
    buffer[outputBuf][tail++]=c;
    slots--;
    if (slots==0)
        outputBufReady.signal();
}

void DoubleBuffer::get (char* buf) {
    if (items==0) {
        outputBufReady.wait();
        inputBuf=!inputBuf;
        items=size;
        head=0;
    }
    for (int i=0; i<chunk; i++) {
        buf[i++] = buffer[inputBuf][head++];
        items--;
    }
    if (items==0)
        inputBufReady.signal();
}

```

1. zadatak, treći kolokvijum, jun 2011.

- [Postavka](#)
- [Rešenje](#)

Realizovati u potpunosti klasu `DoubleBuffer` čiji je interfejs dat. Ova klasa implementira dvostruki bafer. Proizvođač stavlja element tipa `Data*` pozivom operacije `put()`, čime se element stavlja u trenutni „izlazni“ bafer od dva interna bafera, dok potrošač uzima elemente iz trenutnog „ulaznog“ bafera pozivom operacije `get()`. Kada obojica završe sa svojim baferom, baferi zamenjuju uloge. Proizvođač i potrošač su uporedne niti (ne treba ih realizovati), dok je sva potrebna sinhronizacija unutar klase `DoubleBuffer`. Za sinhronizaciju koristiti semafore.

```

class Data;

class DoubleBuffer {
public:
    DoubleBuffer (int size);
    void put (Data*);
    Data* get ();
private:
    ...
};

```

Rešenje

```

class Data;

class DoubleBuffer {

```



```

public:
    DoubleBuffer (int size);
    void put (Data*);
    Data* get ();
private:
    Semaphore inputBufReady, outputBufReady;
    Data** buffer[2];
    int size, head, tail, slots, items, inputBuf, outputBuf;
};

DoubleBuffer::DoubleBuffer (int sz) : inputBufReady(1), outputBufReady(0) {
    buffer[0] = new (Data*)[sz];
    buffer[1] = new (Data*)[sz];
    size = sz;
    head=tail=0;
    slots=size; items=0;
    inputBuf=0; outputBuf=1;
}

void DoubleBuffer::put (Data* d) {
    if (slots==0) {
        inputBufReady.wait();
        outputBuf=!outputBuf;
        slots=size;
        tail=0;
    }
    buffer[outputBuf][tail++]=d;
    slots--;
    if (slots==0)
        outputBufReady.signal();
}

Data* DoubleBuffer::get () {
    if (items==0) {
        outputBufReady.wait();
        inputBuf=!inputBuf;
        items=size;
        head=0;
    }
    Data* d = buffer[inputBuf][head++];
    items--;
    if (items==0)
        inputBufReady.signal();
    return d;
}

```

2. zadatak, drugi kolokvijum, maj 2008.

- [Postavka](#)
- [Rešenje](#)

Jedan proces-proizvođač proizvodi elemente i stavlja ih u ograničeni bafer, dok dva procesa- potrošača uzimaju elemente iz tog bafera. Potrebno je obezbediti odgovarajuću sinhronizaciju između ovih procesa tako da potrošači naizmenično uzimaju po jedan element iz bafera. Realizovati ograničeni bafer koji obezbeđuje ovu sinhronizaciju pomoću klasičnih brojačkih semafora.

Rešenje

```

class Data;
const int N = ...;

class BoundedBuffer {
public:
    BoundedBuffer();
    void put (Data*);
    Data* get (int consumerID); // consumerID should be 1 or 2
private:
    Data* buf[N];
    int head, tail;
    Semaphore mutex, spaceAvailable, itemAvailable, gate1, gate2;
};

BoundedBuffer::BoundedBuffer () :
    head(0), tail(0),
    mutex(1), spaceAvailable(N), itemAvailable(0),
    gate1(1), gate2(0) {}

void BoundedBuffer::put (Data* d) {
    spaceAvailable.wait();
    mutex.wait();
    buf[tail]=d;
    tail=(tail+1)%N;
    mutex.signal();
    itemAvailable.signal();
}

Data* BoundedBuffer::get (int myID) {
    if (myID==1)
        gate1.wait();
    else if (myID==2)
        gate2.wait();
    else return 0; // Error

    itemAvailable.wait();
    mutex.wait();
    Data* d = buf[head];
    head=(head+1)%N;
    mutex.signal();
    spaceAvailable.signal();
    if (myID==1)
        gate2.signal();
    else if (myID==2)
        gate1.signal();
    return d;
}

```

Prikazano rešenje je napisano uz pretpostavku da se jednoj niti daje prednost, tj. unapred je određena nit koja mora prva da uzme podatak iz bafera. To rešenje je u potpunosti ispravno i prihvatljivo, ali je moguće dati i za nijansu bolje rešenje. U nastavku je prikazano rešenje koje ne favorizuje nijednu nit, tj. ona nit koja prva zatraži podatak, dobija ga, a dalje se uzimanje podataka iz bafera obavlja naizmenično. Izmene u odnosu na prethodno rešenje su posebno naznačene.

```

class Data;

```

```

const int N = ...;

class BoundedBuffer
{
public:
    BoundedBuffer();
    void put (Data*);
    Data* get (int consumerID); // consumerID should be 1 or 2
private:
    Data* buf[N];
    int head, tail, **firstTime**; // IZMENA
    Semaphore mutex, spaceAvailable, itemAvailable, gate1, gate2;
};

BoundedBuffer::BoundedBuffer () :
    head(0), tail(0), **firstTime (1)**, // IZMENA
    mutex(1), spaceAvailable(N), itemAvailable(0),
    gate1(0), gate2(0) {}

void BoundedBuffer::put (Data* d) {
    spaceAvailable.wait();
    mutex.wait();
    buf[tail]=d;
    tail=(tail+1)%N;
    mutex.signal();
    itemAvailable.signal();
}

Data* BoundedBuffer::get (int myID) {
    // IZMENA:
    if (firstTime) { //dobro zbog brzine, da se izbegne nepotrebna
                     // sinhronizacija posle prvog puta
        mutex.wait();
        if (firstTime) { //iako je vec provereno, mora se ponovo proveriti
                        // nedeljivo sa promenom
            if (myID==1)
                gate1.signal();
            else if (myID==2)
                gate2.signal();
            firstTime = 0;
        }
        mutex.signal()
    } //firstTime
    // KRAJ IZMENE

    if (myID==1)
        gate1.wait();
    else if (myID==2)
        gate2.wait();
    else return 0; // Error
    itemAvailable.wait();
    mutex.wait();
    Data* d = buf[head];
    head=(head+1)%N;
    mutex.signal();
    spaceAvailable.signal();
}

```

```

    if (myID==1)
        gate2.signal();
    else if (myID==2)
        gate1.signal();
    return d;
}

```

2. zadatak, drugi kolokvijum, maj 2007.

- [Postavka](#)
- [Rešenje](#)

Dat je sledeći interfejs klase koja realizuje ograničeni bafer (*bounded buffer*) kao posrednik između samo jednog proizvođača (*producer*) i samo jednog potrošača (*consumer*) koji su uporedne niti:

```

class BoundedBuffer {
public:
    BoundedBuffer(int capacity);
    void put (char* package, int size);
    void get (char* package, int size);
};

```

U bafer se smeštaju znakovi (`char`). Operacija `put` smešta u bafer dati niz znakova date dužine. Operacija `get` iz bafera uzima `size` znakova i smešta ih u niz na koji ukazuje pokazivač `package`. Ovi nizovi (paketi koji se prenose) su proizvoljne i promenljive dužine (ali svakako manje od kapaciteta bafera). Implementirati ovu klasu korišćenjem standardnih brojačkih semafora.

Rešenje

```

class BoundedBuffer {
public:
    BoundedBuffer(int capacity);
    void put (char* package, int size);
    void get (char* package, int size);
private:
    char* buf;
    int capacity, head, tail;
    Semaphore spaceAvailable, itemAvailable;
};

BoundedBuffer::BoundedBuffer (int cap) :
    buf(new char[cap]), capacity(cap), head(0), tail(0),
    spaceAvailable(cap), itemAvailable(0) {}

void BoundedBuffer::put (char* p, int n) {
    for (int i=0; i<n; i++) {
        spaceAvailable.wait();
        buf[tail]=p[i];
        tail=(tail+1)%capacity;
        itemAvailable.signal();
    }
}

```

Operacija `get` analogno.